



Ma223 : Analyse numérique matricielle

Fascicule de cours avec applications numériques sous Python

G. Couffignal

Version préliminaire du 12/04/2026

Table des matières

I. Autour de la décomposition en valeurs singulières	5
1. Décomposition en valeurs singulières	6
1.1. La structure euclidienne de $\mathcal{M}_{m,n}(\mathbb{R})$	6
1.1.1. La norme de Frobenius	7
1.2. La décomposition en valeurs singulières	9
1.2.1. En décortiquant la formule $Tr(A^T A)$	9
1.2.2. La SVD enfin...	12
1.2.3. Les « atomes » de l'algèbre linéaire	14
1.3. Une heuristique particulière : les valeurs singulières comme une mesure de l'information	15
1.4. Synthèse courte du chapitre	17
2. Le théorème d'Eckart-Young-Mirsky et ses conséquences	18
2.1. Le problème de l'approximation de Schmidt	18
2.1.1. Quelques mots sur l'interprétation géométrique	20
2.2. Un exemple graphique : la compression d'image	21
2.3. L'analyse en composantes principales (version algébrique)	23
2.3.1. Quelques considérations générales	24
2.3.2. Une SVD pour résoudre l'ACP	26
2.3.3. Cas d'un individus en plus	28
2.3.4. Un exemple d'ACP pour comprendre	29
3. L'exemple de l'analyse procustéenne linéaire	33
3.1. L'analyse procustéenne linéaire	33
3.1.1. Résolution théorique de l'analyse procustéenne	33
3.1.1.1. Pour bien débiter...	33
3.1.1.2. Le cas d'une transformation orthogonale	34
3.1.1.3. Le cas d'une transformation affine	35
3.1.2. Exemples d'applications de l'analyse procustéenne avec Python	38
4. Les moindres carrés	41
4.1. Moindres carrés « classique » avec SVD	41
4.1.1. Moindres carrés totaux	45
4.2. Un exemple d'apprentissage supervisé avec les moindres carrés	48
4.2.1. Plusieurs moindres carrés en parallèles	49
4.2.2. Formalisation d'un problème d'apprentissage supervisé	50
4.2.3. Application à l'exemple de la reconnaissance de chiffres manuscrits	51
II. Décompositions biconjugués (ouverture)	55
III. Annexes	56
5. Rappels sur les éléments de base du calcul matriciel	57
5.1. Quelques notions générales	57
5.2. Manipulation des matrices par blocs	61

5.3. Le cas particulier des matrices carrées	62
5.3.1. Valeurs propres et vecteurs propres	64
5.4. Lien entre matrice et application linéaire	67
6. Formes bilinéaires et formes quadratiques	69
6.1. Définitions générales et propriétés de bases	69
7. Produit de Kronecker	71
8. Un peu de python	75
8.1. Les commandes de base	75
8.2. Quelques rappels sur les matrices (carrées principalement)	78
8.3. Les matrices et les images avec Opencv	79
8.3.1. Produire une sortie vidéo avec OpenCV	80
8.3.2. Utilisation de l'apprentissage profond	81
Bibliographie	86

Avertissement

Ces notes de cours, rédigées pour la seconde année de l'IPSA, ne doivent pas être considérées comme un cours exhaustif d'analyse numérique matricielle. Une véritable introduction à ce domaine nécessiterait une maîtrise approfondie de concepts mathématiques qui ne sont pas abordés durant les premières années de formation à l'IPSA.

Ainsi vous trouverez dans ces notes :

- **des formulations simplifiées et moins générales** : de nombreux résultats seront présentés dans des formes moins générales que celles possibles dans un cadre formel plus abstrait.
- **certaines démonstrations seront incomplètes ou absentes** : certaines démonstrations seront volontairement incomplètes ou omises pour privilégier la clarté et progresser plus rapidement vers le versant pratique des méthodes vues.
- **un focus privilégié sur les exemples** : le nombre restreint d'heures d'enseignement a orienté ces notes vers une mise en œuvre rapide d'exemples pratiques et numériques (sous Python). Les codes complets seront principalement vus en séance de travaux dirigés (TD).

Ces notes ont été conçues avec un esprit d'aller rapidement vers des applications concrètes des notions mathématiques vues à l'IPSA. Elles ne visent pas à introduire de nouvelles théories mathématiques, mais à exploiter les outils déjà étudiés dans des contextes appliqués. Par conséquent :

- très peu de nouvelles notions mathématiques seront introduites.
- les méthodes abordées seront principalement des outils courants et classiques, largement utilisés en pratique, mais seront vus de sorte à ne pas nécessiter de bagages mathématiques supplémentaires importants.

Enfin, pour des formulations plus précises et un approfondissement des notions présentées ici, nous renvoyons les lecteurs aux références bibliographiques situées à la fin de ces notes.

Première partie

Autour de la décomposition en valeurs singulières

Décomposition en valeurs singulières

Dans ce cours nous traiterons directement des matrices réelles générales i.e. non nécessairement carrées. Soient m, n deux entiers naturels strictement positifs, on notera :

$$\mathcal{M}_{m,n}(\mathbb{R})$$

les matrices à coefficient réels de m lignes et n colonnes. Par convention un vecteur (colonne) de taille m sera vu comme une matrice de taille $(m, 1)$ i.e. comme un élément de $\mathcal{M}_{m,1}(\mathbb{R})$.

On rappelle que l'ensemble $\mathcal{M}_{m,n}(\mathbb{R})$ des matrices de taille (m, n) est muni d'une structure de $\mathbb{R}$ -espace vectoriel de dimension mn .

1.1. La structure euclidienne de $\mathcal{M}_{m,n}(\mathbb{R})$

Une matrice $A \in \mathcal{M}_{m,n}(\mathbb{R})$ peut servir à modéliser différent objet comme (la liste n'est pas exhaustive) :

- si l'on fixe une base de $\mathbb{R}^n$ et une base de $\mathbb{R}^m$, une matrice est associée à une application linéaire de $\mathbb{R}^n$ dans $\mathbb{R}^m$,
- un tableau de données numériques de m lignes et n colonnes,
- une image (en noir et blanc). Les images numériques sont des tableaux numériques dont les coefficients représentent l'intensité de gris (ou de couleurs).
- des sons (numériques).
- etc...

Dans chacun de ces cas une question naturelle que les scientifiques se posent est : comment comparer deux objets ? Par exemple comment donner un sens au fait de dire si ou non deux images (de même taille) sont « proches » ou au contraire « très différentes » l'une de l'autre¹.

Une notion mathématique importante permettant de réaliser cela est celle de distance et dans le cas particulier qui nous intéresse ici de $\mathcal{M}_{m,n}(\mathbb{R})$ muni de sa structure d'espace vectoriel, c'est la notion de norme qui va nous intéresser.

Rappelons qu'une norme sur l'espace vectoriel $\mathcal{M}_{m,n}(\mathbb{R})$ est la donnée d'une application $\|\bullet\|$ de $\mathcal{M}_{m,n}(\mathbb{R})$ dans $\mathbb{R}^+$ vérifiant les axiomes suivants :

- **séparation** : pour tout $A \in \mathcal{M}_{m,n}(\mathbb{R})$ tel que $\|A\| = 0$ alors $A = 0_{m,n}$.
- **homogénéité** : pour tout $(\lambda, A) \in \mathbb{R} \times \mathcal{M}_{m,n}(\mathbb{R})$ on a : $\|\lambda A\| = |\lambda| \|A\|$,
- **inégalité triangulaire (sous-additivité)** : pour tout $(A, B) \in \mathcal{M}_{m,n}(\mathbb{R})^2$, $\|A + B\| \leq \|A\| + \|B\|$

Lorsque l'on a à disposition une norme on peut grâce à la structure vectorielle définir une distance entre deux objets A et B en posant :

$$d(A, B) := \|A - B\|$$

C'est ce que l'on appelle la distance induite par la norme $\|\bullet\|$.

Ainsi se donner une norme c'est se donner un « critère » formel pour comparer des objets « abstraits ». D'accord mais quelle norme choisir sur $\mathcal{M}_{m,n}(\mathbb{R})$?

1. Les différents formalismes mathématiques (= théorie formelle) forment les langages (au sens d'une syntaxe) des sciences. En tant que « langage » une théorie formelle doit posséder (en autres choses) deux attributs :

- une théorie doit être « non-catégorique » i.e que ces énoncés peuvent s'appliquer à des situations différentes autrement dit sa syntaxe admet un spectre sémantique vaste. Par exemple le second principe de Newton, dans sa version formelle, s'applique aussi bien à la chute d'une pomme qu'au orbite des corps célestes. Ce seul énoncé $F = ma$, englobe une large diversité de situation.
- et doit posséder une certaine « expressivité » : les théories formelles que nous développons doivent pouvoir « dire » des choses fines et avec le moins de point aveugle possible.

1.1.1. La norme de Frobenius

Lorsque je veux comparer deux images ou deux tableaux une idée naturelle est de comparer pixel par pixel ou case par case et de considérer la somme de tous les écarts. Or si l'on souhaite que ces écarts qui peuvent être positifs ou négatifs ne puissent s'annihiler une autre idée naturelle est de considérer la somme au carré de ces écarts. C'est ce que fait la norme euclidienne sur $\mathbb{R}^n$ et c'est ce que nous allons généraliser sur $\mathcal{M}_{m,n}(\mathbb{R})$.

Définition 1.1 :

On définit l'application : $\|\bullet\|_F : \mathcal{M}_{m,n}(\mathbb{R}) \rightarrow \mathbb{R}_+$ défini par :

$$\|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n a_{ij}^2}$$

où a_{ij} note le coefficient i, j de A . Cette application s'appelle la **norme de Frobenius** de A .

On a alors la propriété suivante.

Proposition 1.1 :

La norme de Frobenius est une norme.

Démonstration. Démonstration en TD. □

On a ainsi une distance (induite par cette norme) pour comparer deux matrices $A, B \in \mathcal{M}_{m,n}(\mathbb{R})$:

$$d_F(A, B) := \|A - B\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n (a_{ij} - b_{ij})^2}$$

Cette distance permet ainsi de quantifier à quel point deux matrices A et B sont proches ou loin l'une de l'autre. Ainsi nous allons nous intéresser au fait de regarder les matrices $\mathcal{M}_{m,n}(\mathbb{R})$ par le « spectre » / « filtre » de la norme de Frobenius. Or ce choix comme tout choix a des conséquences : quelles structures pouvons nous ou non voir sur l'espace vectoriel $\mathcal{M}_{m,n}(\mathbb{R})$ lorsque l'on choisit de mettre les « lunettes » de la norme de Frobenius ? Pour le dire formellement quelle est la géométrie de $\mathcal{M}_{m,n}(\mathbb{R})$ muni de $\|\bullet\|_F$? Pour le dire rapidement, une géométrie est l'étude des invariants (on parle de symétrie) par rapport à certains critères (fixés à l'avance). Par exemple la géométrie euclidienne est complètement caractérisée par les transformations qui ne modifient pas les longueurs des objets « transformées ». Je peux prendre un triangle, le faire tourner et le translater, cela ne modifiera pas ce triangle du point de vue de la géométrie euclidienne. Par contre imaginons que nous dessinons un triangle sur une feuille « tordue » (i.e une surface courbe) on ne peut le translater sans le « tordre » si l'on veut rester sur notre feuille tordue comme dans le cas euclidien.

La question est donc quelle est la géométrie vue par les « lunettes » données par la norme de Frobenius ? La propriété suivante répond à cette question. On rappelle la définition des matrices orthogonales. Ces matrices sont les grandes amies des apprentis physiciens, elles incarnent les changements de repère orthonormé mais également des « numériques » en herbe pour leurs propriétés « pratiques/effectives ». Une matrice carrée $U \in \mathcal{M}_n(\mathbb{R})$ est dite **orthogonale** si elle est inversible et si son inverse est sa transposée U^T i.e :

$$U^T U = I_n \quad \text{et} \quad U U^T = I_n$$

On note $\mathbb{O}_n$ l'ensemble des matrices orthogonales de taille n . De la définition découle directement du fait que les vecteurs colonnes (ou lignes) d'une matrice orthogonale sont des vecteurs orthogonaux entre eux et de norme 1. Notons u_i les vecteurs colonnes d'une matrice orthogonale $U \in \mathbb{O}_n$. On a alors :

$$u_i^T u_j = \begin{cases} 0 & \text{si } i \neq j \\ 1 & \text{si } i = j \end{cases}$$

Rappelons également la définition de la trace d'une matrice carrée, notée ici Tr , comme étant la somme de ces éléments diagonaux.

Proposition 1.2 :

L'application :

$$\begin{aligned} \langle \bullet, \bullet \rangle_F : \mathcal{M}_{m,n}(\mathbb{R}) \times \mathcal{M}_{m,n}(\mathbb{R}) &\rightarrow \mathbb{R} \\ (A, B) &\mapsto \langle A, B \rangle_F = \text{Tr}(A^T B) \end{aligned}$$

est un produit scalaire sur $\mathcal{M}_{m,n}(\mathbb{R})$ et sa norme induite est exactement la norme de Frobenius i.e. :

$$\|A\|_F = \sqrt{\langle A, A \rangle_F} = \sqrt{\text{Tr}(A^T A)}$$

En particulier pour toutes matrices $A \in \mathcal{M}_{m,n}(\mathbb{R})$ et matrices orthogonales $U \in \mathbb{O}_m$ et $V \in \mathbb{O}_n$ on a :

$$\|UAV\|_F = \|A\|_F$$

Démonstration. Démonstration en TD. □

Remarque 1. Dans le cas particulier où l'on regarde des vecteurs de $\mathbb{R}^m$ comme des matrices $\mathcal{M}_{m,1}(\mathbb{R})$ ou $\mathcal{M}_{1,m}(\mathbb{R})$, la norme de Frobenius est directement la norme euclidienne. En effet si $v \in \mathcal{M}_{m,1}(\mathbb{R})$:

$$\|v\|_F = \sqrt{\sum_{i=1}^m (v_i)^2} = \|v\|_2$$

et le produit scalaire donnée par la trace est le produit scalaire euclidien usuel :

$$\langle v, w \rangle_F = v^T w = \langle v, w \rangle_2$$

car $v^T w$ est un scalaire donc une matrice $(1, 1)$ et par suite est égal à sa trace. On retrouve le fait que la géométrie euclidienne est invariante aux changements de repères orthonormés i.e par transformation données par une matrice orthogonale.

La formule précédente est riche de conséquence. La norme de Frobenius étant donnée par un produit scalaire nous pouvons généraliser tout ce que nous connaissons de la géométrie euclidienne classique à l'espace des matrices $\mathcal{M}_{m,n}(\mathbb{R})$. Par exemple :

— On peut définir un angle $\theta(A, B)$ entre deux matrices $A, B \in \mathcal{M}_{m,n}(\mathbb{R})$ par :

$$\theta(A, B) := \arccos \left(\frac{\langle A, B \rangle_F}{\|A\|_F \|B\|_F} \right)$$

lorsque $\|A\|_F \neq 0, \|B\|_F \neq 0$. En particulier nous dirons que A et B sont **orthogonales** si $\langle A, B \rangle_F = 0$.

— On retrouve les formules connues comme la généralisation du théorème de Pythagore :

$$\|A + B\|_F^2 = \|A\|_F^2 + \|B\|_F^2 + 2 \langle A, B \rangle_F$$

et en particulier l'inégalité de Cauchy-Schwarz :

$$|\langle A, B \rangle_F| \leq \|A\|_F \|B\|_F$$

avec l'égalité si et seulement si A et B sont colinéaires.

Finissons enfin cette section par un résultat pratique pour étudier la stabilité numérique des calculs effectués utilisant la norme de Frobenius.

Proposition 1.3 : Sous-multiplicativité de la norme

Soient $A \in \mathcal{M}_{m,n}(\mathbb{R})$ et $B \in \mathcal{M}_{n,p}(\mathbb{R})$, on a l'inégalité :

$$\|AB\|_F \leq \|A\|_F \|B\|_F$$

Démonstration. Soient $A \in \mathcal{M}_{m,n}(\mathbb{R})$ et $B \in \mathcal{M}_{n,p}(\mathbb{R})$, on a par définition :

$$\|AB\|_F^2 = \sum_{i=1}^m \sum_{j=1}^p (AB)_{ij}^2 = \sum_{i=1}^m \sum_{j=1}^p \left(\sum_{k=1}^n a_{ik} b_{kj} \right)^2$$

Pour toute paire d'indice (i, j) on applique l'inégalité de Cauchy-Schwartz pour la norme euclidienne i.e :

$$\left(\sum_{k=1}^n a_{ik} b_{kj}\right)^2 = \langle a_{i\bullet}, b_{\bullet j} \rangle^2 \leq \|a_{i\bullet}\|_2^2 \|b_{\bullet j}\|_2^2 = \left(\sum_{k=1}^n a_{ik}^2\right) \left(\sum_{k=1}^n b_{kj}^2\right)$$

avec : $a_{i\bullet}$ le vecteur ligne i de A et $b_{\bullet j}$ le vecteur colonne j de B . Ainsi en injectant cela dans la première formule :

$$\begin{aligned} \|AB\|_F^2 &= \sum_{i=1}^m \sum_{j=1}^p \left(\sum_{k=1}^n a_{ik} b_{kj}\right)^2 \\ &\leq \sum_{i=1}^m \sum_{j=1}^p \left(\sum_{k=1}^n a_{ik}^2\right) \left(\sum_{k=1}^n b_{kj}^2\right) \\ &= \left(\sum_{i=1}^m \sum_{k=1}^n a_{ik}^2\right) \left(\sum_{j=1}^p \sum_{k=1}^n b_{kj}^2\right) \\ &= \|A\|_F \|B\|_F \end{aligned}$$

□

Pour finir cette section contemplons la formule donnant la norme de Frobenius :

$$\|A\|_F = \sqrt{\text{Tr}(A^T A)}$$

Nous savons que la trace d'une matrice carrée est égale à la somme de ces valeurs propres. Comprendre ce que « voit » la norme de Frobenius c'est donc comprendre l'information que porte les matrices carrées de la forme $A^T A$ pour $A \in \mathcal{M}_{m,n}(\mathbb{R})$. C'est ce que nous allons faire dans la section suivante.

1.2. La décomposition en valeurs singulières

Dans cette section nous allons voir une décomposition matricielle extrêmement importante : la **décomposition en valeurs singulières** (singular decomposition values que nous résumerons en SVD). Cette décomposition fera sens pour toutes les matrices (et pas uniquement les matrices carrées). Nous allons commencer par démontrer un théorème sur l'existence de la décomposition en valeurs singulières pour toutes matrices même si elle n'est pas unique en général. C'est en particulier cette généralité additionnée au fait qu'elle fait intervenir des matrices orthogonales (des matrices qui se comportent « bien » avec les normes euclidiennes) qui font de la décomposition en valeurs singulières un outil très puissant pour nombre de problème appliquée.

1.2.1. En décortiquant la formule $\text{Tr}(A^T A)$

Avant d'énoncer le théorème principal reprenons le fil de notre discussion : l'étude de la matrice $A^T A$ ou ce qui revient au même de la matrice AA^T et de leurs traces.

Commençons par rappeler quelques propriétés importantes de l'opérateur de trace. La trace est un opérateur uniquement définie pour des matrices carrés et plus précisément c'est une forme linéaire i.e une application linéaire à valeur dans $\mathbb{R}$:

$$\text{Tr} : \mathcal{M}_n(\mathbb{R}) \rightarrow \mathbb{R}$$

Ce qui fait la richesse des matrices (carrées dans notre cas) par rapport aux réels par exemple est la non-commutativité de son produit i.e

$$AB \neq BA$$

en général pour $A, B \in \mathcal{M}_n(\mathbb{R})$. Or la trace à un rôle structurel important dans la théorie des matrices : c'est l'unique (à un facteur scalaire près) forme linéaire qui est aveugle à la non-commutativité du produit matriciel ou pour le dire de manière « positive » c'est l'unique mesure du « commun » entre AB et BA . Rendons cet énoncé précis.

Proposition 1.4 : Caractérisation de la trace

Soit $f : \mathcal{M}_n(\mathbb{R}) \rightarrow \mathbb{R}$ une forme linéaire sur les matrices carrées de taille n telle que pour toutes matrices $A, B \in \mathcal{M}_n(\mathbb{R})$:

$$f(AB - BA) = 0$$

i.e par linéarité $f(AB) = f(BA)$. Alors il existe un unique scalaire $\lambda \in \mathbb{R}$ tel que :

$$f = \lambda \text{Tr}$$

En particulier si l'on impose de plus que $f(I_n) = n$, la seule forme linéaire possible est $f = \text{Tr}$.

Démonstration. Comme f est une forme linéaire il suffit de regarder ses valeurs pour une base de donnée. L'espace vectoriel des matrices carrées de taille $\mathcal{M}_n(\mathbb{R})$ admet pour base les matrices $(E_{i,j})_{1 \leq i,j \leq n}$ dont les coefficients sont tous nuls sauf en i, j où $(E_{i,j})_{i,j} = 1$. Rappelons que le produit matriciel donne la relation suivante :

$$E_{i,j}E_{k,l} = \delta_{j,k}E_{i,l}$$

avec $\delta_{j,k}$ le symbole de Kronecker (qui 1 si $j = k$ et 0 sinon). Fixons $i \neq j$ et regardons les produits :

$$E_{i,i}E_{i,j} = E_{i,j} \quad \text{et} \quad E_{i,j}E_{i,i} = 0_n$$

Alors par hypothèse sur f on a :

$$f(E_{i,j}) = f(E_{i,i}E_{i,j}) = f(E_{i,j}E_{i,i}) = f(0_n) = 0$$

par linéarité. D'où f s'annule sur toutes les matrices élémentaires non « diagonales ». Regardons à présent les termes diagonaux. Soient $i, j \in \llbracket 1, n \rrbracket$ quelconques, on a :

$$f(E_{i,i}) = f(E_{i,j}E_{j,i}) = f(E_{j,i}E_{i,j}) = f(E_{j,j})$$

Ainsi tous les éléments diagonaux ont même image par f . Notons $\lambda = f(E_{1,1}) = \dots = f(E_{n,n})$. Soit $M \in \mathcal{M}_n(\mathbb{R})$, alors M s'écrit dans la base précédente sous la forme :

$$M = \sum_{i=1}^n \sum_{j=1}^n m_{i,j} E_{i,j}$$

et par linéarité de M on a par les calculs précédents :

$$f(M) = \sum_{i=1}^n \sum_{j=1}^n m_{i,j} f(E_{i,j}) = \lambda \sum_{i=1}^n m_{i,i} = \lambda \text{Tr}(M)$$

Si l'on impose que $f(I_n) = n$ alors on a :

$$f(I_n) = n \Leftrightarrow \lambda \text{Tr}(I_n) = n \Leftrightarrow \lambda = 1$$

Ce qu'il fallait démontrer. □

La caractérisation précédente permet de déduire immédiatement les propriétés utiles suivantes.

Proposition 1.5 :

On a les propriétés suivantes :

1. Pour toute matrice $A \in \mathcal{M}_n(\mathbb{R})$: $\boxed{Tr(A) = Tr(A^T)}$.
2. Pour toute matrice $A \in \mathcal{M}_n(\mathbb{R})$ et pour toute matrice P inversible i.e $P \in GL_n(\mathbb{R})$:

$$\boxed{Tr(PAP^{-1}) = Tr(A)}$$

Ainsi la trace est invariante par changement de base. En particulier :

$$\boxed{Tr(A) = \sum_{\lambda \in Spec(A)} \lambda}$$

3. La trace est invariante par permutation circulaire i.e pour toutes matrices A, B, C de tailles compatibles pour les produits :

$$Tr(ABC) = Tr(CAB) = Tr(BCA)$$

Démonstration. Démonstration en TD. □

Remarque 2. Disons quelques mots sur la propriété 2. Cette propriété est très importante conceptuellement, elle dit en effet que ce que mesure la trace est une quantité qui ne dépend pas du choix de la base dans laquelle on exprime la matrice A . Cette propriété est essentielle en analyse de données mais également et surtout en physique. En effet le choix d'une base correspond en physique à un choix de repère. Or l'acte abstrait de choisir un repère, pour résoudre par exemple un problème de mécanique, ne doit pas avoir de sens physique.

Très bien, continuons à « déconstruire » notre formule $Tr(A^T A)$ et voyons pour une matrice $A \in \mathcal{M}_{mn}(\mathbb{R})$ les matrices : $A^T A$ et AA^T .

Proposition 1.6 :

Soit $A \in \mathcal{M}_{mn}(\mathbb{R})$ alors les matrices $A^T A$ et AA^T :

1. sont symétriques,
2. ont les mêmes valeurs propres non nulles,
3. toutes ces valeurs propres (non nulles) sont toutes strictement positives.
4. Les rangs de $A^T A$ et AA^T sont égaux et valent le rang de A .

Démonstration. Démonstration en TD. □

On pose alors l'importante définition suivant :

Définition 1.2 :

On appelle les racines carrées des valeurs propres non nulles de $A^T A$ (ou de AA^T) : **les valeurs singulières de A .**

Avant d'aller plus loin, mettons un peu les « mains dans le cambouis mathématique » de ce qu'incarne les matrices $A^T A$ ou AA^T . Effet nous avons vu la norme de Frobenius d'une matrice générale $A \in \mathcal{M}_{m,n}(\mathbb{R})$ est donnée par :

$$\|A\|_F^2 = Tr(A^T A)$$

c'est à dire que cette quantité incarne un invariant de base pour la matrice $A^T A$ ou AA^T . Raisonnons sur $A^T A$ (le résultat suivant se transpose aisément à AA^T). Écrivons par blocs la matrice A en la partitionnant en vecteur colonne de la manière suivante :

$$A = (a_1 \quad a_2 \quad \cdots \quad a_n)$$

où les $a_i \in \mathcal{M}_{m,1}(\mathbb{R})$ sont les colonnes de A . On peut alors considérer le produit matriciel de A^T et A sous le point de vue de cette partition et on a donc :

$$A^T A = \begin{pmatrix} a_1^T \\ a_2^T \\ \vdots \\ a_n^T \end{pmatrix} (a_1 \quad a_2 \quad \cdots \quad a_n) = \begin{pmatrix} a_1^T a_1 & a_1^T a_2 & \cdots & a_1^T a_n \\ a_2^T a_1 & a_2^T a_2 & \cdots & \vdots \\ \vdots & \vdots & \ddots & \vdots \\ a_n^T a_1 & \cdots & \cdots & a_n^T a_n \end{pmatrix}$$

Ainsi le coefficient (i, j) de la matrice $A^T A$ est donné par le produit scalaire euclidien du vecteur colonne a_i avec le vecteur colonne a_j :

$$(A^T A)_{ij} = a_i^T a_j = \langle a_i, a_j \rangle_2$$

où $\langle \bullet, \bullet \rangle_2$ note le produit scalaire euclidien. On rappelle également que le produit scalaire euclidien est donné géométriquement par la formule :

$$\langle a_i, a_j \rangle_2 = \|a_i\|_2 \|a_j\|_2 \cos(\theta(a_i, a_j))$$

où $\|\bullet\|_2$ est la norme euclidienne et $\cos(\theta(a_i, a_j))$ note le cosinus de l'angle orienté entre les vecteurs a_i et a_j .

Il vient donc que les coefficients de la matrice $A^T A$ donnent une information sur les projections des colonnes de A sur elles-mêmes ou pour le dire autrement la matrice $A^T A$ mesure l'alignement (donné par le cos) entre les colonnes de la matrice. Pour le dire plus précisément **cette matrice condense absolument toute la géométrie (euclidienne) intrinsèque portée par les colonnes de A** (le même raisonnement montre que AA^T correspond aux lignes de A) et la norme de Frobenius en ne retenant que la trace i.e. la somme des valeurs propres de $A^T A$ (ou de AA^T) capture la somme des déformations portées par cette géométrie/configuration intrinsèque donnée par les colonnes (ou lignes) de A .

Nous verrons dans la suite sur des exemples en quoi les valeurs singulières permettent de effectivement de quantifier « l'information » contenu dans une matrice A .

1.2.2. La SVD enfin...

On peut alors énoncer le théorème principal de ce chapitre.

Théorème 1.1 : Décomposition en valeurs singulières (SVD)

Pour toute matrice $A \in \mathcal{M}_{m,n}(\mathbb{R})$ de rang r , il existe deux matrices orthogonales $U \in \mathbb{O}_m$ et $V \in \mathbb{O}_n$ et une matrice diagonale $\Sigma \in \mathcal{M}_{mn}(\mathbb{R})$ telle que :

$$A = U \Sigma V^T$$

$$\text{avec : } \Sigma = \begin{pmatrix} \sigma_1 & & & & \\ & \ddots & & & \\ & & \sigma_r & & \\ & & & 0 & \\ & & & & \ddots \\ & & & & & 0 \end{pmatrix} \quad \text{où } \sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0 \text{ sont les valeurs singulières de } A.$$

Démonstration. La matrice $A^T A$ étant symétrique et à valeurs propres réelles positives, le théorème spectral nous dit qu'elle est diagonalisable dans une base orthonormée et que ses valeurs propres sont réelles i.e qu'il existe une décomposition :

$$A^T A = V D V^T$$

$$\text{avec } D = \begin{pmatrix} \sigma_1^2 & & & & \\ & \ddots & & & \\ & & \sigma_r^2 & & \\ & & & 0 & \\ & & & & \ddots \\ & & & & & 0 \end{pmatrix} \quad \text{où } \sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0 \text{ et } r \text{ est le rang de } A \text{ et } V \in \mathbb{O}_n.$$

En notant v_i les vecteurs colonnes de V . Posons pour tout $i \in \llbracket 1, r \rrbracket$, $u_i = \frac{1}{\sigma_i} A v_i$. Ces vecteurs sont orthogonaux deux à deux par construction et complétons cette famille en une base orthonormée de $\mathbb{R}^m$ (par l'algorithme de Gram-Schmidt). Notons $U \in \mathbb{O}_m$ la matrice obtenue.

Par construction encore on a bien :

$$A = U \begin{pmatrix} \sigma_1 & & & & \\ & \ddots & & & \\ & & \sigma_r & & \\ & & & 0 & \\ & & & & \ddots \\ & & & & & 0 \end{pmatrix} V^T$$

□

Commençons par quelques remarques sur cette décomposition et sa démonstration :

1. La démonstration précédente est « constructive » i.e. elle fournit un algorithme de calcul d'une décomposition en valeurs singulières.
2. Elle n'est en général pas unique. En effet si le rang de la matrice n'est pas plein on a une liberté de choix de la base à orthonormaliser dans l'algorithme de Gram-Schmidt.
3. Les u_i sont les vecteurs propres normés de AA^T et les v_i sont les vecteurs propres normés de $A^T A$. De même pour tout $i \in \llbracket 1, r \rrbracket$, où r est le rang de A , on a : $Av_i = \sigma_i u_i$ et $A^T u_i = \sigma_i v_i$, avec σ_i les valeurs singulières de A .
4. En considérant par blocs colonnes les matrices U et V i.e en notant :

$$U := (u_1, \dots, u_m) \quad \text{et} \quad V := (v_1, \dots, v_n)$$

On a l'écriture suivante (en notant classiquement e_i les vecteurs de la base canonique) :

$$\begin{aligned} A &= U \Sigma V^T \\ &= (u_1, \dots, u_m) \begin{pmatrix} \sigma_1 & & & & & \\ & \ddots & & & & \\ & & \sigma_r & & & \\ & & & 0 & & \\ & & & & \ddots & \\ & & & & & 0 \end{pmatrix} \begin{pmatrix} v_1^T \\ \vdots \\ v_n^T \end{pmatrix} \\ &= (u_1, \dots, u_m) \begin{pmatrix} \sigma_1 v_1^T \\ \vdots \\ \sigma_r v_r^T \\ 0 v_{r+1}^T \\ \vdots \\ 0 v_n^T \end{pmatrix} \\ &= \sum_{i=1}^r \sigma_i u_i v_i^T \end{aligned}$$

Ainsi A se décompose sous la forme d'une somme de matrice de rang 1 que sont les $\sigma_i u_i v_i^T$.

Nous allons voir que cette dernière expression :

$$A = \sum_{i=1}^r \sigma_i u_i v_i^T$$

est extrêmement importante. Elle permet en outre de définir une **décomposition en valeurs singulières réduite (ou partielle)** en ne conservant que les r premiers vecteurs u_i et v_i que l'on écrira :

$$A := U_r \Sigma_r V_r^T$$

où :

$$\begin{cases} U_r := (u_1, \dots, u_r) \\ V_r := (v_1, \dots, v_r) \\ \Sigma_r := \begin{pmatrix} \sigma_1 & & \\ & \ddots & \\ & & \sigma_r \end{pmatrix} \end{cases} \in \mathcal{M}_{r,r}(\mathbb{R})$$

Exemple 3. En utilisant l'algorithme de la démonstration de l'existence de la SVD pour une matrice A . Nous allons calculer à la main en guise d'exemple une telle décomposition pour une matrice simple :

$$A = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{pmatrix}$$

On commence par calculer les valeurs singulières de A . Comme A est clairement de rang 1, il n'y en a qu'une seule valeur singulière. La matrice :

$$A^T A = \begin{pmatrix} 3 & 3 \\ 3 & 3 \end{pmatrix}$$

a pour spectre : $\text{Spec}(A^T A) = \{0, 6\}$. Ainsi $\sigma_1 = \sqrt{6}$. Le vecteur $v_1 := \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$ est clairement un vecteur propre normée de $A^T A$. On complète ce vecteur en base orthonormée en posant de manière évidente :

$$v_2 := \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

Comme : $Av_1 = \sigma_1 u_1$, on trouve :

$$u_1 := \sqrt{\frac{1}{3}} \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$

et en complétant ce vecteur en une base orthonormée (Gram-Schmidt) on trouve :

$$U := \begin{pmatrix} \sqrt{\frac{1}{3}} & -\sqrt{\frac{2}{3}} & 0 \\ \sqrt{\frac{1}{3}} & \sqrt{\frac{1}{6}} & -\frac{1}{\sqrt{2}} \\ \sqrt{\frac{1}{3}} & \sqrt{\frac{1}{6}} & \frac{1}{\sqrt{2}} \end{pmatrix}$$

Par suite on a :

$$A = \begin{pmatrix} \sqrt{\frac{1}{3}} & -\sqrt{\frac{2}{3}} & 0 \\ \sqrt{\frac{1}{3}} & \sqrt{\frac{1}{6}} & -\frac{1}{\sqrt{2}} \\ \sqrt{\frac{1}{3}} & \sqrt{\frac{1}{6}} & \frac{1}{\sqrt{2}} \end{pmatrix} \begin{pmatrix} \sqrt{6} & 0 \\ 0 & 0 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} \sqrt{\frac{1}{2}} & \sqrt{\frac{1}{2}} \\ \sqrt{\frac{1}{2}} & -\sqrt{\frac{1}{2}} \end{pmatrix}$$

Ce qui en utilisant la forme réduite s'écrit finalement :

$$A = \sigma_1 u_1 v_1^T = \sqrt{6} \begin{pmatrix} \sqrt{\frac{1}{3}} \\ \sqrt{\frac{1}{3}} \\ \sqrt{\frac{1}{3}} \end{pmatrix} \begin{pmatrix} \sqrt{\frac{1}{2}} \\ \sqrt{\frac{1}{2}} \end{pmatrix}^T$$

Remarque 1.1 :

Dans la pratique l'algorithme de calcul d'une SVD précédent (i.e. celui du théorème) est peu efficace. Lorsque nous aurons à effectuer des calculs numériques sous Python nous utiliserons la fonction : **np.linalg.svd** de Numpy. L'algorithme utilisé par Numpy utilise une réduction bidiagonale : la matrice d'entrée est réduite à une forme bidiagonale, ce qui permet de simplifier les calculs de la décomposition. Cette méthode est apparue dans les travaux d'algèbre linéaire numérique dès les années 1960. On la doit en particulier aux mathématiciens Gene H. Golub et William Kahan qui ont été des figures clés dans le développement des algorithmes pour la SVD. Leur article fondateur en 1965, "Calculating the Singular Values and Pseudo-Inverse of a Matrix," a posé les bases pour la réduction bidiagonale comme étape préalable à la SVD. Cette décomposition se retrouvant dans de nombreuses solutions théoriques de problèmes concrets, l'algorithme produit par Gene H. Golub et William Kahan a rendu possible la résolution de très nombreux problèmes irréalisables en des temps « raisonnables » avant.

Nous ne verrons pas le détail de cet algorithme (nous l'utiliserons comme une boîte noire) car cela nous demanderait d'introduire des notions et concepts que nous n'allons pas réutiliser par la suite.

1.2.3. Les « atomes » de l'algèbre linéaire

Nous avons qu'une décomposition SVD montre qu'une matrice quelconque réelle A s'écrit sous la forme :

$$A := \sum_{i=1}^r \sigma_i u_i v_i^T$$

où chacun des termes $u_i v_i^T$ est une matrice de rang 1. Dans cette sous-section nous allons dire quelques mots sur les matrices de rang 1 qui, comme nous montre l'existence d'une SVD pour une matrice quelconque A sont des atomes à partir desquels reconstruire toutes les matrices.

Proposition 1.7 :

Soit $M \in \mathcal{M}_{m,n}(\mathbb{R})$ alors M est une matrice de rang 1 si et seulement si il existe $u \in \mathbb{R}^m \simeq \mathcal{M}_{m,1}(\mathbb{R})$ et $v \in \mathbb{R}^n \simeq \mathcal{M}_{n,1}(\mathbb{R})$ deux vecteurs non nuls tels que :

$$M = uv^T$$

En particulier si l'on identifie M à l'application linéaire $M : \mathbb{R}^n \rightarrow \mathbb{R}^m$ définit par $x \mapsto Mx$ on a :

— $\text{Im}(M) = \text{Vect}(u)$ i.e que l'image de M est uniquement portée par le vecteur u . En effet :

$$Mx = \langle v, x \rangle_2 u$$

— $\ker(M) = v^\perp$: conséquence pour un vecteur $x \in \mathbb{R}^n$, M annihile tout ce qui est orthogonal à v .

— L'unique valeur singulière de M est donnée par : $\|u\|_2 \|v\|_2$. En particulier :

$$\|M\|_F = \|u\|_2 \|v\|_2$$

Démonstration. Exercice. □

D'un point de vue des données les matrices de rang 1 incarnent le niveau maximal de redondance. En effet pour une matrice pleine A de taille (m, n) nécessite de stocker mn coefficients indépendants. Or si A est de rang 1 seul $m + n$ coefficients suffisent. Par exemple si $m = n = 1000$, au lieu de stocker $mn = 10^6$ coefficients, il suffit de $m + n = 2000$ coefficients lorsque cette matrice est de rang 1.

En réalité on peut mieux faire, en effet la décomposition d'une matrice de rang n n'est clairement unique :

$$M = uv^T = (\alpha u) \left(\frac{1}{\alpha} v \right)^T$$

pour $\alpha \neq 0$ sont deux décompositions. En imposant que $\|u\|_2 = \|v\|_2 = 1$, on a :

$$M = \sigma uv^T$$

Cette matrice demande $m + n - 1$ coefficients pour être stockée (deux vecteurs sur la sphère unité et un scalaire).

Ce point de vue « stockage » sur les matrices s'appuie également, lorsque l'on pense une matrice de rang 1 comme une application linéaire (point de vue « dynamique » sur une matrice), comme brique atomique des opérations d'écriture et de lecture. En effet dire que $\ker(M) = v^\perp$ c'est décrire une étape de lecture (ou de filtrage) où tout ce qui est orthogonal à v est ignoré et $\text{Im}(M) = \text{Vect}(u)$ signifie que l'on écrit (génère) exclusivement sur u (fois un facteur d'échelle).

1.3. Une heuristique particulière : les valeurs singulières comme une mesure de l'information

Avant de voir des exemples d'utilisations de la SVD nous allons formuler dans cette (relativement) courte section **une heuristique** sur cette décomposition : **les valeurs singulières mesurent une quantité « d'information » robuste, structurée et non redondante** contenu dans un tableau de données modélisé par une matrice $A \in \mathcal{M}_{mn}(\mathbb{R})$. Nous allons dans cette section justifier « semi »formellement cette affirmation. Ici « semi » formelle signifie que nous n'essaierons pas de définir explicitement la notion « d'information », une telle définition faisant l'objet d'ouvrages entiers, nous nous limiterons à une version « intuitive ».

La décomposition en valeurs singulières fait apparaître des notions qui affinent le raisonnement heuristique débuté dans les sections précédentes :

— En écrivant la SVD sous la forme :

$$A = U \Sigma V^T \Leftrightarrow \Sigma = U^T A V$$

on fait apparaître les valeurs singulières (i.e. la matrice diagonale Σ) comme une mesure de l'information « pure » où les « **redondances** » sont supprimées en quelques sortes. Les matrices orthogonales U et V incarnant les changements de repères/bases qui « décorrèlent » (en un sens semi-formel) les données contenus dans la matrice A .

— Par ailleurs le fait que les valeurs singulières soient des nombres réels donc ordonnables :

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$$

permet de **structurer** l'information. La matrice $\sigma_1 u_1 v_1^T$ possède une plus grande importance (sur le plan informationnel) que les autres, etc...

— En outre le fait que le rang r de la matrice A apparaisse dans la SVD permet de mesurer une sorte de **complexité** de l'information contenu dans A . En effet on rappelle que le rang indique le nombre de vecteurs colonnes (ou lignes) qu'il faut pour « reconstruire » une matrice en utilisant des combinaisons linéaires de ceux-ci.

Finissons par un dernier résultat qui permet d'étayer notre heuristique : le **théorème d'entrelacement**. Ce théorème répond à la question suivante : si l'on supprime une colonne (ou une ligne) d'une matrice A comment se comporte l'information contenue dans la nouvelle matrice A' par rapport à celle contenue originellement dans A ? Le théorème d'entrelacement montre que l'information contenue dans une matrice est hiérarchiquement organisée et que les perturbations locales ont un impact limité sur les valeurs singulières dominantes. **Les résultats suivants ne sont pas à connaître par cœur mais leurs présences dans ce cours permettent de justifier et d'affiner notre compréhension de la SVD.**

Commençons un lemme simple.

Corollaire 1.1 :

Soit $A \in \mathcal{M}_{mn}(\mathbb{R})$ et $P(i, j) \in \mathcal{M}_n(\mathbb{R})$ une matrice de transposition. Alors les valeurs singulières de A et de $AP(i, j)$ sont les mêmes.

Démonstration. Soit (λ, v) un couple propre de $A^T A$. Notons $P = P(i, j)$ pour simplifier les notations. Alors (λ, Pv) est un couple propre de $AP(i, j)$. En effet :

$$\begin{aligned} P^T A^T AP(Pv) &= P^T A^T Av \\ &= P^T \lambda v \\ &= \lambda Pv \end{aligned}$$

car $P = P^T$. Ce qui suffit à démontrer le résultat. □

Pour démontrer le théorème d'entrelacement nous allons admettre le résultat suivant.

Proposition 1.8 :

Soient P et Q deux polynômes à coefficients réels et dont toutes leurs racines sont réelles. On suppose que $\deg(P) = n$ et $\deg(Q) = n - 1$. Notons :

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n \quad \text{et} \quad \mu_1 \geq \mu_2 \geq \dots \geq \mu_{n-1}$$

les racines ordonnées de P et Q respectivement. Alors si pour tout $x \in \mathbb{R}$, les racines du polynôme : $M = P + xQ$, sont toutes réelles, les racines de P et Q sont entrelacées i.e. :

$$\lambda_1 \geq \mu_1 \geq \lambda_2 \geq \mu_2 \geq \dots \geq \lambda_{n-1} \geq \mu_{n-1} \geq \lambda_n$$

Nous pouvons alors énoncer le théorème d'entrelacement.

Théorème 1.2 : Théorème d'entrelacement

On suppose que $m \geq n$. Soit $A \in \mathcal{M}_{mn}(\mathbb{R})$. Nous noterons $A_{/c_i}$ la matrice obtenue de A en supprimant la i -ème colonne. En notant σ_i les valeurs singulières de A et σ'_i les valeurs singulières de $A_{/c_i}$, on a :

$$\sigma_1 \geq \sigma'_1 \geq \sigma_2 \geq \sigma'_2 \geq \dots \geq \sigma_{n-1} \geq \sigma'_{n-1} \geq \sigma_n \geq 0$$

(où l'on a étendu ici la définition des valeurs singulières à des valeurs nulles possibles pour alléger l'énoncé).

Démonstration. D'après le lemme précédent on peut supposer que $A_{/c_i} = A_0$ tel que :

$$A = \begin{pmatrix} A_0 & a_n \end{pmatrix}$$

où a_n est le dernier vecteur colonne de A .

Soit $(x, \lambda) \in \mathbb{R}^2$ et posons la matrice suivante :

$$M(x) = \begin{pmatrix} A_0^T A_0 & A_0^T a_n \\ a_n^T A_0 & a_n^T a_n + x \end{pmatrix}$$

Regardons alors le polynôme caractéristique de $M(x)$:

$$\begin{aligned} P_{M(x)}(\lambda) &= \det(M(x) - \lambda I_n) \\ &= \begin{vmatrix} A_0^T A_0 - \lambda I_{n-1} & A_0^T a_n \\ a_n^T A_0 & a_n^T a_n - \lambda \end{vmatrix} + x \begin{vmatrix} A_0^T A_0 - \lambda I_{n-1} & 0 \\ a_n^T A_0 & 1 \end{vmatrix} \\ &= P_{A^T A}(\lambda) + x P_{A_0^T A_0}(\lambda) \end{aligned}$$

où $P_{A^T A}(\lambda)$ est le polynôme caractéristique de $A^T A$ et $P_{A_0^T A_0}(\lambda)$ celui de $A_0^T A_0$. On a utilisé ici la linéarité du déterminant par rapport à la dernière colonne.

Comme $M(x)$ est symétrique, les racines de $P_{M(x)}(\lambda)$ sont réelles quelque soit $x \in \mathbb{R}$ et donc d'après la propriété précédente on en déduit le résultat voulu. $\square$

Ainsi nous pouvons dire que les valeurs singulières en tant que mesure de l'information contenue dans une matrice est une mesure **robuste** au sens où une perturbation de celle-ci par la suppression d'une colonne (ou d'une ligne) modifie de manière « cohérente » les répartitions des valeurs singulières (elles sont entrelacées).

La discussion précédente pourrait être encore longue, très longue même : des théorèmes comme le théorème min-max de Courant-Fischer permet de déduire une multitude de résultat et d'étayer l'interprétation informationnelle des valeurs singulières. Arrêtons nous ici et gardant cette heuristique en tête afin de comprendre en quoi les résultats des chapitres suivants s'intègrent et suivent de cette interprétation.

1.4. Synthèse courte du chapitre

Dans ce chapitre nous avons fixé le cadre de ce cours : celui des matrices réelles générales $A \in \mathcal{M}_{m,n}(\mathbb{R})$ vues sous le prisme (=les lunettes) de la norme de Frobenius. Nous avons vu que :

- la norme de Frobenius est invariante par composition à gauche et à droite par des matrices orthogonales et que cette norme est issue d'un produit scalaire donnée par la trace,
- cette norme de Frobenius nous a révélé l'existence d'invariant : les valeurs singulières et conduit à montrer que toute matrice A de rang r se décompose sous la forme d'une somme de matrices de rang 1 par ce que l'on appelle une décomposition en valeurs singulières :

$$A = U \Sigma V^T = \sum_{i=1}^r \sigma_i u_i v_i^T$$

avec $U \in \mathbb{O}_m$ et $V \in \mathbb{O}_n$ et une matrice diagonale $\Sigma \in \mathcal{M}_{mn}(\mathbb{R})$ contenant les valeurs singulières ordonnées $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$ de A .

- cette décomposition en valeurs singulières n'est pas unique à la différence des valeurs singulières elles-mêmes mais permet de donner une « décomposition atomique », structurée (=hiérarchisée) et robuste de l'information portée par une matrice $A \in \mathcal{M}_{m,n}(\mathbb{R})$.

Le théorème d'Eckart-Young-Mirsky et ses conséquences

2.1. Le problème de l'approximation de Schmidt

Dans cette section nous noterons, sauf mention explicite du contraire, $A \in \mathcal{M}_{mn}(\mathbb{R})$ une matrice de rang $rg(A) = r$ et $\alpha \in \mathbb{N}^*$ tel que : $\alpha < r$.

Dans la discussion du chapitre précédent nous avons interpréter le rang d'une matrice comme une mesure de la complexité de l'information contenue dans celle-ci. Or plus une information est complexe plus elle est difficile à interpréter mais également en un certains sens difficile à « résumer ». Ici le terme « résumer » renvoie à la capacité à stocker de manière efficace l'information. Ainsi un problème naturel est celui de la réduction de cette complexité. En d'autres termes peut-on trouver une matrice moins complexes sur le plan informatif approchant une matrice donnée ou pour le dire encore autrement : quitte à perdre un peu d'information peut-on trouver une manière plus économe de stocker l'information incarnée par une matrice ? La formalisation de ce problème nous conduit au problème de l'approximation de Schmidt.

Définition 2.1 : Le problème de l'approximation de Schmidt

L'approximation de Schmidt de rang α de A consiste à trouver une matrice X de même taille que A mais dont le rang est α et qui minimise la quantité :

$$\min_{X \in \mathcal{M}_{mn}(\mathbb{R}), rg(X)=\alpha} \|A - X\|_F$$

La résolution de ce problème devient une application évidente de la décomposition en valeurs singulières.

Théorème 2.1 : Théorème d'Eckart-Young-Mirsky

Le problème d'approximation de Schmidt admet une solution et cette solution est donnée par la matrice :

$$X = \sum_{i=1}^{\alpha} \sigma_i u_i v_i^T$$

où les σ_i sont les valeurs singulières de A , u_i et v_i les vecteurs colonnes de U et V respectivement dans une SVD de $A := U\Sigma V^T$.

Démonstration. Soit $A = U\Sigma V^T$, une décomposition en valeurs singulières de A et $X \in \mathcal{M}_{mn}(\mathbb{R})$. On a alors en utilisant les propriétés de la norme de Frobenius :

$$\|A - X\|_F = \|\Sigma - U^T X V\|_F$$

En notant $Y = U^T X V$, on a donc :

$$\|\Sigma - U^T X V\|_F^2 = \|\Sigma - Y\|_F^2 = \sum_{i=1}^r (\sigma_i - y_{ii})^2 + \sum_{i=r+1}^{\min(m,n)} y_{ii}^2 + \sum_{i,j,i \neq j} y_{ij}^2$$

Le minimum de cette quantité est alors atteint pour :

$$\begin{cases} y_{ii} = \sigma_i & i \in \llbracket 1, r \rrbracket \\ y_{ij} = 0 & \text{sinon} \end{cases}$$

Mais si l'on impose $rg(X) = \alpha$, alors comme $rg(Y) = rg(X) = \alpha$, on obtient :

$$\begin{cases} y_{ii} = \sigma_i & i \in \llbracket 1, \alpha \rrbracket \\ y_{ii} = 0 & i \in \llbracket \alpha + 1, r \rrbracket \end{cases}$$

Ainsi une solution au problème de l'approximation de Schmidt est :

$$Y := \Sigma_\alpha = \begin{pmatrix} \sigma_1 & & & & \\ & \ddots & & & \\ & & \sigma_\alpha & & \\ & & & 0 & \\ & & & & \ddots \end{pmatrix} = U^T X V$$

Et par suite :

$$X = U \Sigma_\alpha V^T = \begin{pmatrix} u_1 & \cdots & u_m \end{pmatrix} \Sigma_\alpha \begin{pmatrix} v_1^T \\ \vdots \\ v_n^T \end{pmatrix} = \sum_{i=1}^{\alpha} \sigma_i u_i v_i^T$$

□

Remarque 4. Insistons sur les deux points suivants :

— L'erreur d'approximation est alors :

$$\min_{X \in \mathcal{M}_{mn}(\mathbb{R}), rg(X)=\alpha} \|A - X\|_F = \sqrt{\sum_{i=\alpha+1}^r \sigma_i^2}$$

Cette erreur est ainsi une mesure quantitative de l'information perdue i.e. des détails négligés.

— Si $\sigma_\alpha = \sigma_{\alpha+1}$, la solution n'est pas unique puisque :

$$\tilde{X} = \sum_{i=1}^{\alpha-1} \sigma_i u_i v_i^T + \sigma_{\alpha+1} u_{\alpha+1} v_{\alpha+1}^T$$

est également une solution qui minimise la norme de Frobenius.

Exemple 5. Un exemple simple avec Numpy.

$$\text{Soit } A = \begin{pmatrix} 3 & 0 & 6 \\ 1 & 0,1 & 2,1 \\ 0 & 0,1 & 0,1 \end{pmatrix}.$$

Le code suivant :

```
1 import numpy as np
2 A=np.array([[3,0,6],[1,0.1,2.1],[0,0.1,0.1]])
3 r=np.linalg.matrix_rank(A)
4 U,S,VT=np.linalg.svd(A)
```

donne :

$$rg(A) = 2 \quad S = \begin{pmatrix} 7.10052 & 0.15037 & 5.6937e-17 \end{pmatrix}$$

On voit que la dernière valeur singulière devrait être égale à 0. L'approximation de rang 1 de A est alors donnée par :

```
1 A1=S[0]*U[:,[0]]@VT[[0],:]
```

Ce qui renvoie :

$$A(1) = \begin{pmatrix} 2.98706 & 0.0321462 & 6.00626 \\ -0.740453 & -0.00796862 & -1.48887 \\ -0.725365 & -0.00780625 & -1.45854 \end{pmatrix}$$

2.1.1. Quelques mots sur l'interprétation géométrique

Dans cette sous-section nous allons donner une interprétation (=formalisation) géométrique du théorème précédent.

Fixons α un entier plus petit que $\min(m, n)$ et donnons nous une famille de α vecteurs de taille n formant une famille orthonormée $\{v_j\}_{j \in \llbracket 1, \alpha \rrbracket}$. On posera la matrice :

$$V_\alpha := (v_1, \dots, v_\alpha) \in \mathcal{M}_{n, \alpha}(\mathbb{R})$$

On définit la matrice carrée $\Pi_\alpha := V_\alpha V_\alpha^T \in \mathcal{M}_n(\mathbb{R})$. Notons : $\mathcal{E}_\alpha$ l'espace des matrices dont les lignes sont dans $Vect(v_1, \dots, v_\alpha)$ i.e :

$$\mathcal{E}_\alpha := \{M \in \mathcal{M}_{m, n}(\mathbb{R}) \mid M_{i, \bullet} := (m_{i, 1}, \dots, m_{i, n}) \in Vect(v_1, \dots, v_\alpha), i \in \llbracket 1, m \rrbracket\}$$

On a alors la proposition suivante.

Proposition 2.1 :

L'application linéaire définit par :

$$\begin{aligned} \Phi_\alpha : \mathcal{M}_{m, n}(\mathbb{R}) &\rightarrow \mathcal{E}_\alpha \\ M &\mapsto M\Pi_\alpha \end{aligned}$$

est un projecteur orthogonale sur $\mathcal{E}_\alpha$ pour la norme de Frobenius. En particulier on a le théorème de Pythagore :

$$\|M\|_F^2 = \|\Phi_\alpha(M)\|_F^2 + \|M - \Phi_\alpha(M)\|_F^2$$

Démonstration. On a d'abord de manière évidente :

$$\Phi_\alpha \circ \Phi_\alpha = \Phi_\alpha$$

car : $\Pi_\alpha \Pi_\alpha = V_\alpha V_\alpha^T V_\alpha V_\alpha^T = V_\alpha I_\alpha V_\alpha^T = \Pi_\alpha$. De plus on a :

$$\begin{aligned} \langle \Phi_\alpha(M), M - \Phi_\alpha(M) \rangle_F &= Tr(\Phi_\alpha(M)^T (M - \Phi_\alpha(M))) \\ &= Tr(\Pi_\alpha^T M^T (M - M\Pi_\alpha)) \\ &= Tr(\Pi_\alpha^T M^T M) - Tr(\Pi_\alpha^T M^T M\Pi_\alpha) \\ &= Tr(\Pi_\alpha M^T M) - Tr(\Pi_\alpha M^T M\Pi_\alpha) \\ &= Tr(\Pi_\alpha M^T M) - Tr(M^T M\Pi_\alpha \Pi_\alpha) \\ &= 0 \end{aligned}$$

d'où le résultat. □

Le théorème d'Eckart-Young-Mirsky nous dit donc que la meilleure approximation de rang α d'une matrice A :

$$A(\alpha) = \sum_{i=1}^{\alpha} \sigma_i u_i v_i^T$$

avec $A = U\Sigma V^T$ est exactement :

$$\Phi_\alpha(A) = AV_\alpha V_\alpha^T$$

pour $V_\alpha = (v_1, \dots, v_\alpha)$, les α -premier vecteurs de V . En effet :

$$\Phi_\alpha(A) = AV_\alpha V_\alpha^T = U\Sigma V^T V_\alpha V_\alpha^T = U\Sigma(e_1, \dots, e_\alpha) V_\alpha^T = \sum_{i=1}^{\alpha} \sigma_i u_i v_i^T = A(\alpha)$$

où les e_i sont les vecteurs de la base canonique de $\mathbb{R}^n$ (vu sous forme de vecteur colonne toujours). Dans notre cas avec ce choix de V_α issu de la SVD on a :

$$\|\Phi_\alpha(M)\|_F^2 = \sum_{i=1}^{\alpha} \sigma_i^2 \quad \text{et} \quad \|M - \Phi_\alpha(M)\|_F^2 = \sum_{i=\alpha+1}^r \sigma_i^2$$

La SVD et le théorème étant symétriques en les lignes ou les colonnes on a également une formulation ligne de la propriété précédente. On notera pour les lignes :

$$\Phi^\alpha(M) := U_\alpha U_\alpha^T M$$

pour $U_\alpha := (u_1, \dots, u_\alpha)$ avec les u_i une famille de taille α de vecteurs de taille m cette fois. Lorsque ces u_i sont les α -premier vecteur colonne de U , on retrouve le théorème d'Eckart-Young-Mirsky.

Dans la suite nous verrons que les espaces vectoriels :

$$\begin{cases} L_\alpha := Vect(u_1, \dots, u_\alpha) \\ C_\alpha := Vect(v_1, \dots, v_\alpha) \end{cases}$$

seront d'une grande importance dans l'interprétation des applications du théorème d'Eckart-Young-Mirsky.

2.2. Un exemple graphique : la compression d'image

Nous allons voir dans un exemple de l'utilisation de la SVD et du théorème d'Eckart-Young-Mirsky dans le cadre de la compression d'images.

Prenons l'image suivante :

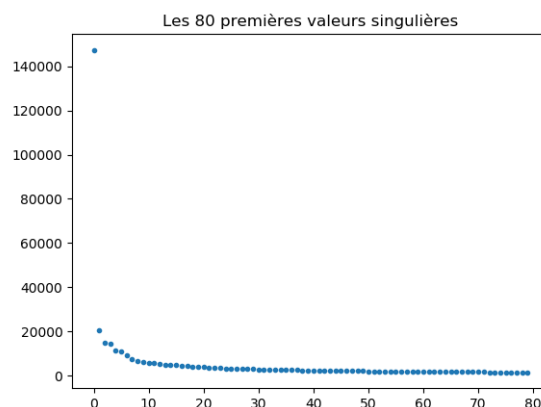


Importons cette image (en noir et blanc pour simplifier) sous forme de matrice :

```
1 import numpy as np
2 import cv2
3 matM=cv2.imread('oiseau.jpg',0)
4 cv2.imshow("Sortie", matM) #pour l'affichage
```

L'élément $matM$ est une matrice de taille : 1280×1920 et de rang : $rg(A) = 1280$. Le fait que le rang soit plein i.e. égal au minimum entre le nombre de ligne et celui de colonne nous dit simplement qu'une image d'une scène réelle est quelque chose de complexe sur le plan informationnel.

Voyons ce que nous donne les valeurs singulières de cette image (noir et blanc) vue ici comme une matrice. On calcule la SVD de $matM$ grâce à la fonction de Numpy et l'on s'intéresse aux valeurs singulières, plus particulièrement à leurs répartitions numériques. Traçons les 80 premières :



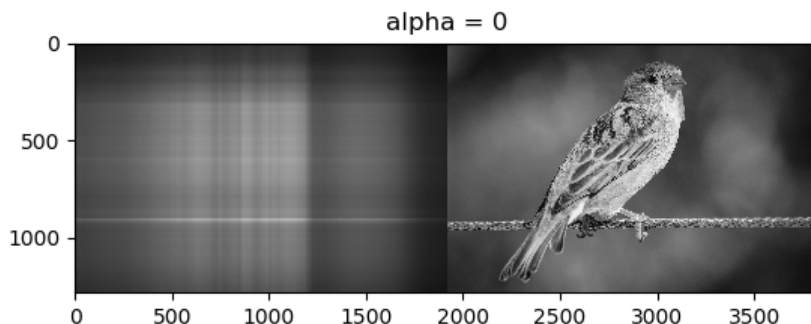
On remarque une décroissance rapide des valeurs singulières. C'est ce phénomène qui va motiver l'utilisation de l'approximation de Schmidt trouvée grâce à la SVD. En effet très peu de valeurs singulières contiennent beaucoup d'information. Ainsi nous allons voir ce qui se passe lorsque nous passons d'une matrice avec un rang élevé (ici 1280) à une version moins « complexe » en terme d'information i.e. à une matrice de rang plus faible.

On calcule donc :

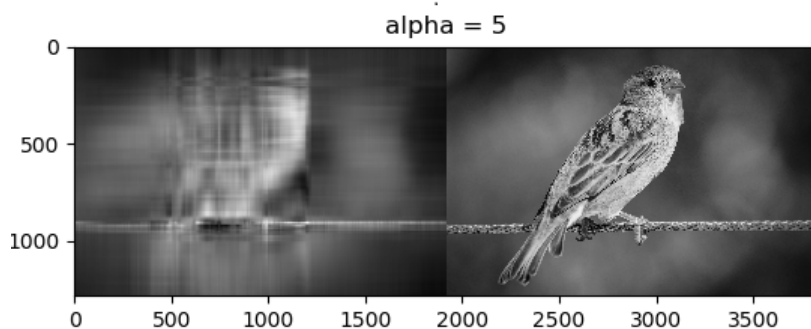
$$A(\alpha) = \sum_{i=0}^{\alpha} \sigma_i u_i v_i^T$$

pour différentes valeurs de α . Voyons ce que cela nous donne (dans Numpy les indices des matrices commencent à 0). On affichera à gauche l'approximation et à droite l'image complète :

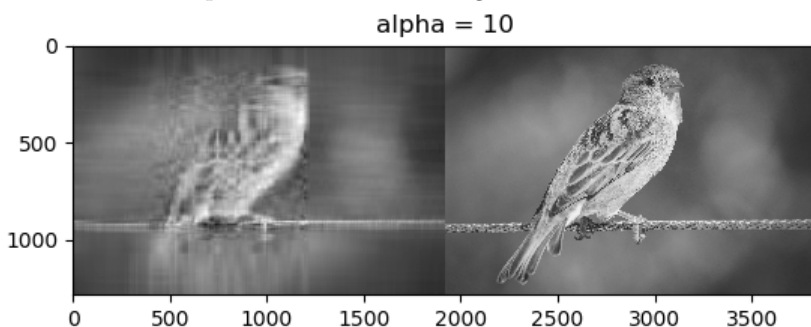
- Pour $\alpha = 0$, i.e. en approximant *matM* par une matrice de rang 1 :



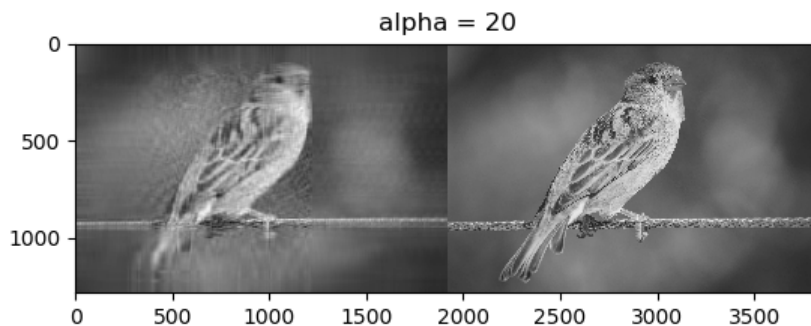
- Pour $\alpha = 5$, i.e. en approximant *matM* par une matrice de rang 5 :



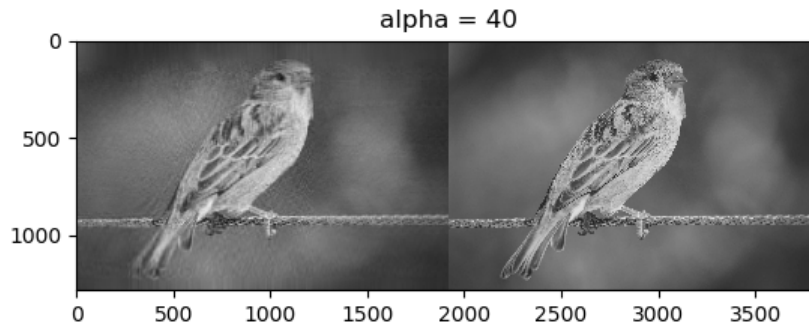
- Pour $\alpha = 10$, i.e. en approximant *matM* par une matrice de rang 10 :



- Pour $\alpha = 20$:



- Pour $\alpha = 40$:



Morale : on voit que l'on peut passer d'une image engendrée par 1280 vecteurs à une image engendrée par 40 vecteurs uniquement sans porter un gros préjudice au contenu de l'image. Ainsi on peut construire des algorithmes de compression des données qui permettent de réduire fortement la taille en mémoire de stockage d'une image (ou plus généralement d'une matrice). Ce que nous venons de voir est l'archétype même d'une méthode descriptive : nous avons produit une manière d'explorer les structures des données considérées et cela nous a permis d'en générer un « résumé » formel cohérent plus « léger ».

Nous verrons dans la section suivante comment les idées « algébriques » précédentes en les interprétant d'un point de vu informationnel nous amèneront vers ce que l'on appelle des méthodes factorielles comme l'analyse en composantes principales.

2.3. L'analyse en composantes principales (version algébrique)

Déroulons encore les conséquences du théorème d'Eckart-Young-Mirsky. L'exemple d'application sur les images précédent était instructif mais la modélisation des images comme des matrices n'est pas l'utilisation la plus courante des matrices dans les domaines d'applications actuels. Les matrices servent très souvent à modéliser des tableaux de données dans ce que l'on nomme (entre autres, d'autres méthodes existent) les méthodes factorielles.

Les méthodes factorielles (ou analyse factorielle) forment un ensemble d'outils mathématiques qui consiste grossièrement à interpréter un tableau de données numériques, appelées **variables quantitatives**, ou non numériques, appelées **variables qualitatives** comme la données pour des individus, individus incarnées par les lignes, de données observées sur ces individus, les colonnes. On a ainsi une distinction entre « individus » et « variables ». C'est le cas classique des tableaux de données où nous ferons une analyse en composantes principales. Il existe néanmoins des tableaux où la distinction entre « variables » et « individus » n'a pas de sens, dans ce cas on parlera de **table de contingence** : ce type de tableau numérique sera traité par l'analyse factorielle des correspondances (AFC).

Afin de fixer les idées donnons un exemple de type « individus » et « variables ». Soit le tableau numérique déjà vu représentant des données sur 5 clients collectées par une entreprise :

Client	Age	Revenu annuel (k€)	Dépenses mensuelles (k€)	Score satisfaction
Client 1	22	18	1.5	80
Client 2	25	20	1.7	85
Client 3	47	45	3.8	60
Client 4	35	30	2.5	70
Client 5	46	50	4.2	55

D'un point de vu statistique le tableau précédent sera interprété comme une réalisation d'un m -échantillon :

$$(Client_1, Client_2, \dots, Client_m)$$

(ici $m = 5$) d'un vecteur aléatoire $X := (X_1, X_2, \dots, X_n)$ (ici $n = 4$) défini sur un espace probabilisé $(\Omega, \mathbb{A}, P)$ (**les vecteurs aléatoires seront considérés comme des vecteurs lignes**). Dans le tableau précédent X_1 incarne une variable aléatoire modélisant l'âge des clients. Pour des variables non numérique i.e. pour des variables qualitatives (par exemple des réponses par « oui » ou « non » à des questions, des plages de valeurs, etc...), nous verrons que l'on peut traiter ces cas en se ramenant à des variables quantitatives.

Plus généralement nous penserons dans cette partie les tableaux numériques (variables quantitative) comme des réalisation d'un échantillon d'un vecteur aléatoire. Ce que l'on se représente sous la forme d'un tableau/matrice :

	X_1	X_2	$\dots$	X_n
Ind_1	X_{11}	X_{12}	$\dots$	X_{1n}
Ind_2	X_{21}	X_{22}	$\dots$	X_{2n}
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
Ind_m	X_{m1}	X_{m2}	$\dots$	X_{mn}

Cette matrice est de taille (m, n) et est à coefficient des fonctions $X_{ij} : \Omega \rightarrow \mathbb{R}$. Ainsi un vecteur colonne :

$$(X_{1i}, X_{2i}, \dots, X_{mi})^T$$

est la donnée d'un m -échantillon de la variable aléatoire X_i . Les lignes incarnant des « individus » à un sens très large pour lesquels l'on peut mesurer des données (les v.a. X_i).

L'objectif principal est alors de révéler des structures sous-jacentes dans les données en les projetant dans des espaces de dimension réduite tout en préservant l'information essentielle portée par ces variables aléatoires. C'est ici que notre interprétation du théorème d'Eckart-Young-Mirsky sera très utile. Toujours sur l'exemple précédent, peut-on par exemple regrouper et en quel sens des clients entre eux afin de créer des « profils » types de clients ?

Afin de répondre à ce type de questions nous verrons des méthodes qui comprennent plusieurs approches classiques que nous explorerons dans ce cours et/ou en TDs :

- **l'Analyse en Composantes Principales (ACP)**, utilisée pour réduire la dimension des données quantitatives en préservant au maximum l'information contenue dans les données.
- **l'Analyse Factorielle des Correspondances (AFC) et l'Analyse des Correspondances Multiples (ACM)**, qui se concentrent sur l'étude des relations entre variables qualitatives ou qualitatives multiples. Les variables qualitatives sont des données non numériques comme une réponse par « oui » ou « non » à un questionnaire. Ces deux méthodes permettent ainsi de produire des calculs pertinents sur des données non directement numériques.

En termes techniques, ces méthodes cherchent à décomposer les données en composantes principales ou factorielles, ce qui revient en général à rechercher des bases orthogonales (pour un certains produits scalaires) maximisant une mesure pertinente. Cette décomposition mathématique permet de « simplifier » les données tout en préservant les éléments significatifs pour une interprétation plus « claire » et « intuitive ».

Les méthodes factorielles jouent un rôle crucial dans l'exploration et la visualisation des données complexes, tout en fournissant des outils de réduction de dimension et de compression adaptés aux différents types de données. Elles permettent également de préparer efficacement les données pour des analyses plus avancées, comme les modèles prédictifs ou les algorithmes de clustering.

2.3.1. Quelques considérations générales

Nous allons commencer par voir l'importantissime : **analyse en composantes principales (ACP)**. C'est une méthode dite **multidimensionnelle** : un grand nombre de variable aléatoire sera traité en même temps. Cette méthode est si essentielle qu'elle est l'étape première de quasiment toutes les études de données très complexes comme en génétique, anthropologie, données clients, etc...

Considérons un tableau numérique (= matrice) que l'on va penser comme une réalisation R d'un m -échantillon d'un vecteur aléatoire $X := (X_1, X_2, \dots, X_n)$ i.e. R est la donnée d'une matrice de taille (m, n) :

$$R := \begin{pmatrix} x_{11} & \cdots & x_{1n} \\ \vdots & \cdots & \vdots \\ x_{m1} & \cdots & x_{mn} \end{pmatrix} \in \mathcal{M}_{m,n}(\mathbb{R})$$

Chaque ligne de cette matrice incarne des mesures données de variables quantitatives sur un individu précis (l'individu représentant la ligne). En résumé on peut représenter une telle situation avec le formalisme « tableau » de la forme :

Variables	X_1	X_2	$\cdots$	X_n
Individus				
Ind_1	x_{11}	x_{12}	$\cdots$	x_{1n}
Ind_2	x_{21}	x_{22}	$\cdots$	x_{2n}
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
Ind_m	x_{m1}	x_{m2}	$\cdots$	x_{mn}

L'objectif principal de l'ACP est de « rapprocher » / de voir des « liens » entre certains individus. On parle souvent de l'ACP comme d'une méthode de recherche de ressemblances. Qui dit ressemblance entre des individus que l'on peut penser comme des vecteurs de $\mathbb{R}^n$ dit norme¹. Ainsi grâce à une norme on peut définir une distance entre deux individus en prenant la norme de leur différence dans $\mathbb{R}^n$. Dans tout cette section on prendra la norme euclidienne ou norme 2 pour :

1. En réalité la notion de distance suffirait ici. On retrouve des distances qui ne sont pas induites d'une norme dans des contextes proches en Machine Learning.

- d'une part sa « naturalité » : c'est la norme portée par l'intuition géométrique immédiate,
- d'autre part on peut montrer que c'est la seule norme (vectorielle) invariante par changement de repère orthonormé. Cette dernière propriété nous garantit que la forme du nuage de points incarnés par les lignes de la matrice R préserve la mesure de sa forme après changement de repère orthonormé. Par exemple on peut faire « tourner » ensemble tous les points du nuage de points sans en modifier sa forme (forme mesurée par la distance euclidienne).

Ainsi on définit la distance entre l'individu Ind_i et Ind_j par :

$$d(Ind_i, Ind_j) := \|x_{i\bullet} - x_{j\bullet}\|_2 = \sqrt{\sum_{k=1}^n (x_{ik} - x_{jk})^2}$$

où l'on a noté : $x_{i\bullet}$ le vecteur ligne : $(x_{i1}, \dots, x_{in})$. Or en faisant cela on accorde une même importance aux différentes variables, ce qui peut poser d'important problème dans la pratique. Par exemple les unités en lesquelles sont mesurées les colonnes peuvent avoir des ordres de grandeurs trop différents. Les grandes valeurs numériques peuvent masquer les petites valeurs. Afin d'éviter cela lorsque l'on réalise une ACP on commence par centrer les données puis, et c'est le cas le plus courant, on réduira nos données aux sens que l'on forcera la norme à être égale à 1 pour chaque colonne (=chaque variable). On parlera dans ce cas d'**ACP normée**. Comme c'est le cas plus courant en pratique nous ferons le raccourci suivant : une ACP pour nous sera une ACP normée, sauf mention explicite du contraire.

Lorsque l'on étudie des données disons par exemple les peintures d'un groupe d'individus ce n'est pas tant les valeurs brutes qui sont porteuses de sens et d'information mais leurs écarts à la moyenne. Par analogie avec un système physique si l'on pense la moyenne comme l'état « stable », la physique apparaît par l'écart à cet état stable. Pour un objet mobile c'est l'écart à son état d'inertie qui définit la notion de force.

Commençons donc par supprimer des données leurs moyennes : c'est ce que l'appelle l'opération de **centrage**. L'idée du centrage des données est de transformer une matrice $R \in \mathcal{M}_{mn}(\mathbb{R})$ en une matrice $R^{(c)} \in \mathcal{M}_{mn}(\mathbb{R})$ dont la moyenne des vecteurs lignes est nulle. Afin de réaliser cela nous allons chercher le barycentre dans $\mathbb{R}^n$ des points données, ce barycentre représente l'individu idéal moyen, idéal car il n'est souvent incarné exactement par aucun des individus du tableau. Notons $\overline{x_{\bullet}}$ le vecteur ligne moyen de R (c'est l'individu « moyen ») :

$$\overline{x_{\bullet}} = \frac{1}{m} \sum_{i=1}^m x_{i\bullet} \in \mathcal{M}_{1,n}(\mathbb{R})$$

où $x_{i\bullet}$ représentent les vecteurs lignes de R . Matriciellement cela revient à faire le calcul suivant :

$$\overline{x_{\bullet}} = \frac{1}{m} 1_m^T R$$

où 1_m est le vecteur colonne de taille m ne contenant que des 1. On peut alors définir la matrice centrée par :

$$R^{(c)} := \begin{pmatrix} x_{1\bullet} - \overline{x_{\bullet}} \\ \vdots \\ x_{m\bullet} - \overline{x_{\bullet}} \end{pmatrix}$$

ce qui s'écrit encore matriciellement :

$$R^{(c)} := R - \frac{1}{m} 1_m 1_m^T R$$

Nous allons réduire à présent ce tableau de données, i.e. normaliser la dispersion des données à 1 et rendre indépendant de l'échelle (i.e. l'unité de mesure). Pour cela on définit ainsi la version centrée-réduite de R par :

$$R^{(cr)} := R^{(c)} D^{-1}$$

où D est la matrice diagonale de taille n dont les termes diagonaux sont par (car ici la matrice est centrée sur les lignes) :

$$d_{ii} := S_{\bullet i} := \|x_{\bullet i}^{(c)}\| = \sqrt{\sum_{k=1}^m (x_{ki}^{(c)})^2}$$

où les termes $x_{ki}^{(c)}$ note les termes (k, i) de $R^{(c)}$. Le seul cas où cette matrice D ne serait pas inversible est si les termes d'une colonne posséderaient tous une même valeur. Or une telle colonne n'apporterait aucune information à notre analyse et peut être supprimée.

Disons deux mots sur cette convention. Elle nous permet d'avoir pour norme de Frobenius de $R^{(cr)}$:

$$\|R^{(cr)}\|_F^2 = \sum_{j=1}^n \|R_j^{(cr)}\|_2^2 = \sum_{j=1}^n 1 = n$$

On retrouve le nombre de colonne du tableau i.e. le nombre de variable aléatoire pris en compte pour la construction du jeu de données. En liant cela avec les valeurs singulières on a finalement :

$$\sum_{i=1}^r \sigma_i^2 = n$$

où les σ_i sont les valeurs singulières de $R^{(cr)}$.

Dans tout ce chapitre on considérera donc que la matrice R est une matrice centrée-réduite avec le processus décrit plus haut.

L'idée de l'ACP est de trouver un changement de repère orthonormée (i.e. qui ne change pas la mesure euclidienne de la « forme » du nuage de point) pour les colonnes de la matrice i.e. de trouver une matrice $V \in \mathbb{O}_n$ telle que la matrice résultante :

$$P := RV$$

soit une matrice dont les vecteurs sont orthogonaux entre eux i.e. que les nouvelles variables aléatoires soient décorréliées et telle que l'information contenu dans P soit hiérarchisée/ordonnée i.e en notant p_i la colonne i de P :

$$\|p_1\|_2 \geq \|p_2\|_2 \geq \dots \geq \|p_n\|_2$$

La condition d'orthogonalité permet de s'assurer de la non redondance de l'information portée par les colonnes de P et la condition sur la hiérarchisation permet quant à elle donner un sens à : « ce qui est une information importante et les détails négligeables ». En effet en revenant à ce que nous avons défini comme mesure de la similitude entre les individus (incarnées par les lignes de la matrice R) on a :

$$d(Ind_i, Ind_j) := \|x_{i\bullet} - x_{j\bullet}\|_2 = \|p_{i\bullet} - p_{j\bullet}\|_2 \geq d^{(\alpha)}(Ind_i, Ind_j) := \|p_{i\bullet}^{(\alpha)} - p_{j\bullet}^{(\alpha)}\|_2$$

où : $p_{i\bullet}$ note la ligne i de P et $p_{i\bullet}^{(\alpha)}$ note la ligne i de la matrice $P^{(\alpha)} \in \mathcal{M}_{m,\alpha}(\mathbb{R})$ composée des $1 \leq \alpha \leq n$ premières colonnes de P . En résolvant le problème précédent on pourra définir de manière rigoureuse grâce la distance $d^{(\alpha)}(\bullet, \bullet)$ une **notion de ressemblance entre les individus qui néglige des détails sur ces individus**.

2.3.2. Une SVD pour résoudre l'ACP

Avec les propriétés que nous avons vu sur la décomposition en valeurs singulières une solution au problème précédent est évidente : c'est l'exacte application du théorème d'Eckart-Young-Mirsky. Soit en effet une SVD de R (qui on le rappelle est centrée-réduite) on a :

$$R = U\Sigma V^T$$

En choisissant (hasard des notations....) pour matrice V dans le problème précédent la matrice V donnée par cette SVD on obtient exactement ce que l'on cherche i.e. :

$$P = RV = (U\Sigma V^T)V = U\Sigma = \begin{pmatrix} \sigma_1 u_1 & \dots & \sigma_r u_r & 0 & \dots & 0 \end{pmatrix}$$

En effet comme U est une matrice orthogonale ses vecteurs colonnes sont orthogonaux (et même orthonormés) entre eux et l'on a :

$$\|p_1\|_2 = \|\sigma_1 u_1\| = \sigma_1 \geq \|p_2\|_2 = \|\sigma_2 u_2\| = \sigma_2 \geq \dots \geq \|p_r\|_2 = \|\sigma_r u_r\| = \sigma_r \geq \|p_{r+1}\|_2 = 0 \geq \dots \geq \|p_n\|_2 = 0$$

où les σ_i sont les valeurs singulières de R et r le rang de cette matrice. Soit les deux conditions que nous demandions.

En résumé faire une ACP sur un tableau de données de type individus/variables quantitatives consiste à :

1. centrer-réduire le tableau de données considéré.
2. En notant R la matrice centrée-réduite précédente calculer une SVD de R :

$$R = U\Sigma V^T$$

3. Choisir un ordre de projection : $1 \leq \alpha \leq n$ et considérer la matrice :

$$P^{(\alpha)} := \begin{pmatrix} \sigma_1 u_1 & \dots & \sigma_\alpha u_\alpha \end{pmatrix}$$

où les σ_i désignent les valeurs singulières de R et u_i les colonnes i de U .

Dans les sections précédentes nous avons vu l'interprétation en terme de projecteur du théorème d'Eckart-Young-Mirsky. Utilisons cela interpréter l'ACP.

Tout d'abord on voit apparaître deux espaces vectoriels importantes :

- **l'espace des individus (lignes)**. En effet $P^{(\alpha)}$ donne exactement les coordonnées de R dans sa projection sur $\mathcal{E}_\alpha$ pour $V_\alpha = (v_1, \dots, v_\alpha)$. En effet on a :

$$\Phi_\alpha(R) = RV_\alpha V_\alpha^T = P^{(\alpha)} V_\alpha^T$$

En passant les lignes de $P^{(\alpha)}$ comme des coordonnées, on peut interpréter géométriquement notre ACP. Nous verrons que la pratique $\alpha = 2$ ou 3 , ce qui permet d'avoir une représentation visuelle de ces coordonnées. Comme V_α est issu d'une SVD c'est le meilleur choix d'espace vectoriel où projeter nos données en conservant au maximum l'information porté par R . On parlera des lignes de $P^{(\alpha)}$ comme des **coordonnées factorielles des individus** (ou lignes).

- **l'espace des variables (colonnes)**. Par symétrie, on a vu qu'il était équivalent de regarder les coordonnées de R dans sa projection sur $\mathcal{D}_\alpha$ pour $U_\alpha = (u_1, \dots, u_\alpha)$. En effet on a :

$$\Phi^\alpha(R) = U_\alpha U_\alpha^T R = U_\alpha (U_\alpha^T R)$$

En posant :

$$G^{(\alpha)} := (U_\alpha^T R)^T = R^T U_\alpha = V_\alpha \Sigma_\alpha$$

cette matrice s'interprète comme les **coordonnées factorielles des variables**. En général ici on prendra $\alpha = r$ afin d'analyser toutes les variables aléatoires entre elles. Ces coordonnées conduisent à ce que l'on appelle le **cercle des corrélations**, cercle pour sa représentation lorsque $r = 2$, le cas le plus étudié en pratique. La matrice on obtient une représentation de la contribution des anciens variables par rapport aux nouvelles en des points de la boule unité. En résumé on tracera en dimension 2 (le plus commun) la ligne j de $G = G^{(r)}$:

$$G_{j,\bullet} := (\sigma_1(v_1)_j, \sigma_2(v_2)_j, \dots, \sigma_r(v_r)_j)$$

avec $j \in \llbracket 1, n \rrbracket$ et $(v_k)_j$ note la composante j du vecteur v_k . C'est une représentation qui permet voir à quel point les anciennes colonnes de R i.e. les variables aléatoires considérés s'incarnent ou non dans les nouveaux axes v_i . Dans cette représentation plus un point sera proche d'un axe mieux la variable sera expliquée par cet axe. Ce vecteur $G_{j,\bullet}$ appartient bien à un « cercle ». En effet comme R a été normalisée on a : $(R^T R)_{jj} = \|R_{\bullet,j}\|_2^2 = 1$ et par suite comme :

$$R^T R = V \Sigma^T \Sigma V^T$$

on a :

$$(R^T R)_{jj} = \sum_{i=1}^r \sigma_i^2 (v_i)_j^2 = \|G_{j,\bullet}\|_2^2 = 1$$

Si l'on considère la matrice V , les coefficients $(v_k)_j$ s'écrivent $(v_k)_j = v_{j,k}$.

La symétrie d'analyse en les lignes et les colonne est subsumée par les valeurs singulières. En effet on a vu que :

$$\|\Phi^\alpha(R)\|_F^2 = \|\Phi_\alpha(R)\|_F^2 = \sum_{i=1}^\alpha \sigma_i^2$$

et par suite l'interprétation en terme de mesure d'information par les valeurs singulières nous fourni un critère simple : la **qualité de projection (ligne ou colonne) des données sur l'axe j** est donnée par :

$$I_j := \frac{\sigma_j^2}{\sum_{i=1}^r \sigma_i^2} = \frac{\sigma_j^2}{\|R\|_F^2} = \frac{\sigma_j^2}{n}$$

On appelle généralement en statistique le terme σ_j^2 **l'inertie projeté** et $\sum_{i=1}^r \sigma_i^2$ **l'inertie totale**. Le terme I_j désigne ainsi la qualité de la projection (= la représentation), on l'appelle souvent pourcentage d'inertie associé à un l'axe j (ici). C'est un nombre compris entre 0 et 1 qui peut donc s'interpréter facilement en pourcentage.

Enfin en utilisant ces deux types de coordonnées on peut définir des mesures de la qualité de représentation et la contribution. Dans la suite les vecteurs u_i, v_i et les termes σ_i sont ceux d'une SVD de R (centrée-réduite toujours) :

- Pour l'individu sur la ligne i , ind_i on définit sa **qualité de représentation** sur l'axe un $k \leq r$ par :

$$\cos_{Ind}^2(i, k) := \frac{\sigma_k^2 u_{ik}^2}{\sum_{j=1}^r \sigma_j^2 u_{ij}^2} = \frac{\sigma_k^2 u_{ik}^2}{\|P_{i,\bullet}\|_2^2}$$

où $P_{i,\bullet}$ note la ligne i de $P = U\Sigma$. Plus cette valeur est proche de 1, meilleure est sa représentation sur l'axe k . On a aussi une formule parfaitement équivalente pour la qualité de représentation des variables aléatoires en remplaçant u_i par v_i et P par $G := R^T U$.

- De la même manière on peut s'intéresser à la contribution d'un individu (ou d'une variable) à un axe i.e. dans quelle mesure il contribue à la formation de l'axe considéré :

Í Pour l'individu sur la ligne i , Ind_i on définit sa **contribution relative à l'axe k** pour un $k \leq r$ par :

$$CTR_{Ind}(i, k) := u_{ik}^2$$

En effet comme la matrice U est orthogonale, il vient que $\sum_{i=1}^m u_{ik}^2 = 1$ et si chaque individu jouait un rôle géométriquement symétrique et équivalent dans la définition de l'axe k , l'inertie serait répartie de manière parfaitement uniforme i.e. $u_{ik}^2 \simeq \frac{1}{m}$. Ainsi dans la pratique on mesure plutôt l'écart de $CTR_{Ind}(i, k)$ avec $\frac{1}{m}$. Si pour un individus le $CTR_{Ind}(i, k) \gg \frac{1}{m}$, cela signifie que l'axe k capture presque exclusivement l'individu i , lui accordant une importance démesurée.

Î Pour une variable j on définit sa **contribution relative à l'axe k** pour un $k \leq r$ par :

$$CTR_{Var}(j, k) := v_{jk}^2$$

Ces différents critères permettent de sélectionner (ou exclure) des individus ou des variables. En effet on peut souhaiter :

1. écarter des individus « atypiques » ou « peu informatifs » :

- Parfois, on identifie des individus qui ont une très faible qualité de représentation ($\cos^2$) sur les axes d'intérêt (par ex., les deux premiers axes). S'ils sont mal « projetés » cela peut signifier qu'ils contiennent des informations qui se « dispersent » sur d'autres axes plus faibles ou qu'ils se comportent comme des « individus » atypiques.
- À l'inverse, des individus peuvent avoir une contribution très forte à un axe, « monopolisant » la variance de cet axe. Ils sont souvent « extrêmes » et peuvent biaiser la représentation des autres. Dans ce cas, on peut se poser la question de les retirer pour faire une analyse « plus homogène ».

2. écarter des variables « redondantes » ou « non explicatives » :

- Si une variable a une qualité de représentation quasi nulle sur tous les axes d'intérêt (i.e. les axes conservés par l'ACP), elle n'apporte pratiquement rien dans l'espace factoriel. On peut se demander si elle est utile.
- Si une variable a un fort taux de corrélation / redondance avec une autre (et donc la même contribution sur l'axe), on pourrait décider de ne conserver qu'une variable représentative pour ne pas surcharger l'analyse.

Pour réaliser ce type de « nettoyage » on fixe souvent des seuils, seuils contenant toujours de l'arbitraire malheureusement. Il faut donc être très vigilant lorsque l'on souhaite faire cela, en particulier lorsqu'on souhaite réduire « intelligemment » des bases de données trop volumineuses.

2.3.3. Cas d'un individus en plus

Dans cette courte section nous allons considérer la situation suivante. Supposons que nous ayons déjà effectué une ACP sur un jeu de données. Si à présent je souhaite ajouter un individu en plus à mon analyse (i.e. rajouter une ligne à R), je dois refaire totalement une ACP. Or si je considère que les individus utilisés pour ma première ACP sont assez représentatifs je peux choisir de simplement projeter les données de mon nouvel individu dans la base V_α . Formellement cela revient à prendre un individu nouveau i.e. un vecteur ligne de $ind_{new} \in \mathbb{R}^n$ puis à utiliser le centrage et la réduction précédente puis de la changer dans la base V_α i.e. :

$$Proj_\alpha(ind_{new}) := \left(ind_{new} - \frac{1}{m} 1_m^T R_{brut} \right) D^{-1} V_\alpha$$

où R_{brut} est la matrice des données avant centrage et réduction.

Nous avons que les valeurs singulières sont une mesure robuste de l'information contenu dans une matrice. Ainsi l'ajout d'un individu en plus par cette projection fait sens car la structure des valeurs singulières est préservée par la suppression ou l'ajout de ligne/colonne.

2.3.4. Un exemple d’ACP pour comprendre

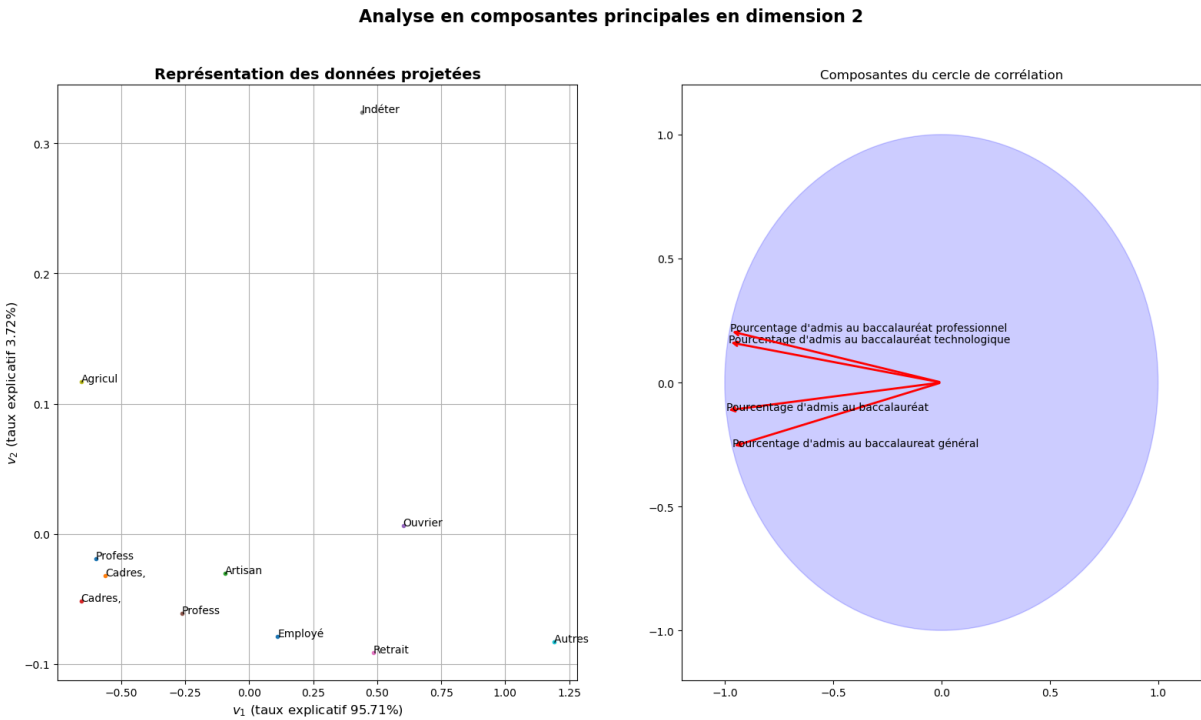
Même si l’ACP est d’autant plus intéressante que le nombre de donnée est grand, nous allons illustrer rapidement (on verra des exemples en TD) les formules précédentes sur un « petit » exemple simple issu du jeu de données : *Réussite au baccalauréat selon l’origine sociale*, du site data.education.gouv.fr². Voici le tableau pour les données de 2025 :

Origine sociale	% d’admis au bac. général	% d’admis au bac. techn.	% d’admis au bac. prof.	% d’admis au bac.
Prof. intermédiaires : instituteurs et assimilés	98.4	95.1	91.0	97.3
Cadres, professions intellectuelles supérieures	98.3	95.0	90.4	97.1
Artisans, commerçants, chefs d’entreprise	96.3	91.8	87.3	93.3
Cadres, professions intellec. sup. : prof. et assimilés	98.8	95.4	91.0	98.0
Ouvriers	93.1	87.9	82.2	87.5
Professions intermédiaires	97.3	93.1	87.9	94.5
Retraités	94.0	88.3	81.9	89.3
Indéterminé	91.7	90.7	85.7	87.8
Agriculteurs exploitants	98.1	97.1	91.7	96.3
Autres personnes sans activité professionnelle	90.9	84.8	75.8	83.5
Employés	95.8	91.3	84.3	91.5

La matrice à étudier est la partie numérique du tableau précédent. Après avoir centrée-réduit les données (même si ici les données sont de types homogènes, ce qui rend l’opération de réduction moins « forte »), on trouve numériquement les valeurs singulières :

$\sigma_1 = 1.95661064, \sigma_2 = 0.38573443, \sigma_3 = 0.13988944, \sigma_4 = 0.05757356$

On remarque la décroissant rapide de celles-ci. Cette décroissance rapide nous permet de ne considérer que les deux premiers axes i.e. les deux premiers axes portés par les deux plus grandes valeurs singulières. On peut alors tracer le plan des coordonnées factorielles des individus, qui est réellement un plan ici car nous prenons les deux premiers axes uniquement ainsi que le cercle de corrélation. On obtient la sortie graphique suivante :



2. Le lien est le suivant : <https://data.education.gouv.fr/explore/assets/fr-en-reussite-au-baccalaureat-origine-sociale/>

On voit que le premier axe v_1 porte 95,71% de l'information du tableau (et 3,72% pour le second). Ainsi cette projection 2D permet de visualiser plus de 99% de l'information portée par ce tableau 4D i.e. que la forme du nuage de points 4D représentée par le tableau original est très correctement représentée sur le plan factoriel des individus trouvé grâce à notre SVD (et par suite notre ACP).

Les individus (ici les origines sociales) sont projetés dans ce plan et l'on voit apparaître des rapprochements entre certaines origines sociales et ces rapprochements se lisent grâce au cercle de corrélation (on rappelle qu'il interne lui l'information contenu par les nouveaux axes d'analyses v_1 et v_2). L'axe 1 indique clairement la distinction entre la réussite (à gauche) et l'échec aux bacs alors que l'axe 2, l'axe vertical, indique quand à lui la différence entre les bacs professionnel/technique et le bac général. Ainsi les origines sociales « privilégiées » sont retrouvées (plan factoriel des individus) et ce regroupement est lié à la réussite aux différents bacs, en particulier général (cercle de corrélation).

On peut mesurer la qualité de représentation de l'axe 1 (puisque c'est lui qui contient la quasi totalité de l'information). Cette quantité répond à la question : dans quelle mesure l'axe k résume-t-il la position de l'individu i ? :

Origine sociale	$\cos^2_{Ind}(i, 1)$
Prof. intermédiaires : instituteurs et assimilés	0.996601
Cadres, professions intellectuelles supérieures	0.9956
Artisans, commerçants, chefs d'entreprise	0.703891
Cadres, professions intellec. sup. : prof. et assimilés	0.992066
Ouvriers	0.992585
Professions intermédiaires	0.945897
Retraités	0.962445
Indéterminé	0.646033
Agriculteurs exploitants	0.95222
Autres personnes sans activité professionnelle	0.993365
Employés	0.56086

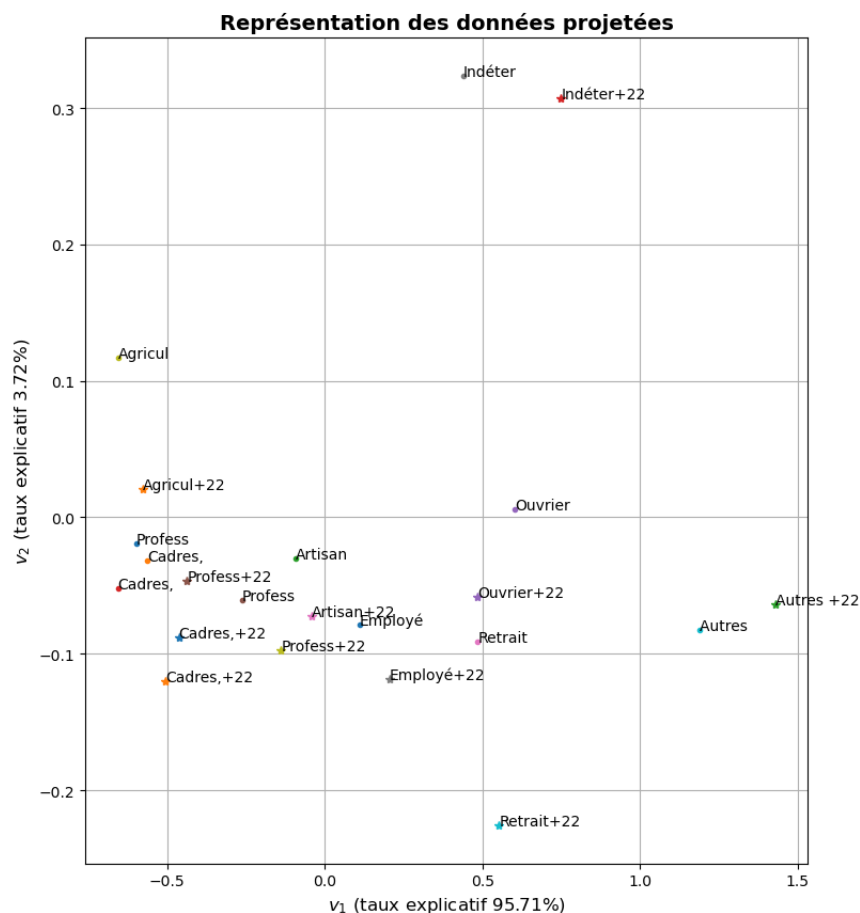
On voit que l'axe 1 capture très l'information de presque toutes les origines sociales sauf pour les catégories : Employés, Indéterminé et dans une moindre mesure : Artisans, commerçants, chefs d'entreprise. Pour ces catégories c'est donc un mélange entre l'axe 2 et cet axe 1 qui permet de décrire les données (même si l'axe 1 est majoritaire) et non uniquement la réussite ou non aux bacs. Par contre pour la catégorie Ouvriers et Autres personnes sans activité, deux catégories bien décrite par l'axe 1, c'est leurs non-réussites aux bacs qui les caractérisent (et les oppose aux catégories plus privilégiées).

Regardons à présent la contribution relative CTR_{ind} i.e. répondons à la question suivante : dans quelle mesure l'individu i est-il responsable de l'existence et de l'orientation de l'axe k ? Regardons sur nos données le résultat :

Origine sociale	$CTR_{Ind}(i, 1)$	$CTR_{Ind}(i, 1) - \frac{1}{m}$
Prof. intermédiaires : instituteurs et assimilés	0.0934274	0.00251832
Cadres, professions intellectuelles supérieures	0.0826775	-0.00823154
Artisans, commerçants, chefs d'entreprise	0.00226723	-0.0886419
Cadres, professions intellec. sup. : prof. et assimilés	0.112364	0.0214546
Ouvriers	0.0945688	0.00365976
Professions intermédiaires	0.0178914	-0.0730177
Retraités	0.0615156	-0.0293935
Indéterminé	0.0503348	-0.0405743
Agriculteurs exploitants	0.112047	0.0211375
Autres personnes sans activité professionnelle	0.369706	0.278797
Employés	0.00320112	-0.087708

On voit que la catégorie : Autres personnes sans activité professionnelle, possède une contribution relative importante par rapport à l'axe 1. Le risque d'une valeur relativement importante est que l'axe 1 ne décrive pas une tendance/structure est une opposition entre cette catégorie « extrême » et le reste des catégories. Dans la pratique on peut être amené à réaliser une nouvelle ACP en supprimant cet individu (ici catégorie). Ici en reprenant l'ACP sans cette catégorie on retrouve les mêmes conclusions que précédemment (Exercice : Faîte le).

Enfin voyons le problème suivant : comment comparer notre analyse de l'année 2025 avec, disons, l'année 2022 ? C'est ici que l'on utilise la formule de projection vue dans la section précédente. On obtient alors la sortie suivante où +22 est accolée aux « labels » de catégorie pour différencier les données de 2022 projetés :



On remarque que la structure des données de 2025 est très similaire à la structure des données de 2022 : l'axe 1 qui marque les différences de réussite est toujours aussi classifiant pour les origines sociales entre 2022 et 2025.

L'exemple de l'analyse procustéenne linéaire

3.1. L'analyse procustéenne linéaire

Dans ce chapitre nous allons nous intéresser au problème de la recherche de la meilleure transformation affine reliant deux jeux de données. Ce problème qui peut sembler très particulier ne l'est en réalité pas en pratique. En effet beaucoup de situations concrètes s'approximent de manière satisfaisante (quant au besoin et à l'usage comme dirait Aristote) par un problème affine.

Commençons par un peu d'histoire.... car la méthode que nous allons voir s'inspire de la mythologie en particulier du personnage de Procuste. Procuste est dans la mythologie grecque, un fils de Poséidon dont l'activité principale est le « brigandage sadique », activité que l'on peut résumer par la pratique coutumière du personnage : possédant deux lits, un petit et un grand, il fait subir à ses victimes un châtement spécial. Si le malheureux est grand, il le couche sur le lit le plus petit et coupe les extrémités de son corps qui dépassent, s'il est petit, il le couche dans le grand lit et lui martèle les jambes pour les étirer jusqu'à atteindre les bords du grand lit. Thésée le tue en lui faisant subir son propre supplice. Cette histoire matelassière a donné naissance à une expression critique : « coucher sur un lit de Procuste », qui signifie l'acte de tordre les données, les faits, pour les adapter à un préconçu. De quoi briller dans vos prochaines copies de sciences humaines...

À l'image de Procuste, l'analyse procustéenne consiste à « faire prendre à des données une forme désirée » (exemple : médecine, biologie, calibrage de mesure, géodésie, reconnaissance de formes, etc...). Dans la suite de cette section nous nous limiterons à l'analyse procustéenne « linéaire » en recherchant la meilleure transformation affine liant deux jeux de données.



FIGURE 3.1.1. – Un exemple de mise en pratique musclée de l'analyse procustéenne....

3.1.1. Résolution théorique de l'analyse procustéenne

3.1.1.1. Pour bien débiter...

D'un point de vue formel, l'analyse procustéenne linéaire consiste à rechercher une correspondance affine ϕ entre deux ensembles de n points de $\mathbb{R}^m$, en demandant que cette correspondance se fasse point par point :

$$a_j \in \mathbb{R}^m \rightsquigarrow_{\phi} b_j \in \mathbb{R}^m$$

On cherche alors à minimiser la quantité :

$$\sum_{j=1}^n \|b_j - \phi(a_j)\|_2^2 = \sum_{j=1}^n \sum_{i=1}^m (b_{ij} - \phi(a_j)_i)^2$$

La quantité précédente s'écrit matriciellement sous la forme suivante :

$$M(\Phi(A)) := \|B - \Phi(A)\|_F^2$$

où : $B \in \mathcal{M}_{m,n}(\mathbb{R})$ est composée des vecteurs b_j et $\Phi(A) \in \mathcal{M}_{m,n}(\mathbb{R})$ est composée des vecteurs $\phi(a_j)$ i.e. :

- B est la matrice dont les vecteurs colonnes sont : $B := (b_1 \ b_2 \ \cdots \ b_n)$,
- Φ est la matrice dont les vecteurs colonnes sont : $\Phi := (\phi(a_1) \ \phi(a_2) \ \cdots \ \phi(a_n))$.

Avant de traiter le cas général il peut être intéressant de commencer par traiter un cas particulier.

3.1.1.2. Le cas d'une transformation orthogonale

Commençons par le cas simple suivant. On suppose que Φ est une transformation orthogonale i.e qu'il existe une matrice orthogonale X telle que :

$$\Phi(A) := XA$$

où A est la matrice de taille m, n dont les vecteurs colonne sont les a_j . Si $A, B \in \mathcal{M}_{m,n}(\mathbb{R})$, on doit donc résoudre :

$$\arg \min_{X^T X = I_m} \|B - XA\|_F^2$$

On notera la fonction à minimiser :

$$M(X) := \|B - XA\|_F^2$$

Or :

$$\|B - XA\|_F^2 = \|B\|_F^2 + \|XA\|_F^2 - 2\text{trace}(B^T XA)$$

Comme $X \in \mathbb{O}_m$, on a donc : $\|XA\|_F^2 = \|A\|_F^2$.

Ainsi trouver le minimum de M revient ainsi à maximiser :

$$\text{trace}(B^T XA) = \text{trace}(XAB^T)$$

puisque B et A sont données.

Prenons une SVD de $AB^T \in \mathcal{M}_m(\mathbb{R})$:

$$AB^T = U\Sigma V^T$$

et posons (changement de variable) :

$$X = VYU^T$$

avec $Y \in \mathbb{O}_m$ à déterminer. En effet $Y \in \mathbb{O}_m$ car X, V et U sont orthogonales également.

On a alors :

$$\begin{aligned} \text{trace}(XAB^T) &= \text{trace}(VYU^T U \Sigma V^T) \\ &= \text{trace}(VY \Sigma V^T) \\ &= \text{trace}(V^T V Y \Sigma) \\ &= \text{trace}(Y \Sigma) \end{aligned}$$

En notant σ_i les valeurs singulières de AB^T :

$$\text{trace}(Y \Sigma) = \sum_{i=1}^{rg(AB^T)} \sigma_i y_{ii} \leq \sum_{i=1}^{rg(AB^T)} \sigma_i$$

car $|y_{ii}| \leq 1$ (car Y matrice orthogonale).

Ainsi la borne supérieure est atteinte pour $Y = I_m$ et donc la matrice solution qui minimise M est :

$$\boxed{X = VU^T}$$

En résumer la solution du problème particulier est donnée par :

$$\boxed{VU^T = \arg \min_{X^T X = I_m} \|B - XA\|_F^2}$$

où les matrice U et V sont les matrices données par une SVD de $AB^T := U\Sigma V^T$.

Exemple 6. Cet exemple est important. Soit $C \in \mathcal{M}_n(\mathbb{R})$. On cherche les matrices orthogonales X qui approchent (au sens de la norme de Frobenius) le plus C . Ce problème revient au cas précédent avec $A = I_n$ et $B = C$:

$$\arg \min_{X^T X = I_n} \|C - X\|_F^2$$

et donc la solution est donnée par :

$$X = UV^T$$

où U et V interviennent dans la SVD de $C = U\Sigma V^T$. Ainsi en résumé la meilleure approximation orthogonale (au sens de la norme de Frobenius) d'une matrice est la donnée d'une décomposition en valeurs singulières « sans » la matrice Σ des valeurs singulières.

Exercice 7. Application (calculs à la main sans logiciel de calcul) : L'approximation orthogonale d'une matrice B revient à trouver une matrice X telle que :

$$\min_{X^T X = I_n} \|B - X\|_F^2$$

1. En déduire de la question 3, une expression de X .
2. On note dans un repère les coordonnées de l'extrémité d'un bras robotique à un temps t :

$$P_t := (x_t, y_t, z_t)^T$$

On cherche à calibrer les éléments mécaniques de notre robot. Entre deux instants t_1 et t_2 on cherche à faire effectuer une rotation par le bras robotique. Les éléments mécaniques du robot donnent comme transformation :

$$BP_{t_1} = P_{t_2}$$

où B est la matrice :

$$B := \begin{pmatrix} \sqrt{2} & 0 & \sqrt{2} \\ 0 & \frac{\sqrt{2}}{3} & 0 \\ 0 & \frac{\sqrt{2}}{3} & 0 \end{pmatrix}$$

- a) Justifier que cette matrice n'est pas orthogonale.
- b) Calculer à la main (i.e. sans utilisation d'un logiciel de calcul) l'approximation orthogonale de la matrice B .
- c) En déduire suivant les axes x, y, z les angles de rotation approchés du bras robotique. On peut se servir de cette analyse afin de calibrer le robot.

3.1.1.3. Le cas d'une transformation affine

Supposons à présent que Φ soit une transformation affine i.e. qu'elle s'écrit matriciellement :

$$\Phi(A) := \lambda XA + t1_n^T$$

où les inconnues sont :

- $\lambda \in \mathbb{R}$ le rapport d'homothétie,
- $X \in \mathbb{O}_m$, une transformation orthogonale,
- $t \in \mathbb{R}^m$ est un vecteur de translation (vu comme vecteur colonne) et 1_n le vecteur (vecteur colonne) de $\mathbb{R}^n$ ne comportant que des 1.

On supposera que $n \geq 2$.

Dans cette partie nous allons résoudre ce problème (cela paraît miraculeux mais oui, ce problème se résout!) :

$$\arg \min_{\lambda \in \mathbb{R}, X \in \mathbb{O}_m, t \in \mathbb{R}^m} \|B - \Phi(A)\|_F^2$$

Comme dans le cas précédent on doit minimiser la quantité :

$$\|B - \Phi(A)\|_F^2$$

et :

$$\|B - \Phi(A)\|_F^2 = \|B\|_F^2 + \|\lambda XA + t1_n^T\|_F^2 - 2\text{trace}(B^T(\lambda XA + t1_n^T))$$

en notant :

$$M(\lambda, X, t) = \|B - \Phi(A)\|_F^2 - \|B\|_F^2$$

Il nous faut donc minimiser :

$$M = \lambda^2 \|A\|_F^2 + \|t1_n^T\|_F^2 + 2\text{trace}((\lambda A^T X^T - B^T) t1_n^T) - 2\lambda \text{trace}(B^T X A)$$

Comme :

$$\|t1_n^T\|_F^2 = n \sum_{i=1}^m t_i^2$$

et :

$$\text{trace}((\lambda A^T - B^T) t1_n^T) = \sum_{i=1}^m \eta_i t_i$$

$$\text{où } \eta = \sum_{j=1}^n (\lambda X a_j - b_j) \in \mathbb{R}^m.$$

Or un minimum est un point critique donc : $\frac{\partial M}{\partial t_i} = 0, \forall i \in \llbracket 1, m \rrbracket$, il vient donc que :

$$t_i = -\frac{\eta_i}{n}$$

Ainsi :

$$t = b_G - \lambda X a_G$$

où :

$$b_G = \frac{1}{n} \sum_{j=1}^n b_j \quad \text{et} \quad a_G = \frac{1}{n} \sum_{j=1}^n a_j$$

sont les centres de gravité des nuages de points B et A .

Centrons notre problème aux **centres de gravité des nuages de points** en posant :

$$A_G = A - a_G 1_n^T \quad \text{et} \quad B_G = B - b_G 1_n^T$$

Il vient :

$$\begin{aligned} M &= \|B - \Phi(A)\|_F^2 \\ &= \|B_G + b_G 1_n^T - (\lambda X(A_G + a_G 1_n^T) + (b_G - \lambda X a_G) 1_n^T)\|_F^2 \\ &= \|B_G - \lambda X A_G\|_F^2 \\ &= \|B_G\|_F^2 + \lambda^2 \|A_G\|_F^2 - 2\lambda \text{trace}(B_G^T X A_G) \end{aligned}$$

Puis au minimum : $\frac{\partial M}{\partial \lambda} = 0$, on trouve :

$$\lambda = \frac{\text{trace}(B_G^T X A_G)}{\|A_G\|_F^2}$$

et donc on est conduit à minimiser :

$$M = \|B_G\|_F^2 - \frac{\text{trace}(B_G^T X A_G)^2}{\|A_G\|_F^2}$$

Ce qui revient à maximiser la quantité :

$$|\text{trace}(B_G^T X A_G)|$$

comme au problème précédent où Φ était une transformation orthogonale.

Les deux solutions pour X sont donc pour une SVD : $A_G B_G^T = U_G \Sigma_G V_G^T$:

$$X = \begin{cases} V_G U_G^T \\ -V_G U_G^T \end{cases}$$

et :

$$\boxed{t = b_G - \lambda X a_G} \quad \text{et} \quad \boxed{\lambda = \frac{\text{trace}(B_G^T X A_G)}{\|A_G\|_F^2} = \frac{\text{trace}(\Sigma_G)}{\|A_G\|_F^2}}$$

On choisit en général la solution : $X = V_G U_G^T$ (le signe moins se distribuant dans les autres inconnues).

Résumons nos calculs par le théorème suivant :

Théorème 3.1 : Analyse procustéenne (linéaire)

La solution du problème matriciel suivant problème se résout !) :

$$\arg \min_{\lambda \in \mathbb{R}, X \in \mathbb{O}_m, t \in \mathbb{R}^m} \|B - \Phi(A)\|_F^2$$

où :

$$\Phi(A) := \lambda X A + t 1_n^T$$

est une transformation affine matricielle à déterminer i.e avec pour inconnus :

- $\lambda \in \mathbb{R}$ le rapport d'homothétie,
- $X \in \mathbb{O}_m$, une transformation orthogonale,
- $t \in \mathbb{R}^m$ est un vecteur de translation (vu comme vecteur colonne) et 1_n le vecteur (vecteur colonne) de $\mathbb{R}^n$ ne comportant que des 1.

Une solution à ce problème est donnée par :

$$X = V_G U_G^T$$

et :

$$t = b_G - \lambda X a_G \quad \text{et} \quad \lambda = \frac{\text{trace}(B_G^T X A_G)}{\|A_G\|_F^2} = \frac{\text{trace}(\Sigma_G)}{\|A_G\|_F^2}$$

avec :

$$A_G = A - a_G 1_n^T \quad \text{et} \quad B_G = B - b_G 1_n^T$$

les matrices centrées (pour les colonnes) et une SVD de la matrice $A_G B_G^T$:

$$A_G B_G^T = U_G \Sigma_G V_G^T$$

et où :

$$b_G = \frac{1}{n} \sum_{j=1}^n b_j \quad \text{et} \quad a_G = \frac{1}{n} \sum_{j=1}^n a_j$$

sont les centres de gravité des nuages de points B et A .

Exercice 8. Application (calculs à la main sans logiciel de calcul) : On repère sur des photographies de visages humains de face les points suivants :

- $P_1 := (x_1, y_1)$: centre de l'œil droit.
- $P_2 := (x_2, y_2)$: centre de l'œil gauche.
- $P_3 := (x_3, y_3)$: extrémité basse du nez.
- $P_4 := (x_4, y_4)$: centre de la bouche.

On recherche une transformation affine $\Phi : \mathcal{M}_{2,4}(\mathbb{R}) \rightarrow \mathcal{M}_{2,4}(\mathbb{R})$ telle que pour deux jeux de points repérés :

$$P = \begin{pmatrix} x_1 & x_2 & x_3 & x_4 \\ y_1 & y_2 & y_3 & y_4 \end{pmatrix} \quad \text{et} \quad P' = \begin{pmatrix} x'_1 & x'_2 & x'_3 & x'_4 \\ y'_1 & y'_2 & y'_3 & y'_4 \end{pmatrix}$$

on est « au mieux » :

$$\Phi(P') = P$$

Afin que les calculs ne soient pas trop complexes « à la main », regardons le cas très simplifié :

$$P = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix} \quad \text{et} \quad P' = \begin{pmatrix} 1 & 1 & 1 & -1 \\ 2 & -2 & 0 & 0 \end{pmatrix}$$

- Tracer sur un repère avec deux couleurs différentes, ces deux jeux de points.
- En utilisant l'analyse procustéenne déterminer Φ puis déterminer l'erreur commise.

3.1.2. Exemples d'applications de l'analyse procustéenne avec Python

Dans cette sous-section nous allons implémenter la résolution du problème posé par l'analyse procustéenne dans la partie précédente.

Nous allons commencer par programmer l'analyse procustéenne précédente en rédigeant une fonction **Procuste(A,B)** qui prend en entrée deux matrices de même taille et qui renvoie en sortie le tuple :

$$(\lambda XA + t1_n^T, [\lambda, X, t])$$

dont les valeurs sont données par la solution théorique trouvée du problème de l'analyse procustéenne.

```
1 def procuste(A,B) :
2     m,n=np.shape(B)
3     aG=np.zeros((m,1))
4     for i in range(0,n) :
5         aG=aG+(1./n)*A[:,[i]]
6     bG=np.zeros((m,1))
7     for i in range(n) :
8         bG=bG+(1./n)*B[:,[i]]
9     u=np.ones((1,n))
10    Ag=A-aG@u
11    Bg=B-bG@u
12    P=Ag @Bg.T
13    Ug,Sg,VgT=np.linalg.svd(P)
14    X=VgT.T @Ug.T
15    l=np.trace(np.diag(Sg))/np.linalg.norm(Ag)**2
16    t=bG-l*(X@aG)
17    return l*X@A+t@u, [l,X,t,u]
```

On appellera la quantité : $\|B - \Phi(A)\|_F^2$: **l'erreur de transformation**. C'est une mesure du défaut à être transformée l'une en l'autre par une transformation affine.

Grâce à cette algorithmes nous allons utiliser l'analyse procustéenne afin de comparer des formes à transformation affine près. Une utilisation classique est en biomorphologie de faire l'hypothèse que la variabilité inter-espèce est une relation affine de la morphologie des membres de cette espèce. Par exemple, chez homo sapiens adulte, il y a des grands et des petits (on parle de variabilité) mais les tailles des os restent en « proportion » i.e. qu'un individu « grand » est en quelque sorte la version « transformée affinement » d'un individu « petit ». Inversement, si le résultat de l'analyse procustéenne ne donne pas de résultat « convainquant », on peut faire l'hypothèse de la présence d'espèce différente.

Réalisons une comparaison entre des crânes d'homo sapiens et un homme de Néanderthal.

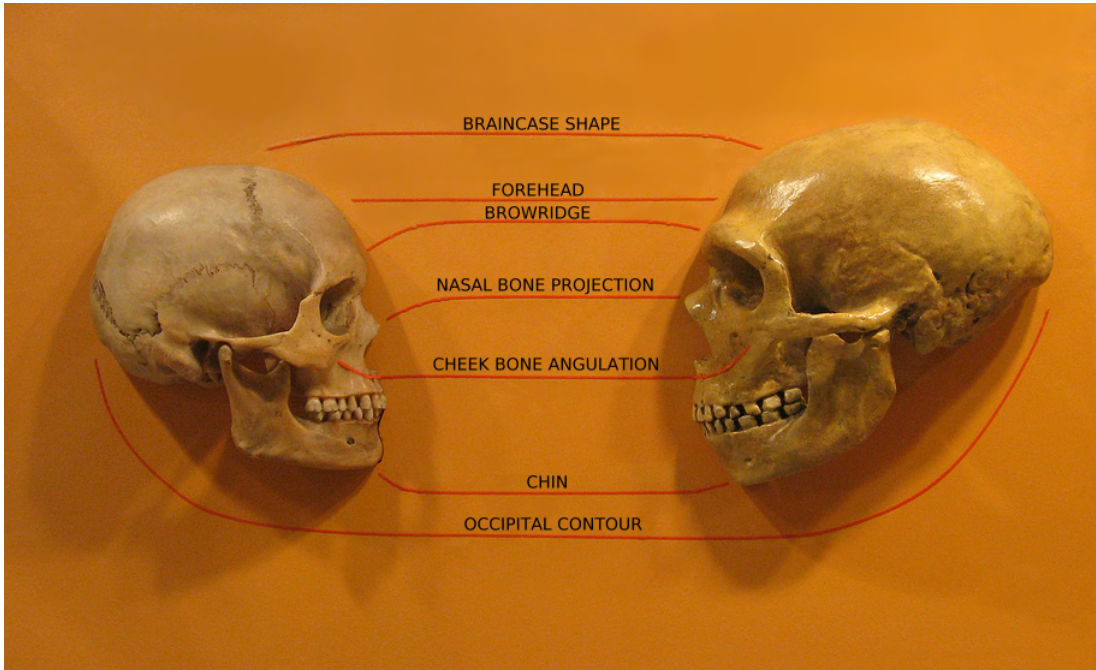


FIGURE 3.1.2. – À gauche : des points marqués sur des points « caractéristiques » d'un crâne d'homo sapiens, à droite, les mêmes points caractéristiques marqués sur un crâne d'homo neanderthalensis.

En guise d'exemple voici un code Python qui permet de réaliser cela. On commence par faire une fonction qui permet de récupérer des points sur une image en utilisant OpenCV.

```
1 import cv2 import numpy as np
2
3 def capture(img, nbpts, taille=(500,500)) :
4     imgcopy=np.copy(img)
5     def points(event,x,y,flags,param):
6         global mouseX,mouseY,cpt
7         if event ==cv2.EVENT_LBUTTONDOWNCLK:
8             cv2.circle(imgcopy,(x,y),5,(255,0,0),-1)
9             cv2.putText(imgcopy,text=f"{cpt}", org=(x+2,y+2), fontFace = cv2.FONT_HERSHEY_DUPLEX,fontScale = 1.0,
10                color = (255, 0, 0), thickness = 2)
11             mouseX,mouseY = x,y
12             L.append([x,y])
13             cpt+=1
14     cv2.namedWindow('image')
15     cv2.setMouseCallback('image',points)
16     global cpt
17     cpt=0
18     L=[]
19     while cpt<nbpts :
20         cv2.imshow('image',imgcopy)
21         k=cv2.waitKey(20)
22         if k == 27:
23             break
24     return np.array(L).T, imgcopy
```

On peut alors marquer les points.

```
1 img2=cv2.imread("Sapiens_neanderthal_comparison.png")
2 img1=cv2.imread("Sapiens_neanderthal_comparison.png")
3 nbpts=7
4 L1,imgcopy1=capture(img1, nbpts)
5 cv2.imshow("ref", imgcopy1)
6 cv2.imwrite("marquage1.png", imgcopy1)
7 L2,imgcopy2=capture(img2, nbpts)
8 cv2.imwrite("marquage2.png", imgcopy2)
9 cv2.destroyAllWindows()
```



FIGURE 3.1.3. — À gauche : des points marqués à la main sur des points « caractéristiques » d'un crâne d'homo sapiens, à droite, les mêmes points caractéristiques marqués à la main sur un crâne d'homo neanderthalensis.

Puis on applique l'analyse procustéenne afin de comparer en « porportion » ces points.

```
1 L1mod,TF=procuste(L1,L2)
2 print(np.linalg.norm(L2-L1mod))
```

On obtient alors le rendu suivant grâce aux codes suivants :

```
1 L1mod=L1mod.astype("int")
2 ligne,col=np.shape(L1mod)
3 imgsorite=imgcopy2
4 for i in range(col) :
5     cv2.circle(imgsorite,tuple(L1mod[:,i]),5,(0,0,255),-1)
```

```

6     cv2.putText(imgcopy2,text=f"{i}", org=tuple(Limod[:,i]), fontFace = cv2.FONT_HERSHEY_DUPLEX,fontScale = 1.0,
7     color = (0, 0, 255), thickness = 2)
8     cv2.imshow("test", imgsortie)
9     cv2.imwrite("resultatfinal.png", imgsortie)

```

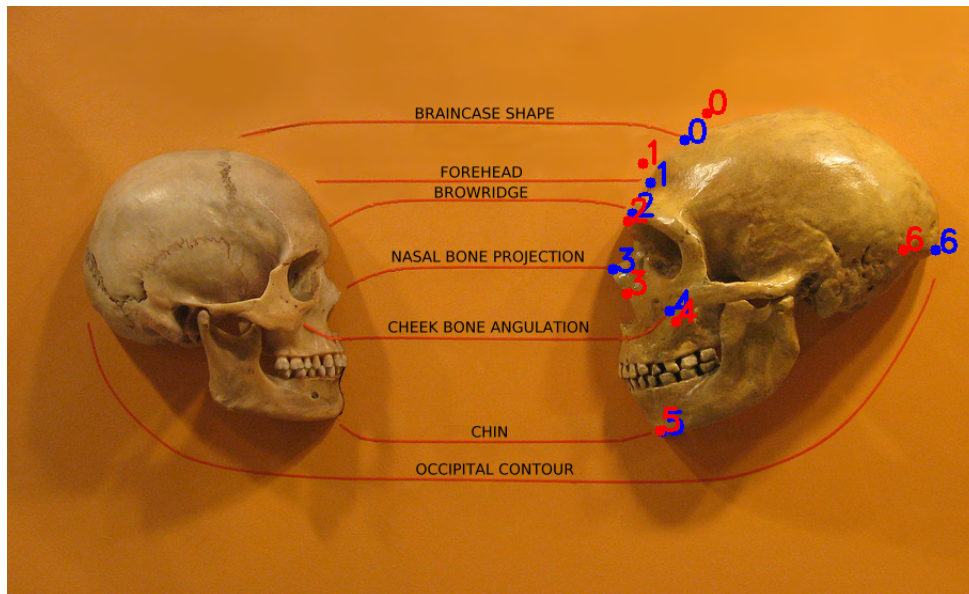


FIGURE 3.1.4. – *Résultat de l'analyse procustéenne. Les points rouges et les points bleus ne sont pas complètement déformable par une transformation affine. Les deux individus ne sont pas complètement en « proportion ».*

Les moindres carrés

4.1. Moindres carrés « classique » avec SVD

Un problème de moindres carrés (linéaire) se présente sous la forme suivante. Soit le système linéaire suivant :

$$Ax = b$$

où $A \in \mathcal{M}_{mn}(\mathbb{R})$ et x et b des vecteurs colonnes de taille n , le vecteur x étant inconnu. On cherche à minimiser la quantité :

$$\phi(x) := \|Ax - b\|_2^2$$

pour $x \in \mathbb{R}^n (\simeq \mathcal{M}_{n,1}(\mathbb{R}))$ i.e on cherche à résoudre :

$$\arg \min_{x \in \mathbb{R}^n} \|Ax - b\|_2^2$$

Nous allons voir que la décomposition SVD permet de donner la forme de la solution la plus générale à ce problème.

Commençons par une petite propriété sur la compatibilité de la norme euclidienne pour les vecteurs et de la norme de Frobenius pour les matrices avec les matrices orthogonales.

Proposition 4.1 :

On a les égalités suivantes :

1. Soit $x \in \mathbb{R}^n$, alors pour toute matrice orthogonale $U \in \mathbb{O}_n$ on a :

$$\|Ux\|_2 = \|x\|_2$$

2. Soit $X \in \mathcal{M}_{m,n}(\mathbb{R})$, alors pour tout couple de matrices orthogonales $(U, V) \in \mathbb{O}_n \times \mathbb{O}_m$ on a :

$$\|UXV\|_F = \|X\|_F$$

Démonstration. Commençons par le point 1. Soit $x \in \mathbb{R}^n$ et $U \in \mathbb{O}_n$, on a :

$$\begin{aligned} \|Ux\|_2^2 &= (Ux)^T (Ux) \\ &= x^T U^T U x \\ &= x^T x \\ &= \|x\|_2^2 \end{aligned}$$

Pour le point 2, fixons $X \in \mathcal{M}_{m,n}(\mathbb{R})$ et un couple de matrices orthogonales $(U, V) \in \mathbb{O}_n \times \mathbb{O}_m$, alors :

$$\begin{aligned} \|UXV\|_F^2 &= \text{trace} \left((UXV)^T (UXV) \right) \\ &= \text{trace} (V^T X^T U^T U X V) \\ &= \text{trace} ((V^T) (X^T X V)) \\ &= \text{trace} ((X^T X V) (V^T)) \\ &= \text{trace} (X^T X) \\ &= \|X\|_F^2 \end{aligned}$$

Ce qui achève la démonstration. □

En résumé les changements de repères orthonormés ne modifie pas les normes euclidienne et de Frobenius. Nous allons utiliser cela pour réécrire le problème des moindres carrés.

Soit une SVD de $A = U\Sigma V^T$:

$$\begin{aligned}\phi(x) &:= \|Ax - b\|_2^2 \\ &= \|U\Sigma V^T x - b\|_2^2 \\ &= \|U(\Sigma V^T x - U^T b)\|_2^2 \\ &= \|\Sigma V^T x - U^T b\|_2^2\end{aligned}$$

En posant les changements de variables suivants :

$$\boxed{\eta := V^T x \quad \text{et} \quad \beta = U^T b}$$

On se retrouve alors à résoudre :

$$\begin{aligned}\phi(x) &:= \|Ax - b\|_2^2 \\ &= \|\Sigma\eta - \beta\|_2^2 \\ &= \sum_{i=1}^r (\sigma_i \eta_i - \beta_i)^2 + \sum_{i=r+1}^m \beta_i^2\end{aligned}$$

où r est le rang de A et les σ_i sont les valeurs singulières de A . Comme nous cherchons à minimiser ϕ et que l'on vient de voir qu'elle s'écrit comme une somme de terme positif, une solution au sens des moindres carrés de système linéaire $Ax = b$ doit donc vérifier pour tout $i \in \llbracket 1, r \rrbracket$ la condition :

$$\begin{aligned}\sigma_i \eta_i - \beta_i &= 0 \\ \Leftrightarrow \boxed{\eta_i = \frac{1}{\sigma_i} \beta_i}\end{aligned}$$

Les conditions précédentes sont en réalité les seules conditions à la résolution des moindres carrés. Pour les η_i avec $i \in \llbracket r+1, n \rrbracket$, on peut compléter le vecteur η par des valeurs arbitraires. Notons :

$$\eta := \eta^{(1)} + \eta^{(2)}$$

avec :

$$\eta^{(1)} := \begin{pmatrix} \frac{1}{\sigma_1} \beta_1 \\ \vdots \\ \frac{1}{\sigma_r} \beta_r \\ 0_{n-r} \end{pmatrix} = \begin{pmatrix} \frac{1}{\sigma_1} & & & \\ & \ddots & & \\ & & \frac{1}{\sigma_r} & \\ & & & 0_{n-r, m-r} \end{pmatrix} U^T b = \begin{pmatrix} \Sigma_r^{-1} & 0_{r, m-r} \\ 0_{n-r, r} & 0_{n-r, m-r} \end{pmatrix} U^T b$$

où Σ_r note la matrice réduite de Σ . En notant :

$$\boxed{\Sigma^\dagger := \begin{pmatrix} \Sigma_r^{-1} & 0_{r, m-r} \\ 0_{n-r, r} & 0_{n-r, m-r} \end{pmatrix}}$$

on obtient la forme réduite :

$$\eta^{(1)} := \Sigma^\dagger U^T b$$

et :

$$\eta^{(2)} := \begin{pmatrix} 0_r \\ z \end{pmatrix}$$

où $z \in \mathbb{R}^{n-r}$ est un vecteur arbitraire. Enfin pour avoir la solution x il ne reste plus qu'à multiplier à droite par V :

$$x = V\eta$$

En développant cette expression on obtient :

$$x = V \left(\eta^{(1)} + \eta^{(2)} \right) = V \Sigma^\dagger U^T b + V \eta^{(2)}$$

Ainsi le résultat précédent nous conduit à définir ce que l'on appelle le pseudo-inverse d'une matrice.

Définition 4.1 : Le pseudo-inverse

Soit $A \in \mathcal{M}_{m,n}(\mathbb{R})$, le **pseudo-inverse** de A est la matrice notée :

$$A^\dagger := V \Sigma^\dagger U^T$$

où U, Σ et V sont obtenus d'une décomposition en valeurs singulières de A . Le pseudo-inverse ne dépend pas de la décomposition SVD choisie.

On peut résumer les résultats de cette section sous la forme du théorème suivant.

Théorème 4.1 : Solution au sens des moindres carrés

Soit le système linéaire suivant :

$$Ax = b$$

où $A \in \mathcal{M}_{mn}(\mathbb{R})$ et x et b des vecteurs colonnes de taille n , le vecteur x étant inconnu. Les solutions au sens des moindres carrés de ce système sont donnée par :

$$x := A^\dagger b + V \eta^{(2)}$$

avec V une matrice orthogonale d'une décomposition en valeurs singulières de $A = U \Sigma V^T$ et :

$$\eta^{(2)} := \begin{pmatrix} 0_r \\ z \end{pmatrix}$$

où $z \in \mathbb{R}^{n-r}$ est un vecteur arbitraire.

La norme 2 de ces solutions vaut :

$$\|x\|_2^2 = \|A^\dagger b\|_2^2 + \|z\|_2^2$$

Ainsi il existe toujours une solution de norme 2 minimale :

$$x^* := A^\dagger b$$

Démonstration. La quasi-intégralité du théorème a été démontré dans les raisonnements précédents. Il ne nous reste plus qu'à démontrer la minimalité de la norme pour x^* . En développant la norme 2 de x :

$$\begin{aligned} \|x\|_2^2 &= x^T x \\ &= \left(V \left(\eta^{(1)} + \eta^{(2)} \right) \right)^T \left(V \left(\eta^{(1)} + \eta^{(2)} \right) \right) \\ &= \eta^{(1)T} \eta^{(1)} + 2\eta^{(1)T} \eta^{(2)} + \eta^{(2)T} \eta^{(2)} \end{aligned}$$

Or d'après la forme de $\eta^{(1)}$ et $\eta^{(2)}$ il est évident que ces vecteurs sont orthogonaux (pour ce produit scalaire) i.e :

$$\eta^{(1)T} \eta^{(2)} = 0$$

d'où le résultat en remplaçant les vecteurs $\eta^{(1)}$ et $\eta^{(2)}$ par leurs expressions respectives. □

On obtient également le corollaire immédiat.

Corollaire 4.1 :

Si le rang de $A \in \mathcal{M}_{mn}(\mathbb{R})$ est plein i.e. si $rg(A) := \min(m, n)$ alors la solution du problème des moindres carrés est unique.

Voyons un exemple simple.

Exemple 9. Soit le système linéaire :

$$\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

Notons $A = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$, $x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$ et $b = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$. Commençons par effectuer la décomposition en valeurs singulières de A . On a :

$$A^T A = \begin{pmatrix} 2 & 2 \\ 2 & 2 \end{pmatrix}$$

dont le polynôme caractéristique est :

$$P_{A^T A}(\lambda) = (2 - \lambda)^2 - 4 = -\lambda(4 - \lambda)$$

Ainsi A possède une unique valeur singulière : $\sigma_1 = \sqrt{4} = 2$. On calcule alors le sous-espace caractéristique de $A^T A$ associé à la valeur propre 4. On obtient :

$$A^T A w = 4w \Leftrightarrow \begin{cases} 2w_1 + 2w_2 = 4w_1 \\ 2w_1 + 2w_2 = 4w_1 \end{cases} \iff w_1 = w_2 \Leftrightarrow w \in \text{Vec} \left(\begin{pmatrix} 1 \\ 1 \end{pmatrix} \right)$$

On pose donc : $v_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$. Il suffit de compléter ce vecteur par $v_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$ (pour ceux qui ne voient pas pourquoi celui là, faites un Gram-Schmidt...). On a ainsi :

$$V = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \text{ et } \Sigma = \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix}$$

En utilisant les propriétés de la SVD on obtient :

$$u_1 = \frac{1}{\sigma_1} A v_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

et de même on complète ce vecteur par un vecteur normée qui lui est orthogonaux (Gram-Schmidt encore...) :

$$u_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

D'où : $U = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$ (= V dans ce cas). On vérifie que l'on a bien :

$$A = U \Sigma V^T = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

On peut à présent résoudre notre système. Ici $r = 1$, on a donc comme $\beta = U^T b = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$:

$$\eta_i = \frac{1}{\sigma_i} \beta_i = \frac{1}{2\sqrt{2}}$$

On complète η par un réel arbitraire $\eta_2 = \alpha \in \mathbb{R}$. Finalement les solutions au sens des moindres carrés de $Ax = b$, au sens où un tel x minimise la quantité $\phi(x) = \|Ax - b\|_2^2$ sont :

$$x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = V \begin{pmatrix} \eta_1 \\ \eta_2 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} \frac{1}{2\sqrt{2}} \\ \alpha \end{pmatrix} = \begin{pmatrix} \frac{1}{4} + \frac{\alpha}{\sqrt{2}} \\ \frac{1}{4} - \frac{\alpha}{\sqrt{2}} \end{pmatrix}$$

Il y a donc une infinité de solutions. Cherchons la solution de norme la plus petite. On a :

$$\|x\|_2 = \sqrt{\frac{1}{8} + \alpha^2}$$

Ainsi la solution de norme minimale est obtenue pour $\alpha = 0$ et est :

$$x^* = \begin{pmatrix} \frac{1}{4} \\ \frac{1}{4} \end{pmatrix}$$

On remarque que cette solution correspond à la solution trouvée en utilisant le pseudo-inverse de A . En effet :

$$A^\dagger b = V \Sigma^\dagger U^T b = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} \frac{1}{2} & 0 \\ 0 & 0 \end{pmatrix} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} \frac{1}{4} \\ \frac{1}{4} \end{pmatrix} = x^*$$

Finissons cette sous-section par une courte discussion sur l'erreur produite. Ici encore la SVD nous donne une réponse. En effet soit x^* la solution au sens des moindres carrés de norme minimale et regardons l'erreur :

$$\begin{aligned}
\phi(x^*) &:= \|Ax^* - b\|_2^2 \\
&= \|U\Sigma V^T (V\Sigma^\dagger U^T b) - b\|_2^2 \\
&= \|U\Sigma V^T (V\Sigma^\dagger U^T b) - UU^T b\|_2^2 \\
&= \|U(\Sigma\Sigma^\dagger - I_m)U^T b\|_2^2 \\
&= \|(\Sigma\Sigma^\dagger - I_m)U^T b\|_2^2 \\
&= \left\| \begin{pmatrix} 0_r \\ u_{r+1}^T b \\ \vdots \\ u_m^T b \end{pmatrix} \right\|_2^2 \\
&= \sum_{i=r+1}^m (u_i^T b)^2
\end{aligned}$$

4.1.1. Moindres carrés totaux

Fixons $A \in \mathcal{M}_{mn}(\mathbb{R})$ une matrice et $b \in \mathbb{R}^n$ un vecteur. On rappelle que lorsque le système linéaire : $Ax = b$ n'est pas compatible (i.e. qu'il n'existe pas de solution), une solution au sens des moindres carrés consiste à rechercher un vecteur x minimisant la quantité suivante :

$$\arg \min_{x \in \mathbb{R}^n} \|Ax - b\|_2^2$$

Supposons que l'on se donne une solution à ce problème x^* (par exemple en utilisant le pseudo-inverse comme à la section précédente). On peut alors écrire en notant δb le vecteur « mesurant » l'erreur :

$$Ax^* - b = \delta b \Leftrightarrow Ax^* = b + \delta b$$

On peut interpréter cela en remarquant que dans la méthode des moindres carrés classique seules les données b sont « contes-tées ». Dans de nombreux exemples de moindre carrés classiques le vecteur b est un vecteur contenant les observations, alors que la matrice A contient le modèle/les variables explicatives. Or suivant le contexte les valeurs de la matrice A elle-même peuvent être entaché d'erreur (s'ils sont issus de mesures par exemple). En admettant que des erreurs de mesures aient pu être faites sur b et sur A , une autre stratégie consiste à rechercher une matrice δA (erreur de mesure sur A) et un vecteur δb (erreur de mesure sur b) dont les coefficients sont « petits » i.e. tels que la quantité :

$$\sum_{i=1}^n \sum_{j=1}^m (\delta A_{ij})^2 + \sum_{i=1}^n (\delta b_i)^2$$

soit la plus proche de 0 possible et tels que le système suivant admette une unique solution (i.e. soit compatible) :

$$(A + \delta A)x = b + \delta b$$

On appelle ce problème : les **moindres carrés totaux**.

Nous allons formaliser matriciellement ce problème. On notera : $\bar{A} := A + \delta A$ et $\bar{b} := b + \delta b$. Avec ces notations :

$$\sum_{i=1}^n \sum_{j=1}^m (\delta A_{ij})^2 + \sum_{i=1}^n (\delta b_i)^2 = \left\| \begin{pmatrix} \delta A & \delta b \end{pmatrix} \right\|_F^2 = \left\| \begin{pmatrix} \bar{A} & \bar{b} \end{pmatrix} - \begin{pmatrix} A & b \end{pmatrix} \right\|_F^2$$

où $\begin{pmatrix} \delta A & \delta b \end{pmatrix}$, $\begin{pmatrix} \bar{A} & \bar{b} \end{pmatrix}$ et $\begin{pmatrix} A & b \end{pmatrix}$ sont des matrices définies par blocs.

On peut commencer à remarquer que l'erreur commise par la méthode des moindres carrés totaux est plus faible que pour les moindres carrés simples en effet :

$$\min_{\bar{b} \in \text{Im}(\bar{A})} \left\| \begin{pmatrix} \bar{A} & \bar{b} \end{pmatrix} - \begin{pmatrix} A & b \end{pmatrix} \right\|_F^2 \leq \min_{\bar{b} \in \text{Im}(A + \delta A), \delta A = 0} \left\| \begin{pmatrix} \bar{A} & \bar{b} \end{pmatrix} - \begin{pmatrix} A & b \end{pmatrix} \right\|_F^2 = \min_{x \in \mathbb{R}^n} \|Ax - b\|_2^2$$

Nous allons résoudre le problème des moindres carrés totaux dans le cas où $\text{rg}(A) = r$ et $Ax = b$ est incompatible (avec $b \neq 0$). Ces hypothèses impliquent que :

$$\text{rg} \left(\begin{pmatrix} A & b \end{pmatrix} \right) = r + 1$$

Ainsi on est ramener à résoudre le problème équivalent (sous les hypothèses précédentes) :

$$\arg \min_{(\bar{A} \ \bar{b}) \in \mathcal{M}_{m,(n+1)}(\mathbb{R}), rg((\bar{A} \ \bar{b}))=r} \|(\bar{A} \ \bar{b}) - (A \ b)\|_F^2$$

En effet comme l'on impose le fait que

$$(A + \delta A)x = b + \delta b \Leftrightarrow \bar{A}x = \bar{b}$$

est une solution unique il vient que $rg((\bar{A} \ \bar{b})) = r$. On retrouve le problème d'approximation de Schmidt dont on connaît les formes des solutions. En utilisant une décomposition en valeurs singulières de $(A \ b) = U\Sigma V^T$ on trouve alors :

$$(\bar{A} \ \bar{b}) = \sum_{i=1}^r \sigma_i u_i v_i^T$$

où les σ_i sont les valeurs singulières de $(A \ b)$, u_i et v_i les vecteurs colonnes de U et V respectivement.

Ainsi une solution x au sens des moindres carrés totaux vérifie :

$$\bar{A}x = \bar{b} \Leftrightarrow (\bar{A} \ \bar{b}) \begin{pmatrix} x \\ -1 \end{pmatrix} = 0_m$$

i.e. que les solutions doivent être dans le noyau de la matrice $(\bar{A} \ \bar{b})$.

On remarque ensuite que tous les vecteurs $(v_{r+1}, \dots, v_{n+1})$ sont dans le noyau de $(\bar{A} \ \bar{b})$ car par définition :

$$(\bar{A} \ \bar{b}) v_{k+1} = \left(\sum_{i=1}^r \sigma_i u_i v_i^T \right) v_{k+1} = 0$$

pour $k \in \llbracket r+1, n+1 \rrbracket$. Dans **ce cas général les solutions ne sont pas uniques et peuvent être produite à partir de combinaisons linéaires des vecteurs** $(v_{r+1}, \dots, v_{n+1})$.

Supposons que $rg(A) = n$, dans ce cas a donc :

$$\ker(\bar{A} \ \bar{b}) = Vect(v_{n+1})$$

Or ce noyau (qui est de dimension 1) ne peut donc contenir un vecteur de la forme :

$$\begin{pmatrix} x \\ -1 \end{pmatrix}$$

que si $v_{n+1,n+1} \neq 0$.

Si l'on suppose que $v_{n+1,n+1} \neq 0$ (la composante $(n+1)$ -ième de v_{n+1}) cela démontre qu'une solution existe (et est unique si $\sigma_n > \sigma_{n+1}$ car on pourrait permuter σ_n et σ_{n+1} dans Σ sinon) et s'écrit :

$$\begin{pmatrix} x \\ -1 \end{pmatrix} = -\frac{1}{v_{n+1,n+1}} v_{n+1} \Leftrightarrow x = -\frac{1}{v_{n+1,n+1}} \begin{pmatrix} v_{1,n+1} \\ \vdots \\ v_{n,n+1} \end{pmatrix}$$

Enfin si $\sigma_n > \sigma_{n+1}$, x est unique et l'on peut trouver une forme plus simple. La définition des valeurs singulières :

$$\begin{aligned} (A \ b)^T (A \ b) v_{n+1} &= \sigma_{n+1}^2 v_{n+1} \\ \Leftrightarrow (A \ b)^T (A \ b) \begin{pmatrix} x \\ -1 \end{pmatrix} &= \sigma_{n+1}^2 \begin{pmatrix} x \\ -1 \end{pmatrix} \\ \Leftrightarrow \begin{pmatrix} A^T A & A^T b \\ b^T A & b^T b \end{pmatrix} \begin{pmatrix} x \\ -1 \end{pmatrix} &= \sigma_{n+1}^2 \begin{pmatrix} x \\ -1 \end{pmatrix} \\ \Rightarrow A^T A x - A^T b &= \sigma_{n+1}^2 x \\ \Leftrightarrow (A^T A - \sigma_{n+1}^2 I_n) x &= A^T b \end{aligned}$$

Il ne nous reste plus qu'à montrer que la matrice $A^T A - \sigma_{n+1}^2 I_n$ est inversible. Or comme x est unique, cette matrice est nécessairement inversible. En effet il suffit de voir que l'assertion contraposée est vraie. Si $A^T A - \sigma_{n+1}^2 I_n$ est non inversible alors il existe un vecteur v non nul dans son noyau i.e. vérifiant :

$$(A^T A - \sigma_{n+1}^2 I_n) v = 0$$

et $x + v$ serait une autre solution. Finalement on trouve dans le cas où $\sigma_n > \sigma_{n+1}$ la forme simple de la solution des moindres carrés totaux :

$$x = (A^T A - \sigma_{n+1}^2 I_n)^{-1} A^T b$$

Finissons cette sous-section par une remarque d'ordre « pratique ». Dans la pratique nous devons donner/fournir une solution même dans le cas général où $rg(A) = r < n$. On trouve souvent dans les algorithmes une solution fournissant une sorte de « compromis » :

$$x = -\frac{1}{\sum_{i=r+1}^{n+1} v_{n+1,i}^2} \sum_{i=r+1}^{n+1} v_{n+1,i} \begin{pmatrix} v_{1,i} \\ \vdots \\ v_{n,i} \end{pmatrix}$$

et si la somme : $\sum_{i=r+1}^{n+1} v_{n+1,i}^2 = 0$, on se restreint à une partie uniquement des indices i en prenant une partie stricte de sous-indice de $\llbracket r+1, n+1 \rrbracket$ où cette somme est non nulle.

Exemple 10. Retrouver un cercle à partir d'un nuage de points

Soient $E = \{(x_i, y_i)\}_{i \in \llbracket 1, n \rrbracket}$ un ensemble de n points de $\mathbb{R}^2$ des points mesurés par un instrument de mesure qui produit des « erreurs ». Théoriquement ces points devraient être sur un cercle. On obtient les points suivants :

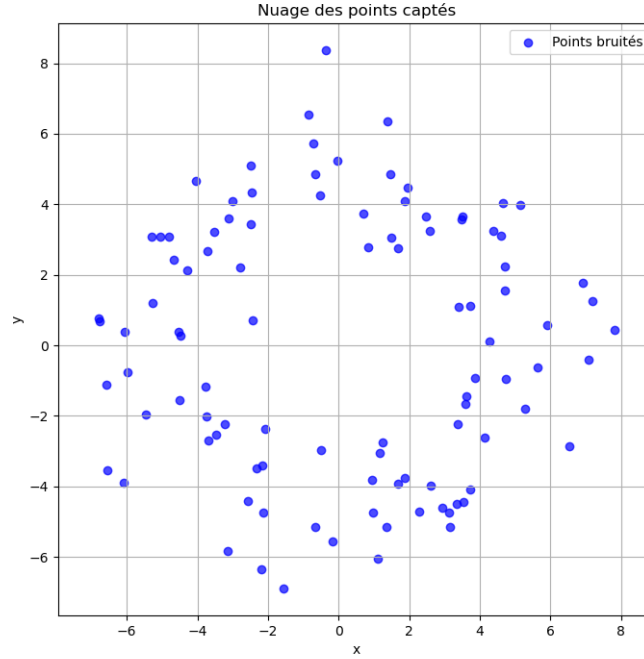


FIGURE 4.1.1. – Points mesurés avec erreurs

On cherche un cercle Γ qui « approxime » au mieux cet ensemble de points, c'est-à-dire vérifiant « au mieux » :

$$\Gamma : (x_i - \alpha)^2 + (y_i - \beta)^2 = R^2$$

Nous devons donc déterminer le centre du cercle (α, β) et son rayon R .

En développant pour tout $i \in \llbracket 1, n \rrbracket$ et en « isolant » les termes inconnus :

$$\begin{aligned} (x_i - \alpha)^2 + (y_i - \beta)^2 &= R^2 \\ \Leftrightarrow 2x_i\alpha + 2y_i\beta + R^2 - \alpha^2 - \beta^2 &= x_i^2 + y_i^2 \\ \Leftrightarrow 2x_i\alpha + 2y_i\beta + \gamma &= x_i^2 + y_i^2 \end{aligned}$$

avec $\gamma = R^2 - \alpha^2 - \beta^2$.

Les termes x_i et y_i étant connus, ce changement de variable permet se ramener à une équation linéaire en α, β et γ . Nous devons donc résoudre le système linéaire suivant :

$$\begin{pmatrix} 2x_1 & 2y_1 & 1 \\ 2x_2 & 2y_2 & 1 \\ \vdots & \vdots & \vdots \\ 2x_N & 2y_N & 1 \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \\ \gamma \end{pmatrix} = \begin{pmatrix} x_1^2 + y_1^2 \\ x_2^2 + y_2^2 \\ \vdots \\ x_N^2 + y_N^2 \end{pmatrix}$$

En posant :

$$A := \begin{pmatrix} 2x_1 & 2y_1 & 1 \\ 2x_2 & 2y_2 & 1 \\ \vdots & \vdots & \vdots \\ 2x_N & 2y_N & 1 \end{pmatrix} \quad \text{et} \quad b := \begin{pmatrix} x_1^2 + y_1^2 \\ x_2^2 + y_2^2 \\ \vdots \\ x_N^2 + y_N^2 \end{pmatrix}$$

ce problème revient à résoudre un problème de moindres carrés :

$$A \begin{pmatrix} \alpha \\ \beta \\ \gamma \end{pmatrix} = b$$

En résolvant ce problème avec d'un côté les moindres carrés totaux et les moindres carrés « classique » on trouve les cercles suivants :

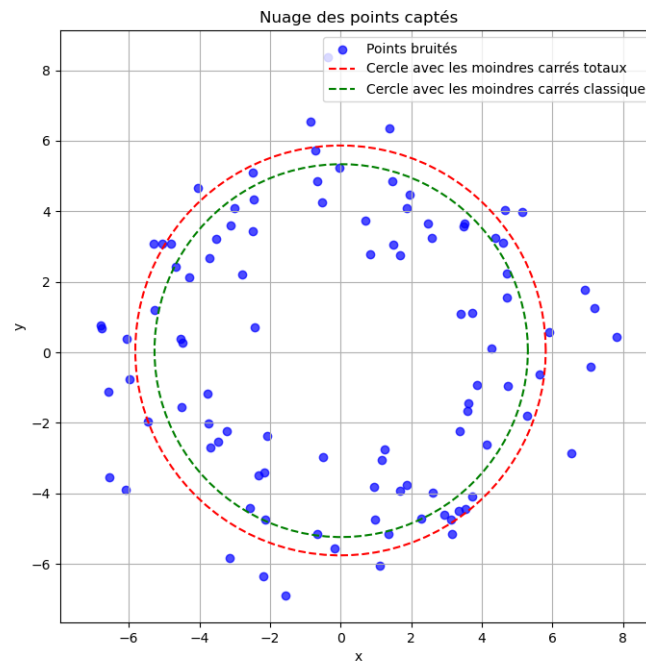


FIGURE 4.1.2. – Comparaison entre les moindres carrés totaux et les moindres carrés ordinaires.

4.2. Un exemple d'apprentissage supervisé avec les moindres carrés

Les méthodes d'apprentissages supervisés consistent à partir de la donnée d'exemples négatifs/ne vérifiant pas ou positifs/-vérifiant une ou des propriétés de prédire sur un nouveau cas s'il vérifie ou non la ou les propriétés considérées. Comme par exemple reconnaître une image représentant une voiture en « entraînant » un algorithme sur des images de voitures et de « non-voitures ». Ce sont des exemples classiques de méthodes prédictives : ici on a un modèle de prédiction incarné par une fonction donc certains paramètres sont à déterminer en fonction des exemples d'entraînements.

Dans cette sous-section on se propose de donner un exemple de formalisation et de résolution de ce type de problème grâce à la méthode des moindres carrés. Nous nous servirons de cette exemple comme « référent/comparant » pour les méthodes que nous verrons dans la partie 3 de ce cours. Nous commencerons par une première partie de formalisation puis nous implémenterons les résultats trouvés sous Python sur l'exemple de la reconnaissance des chiffres manuscrits.

4.2.1. Plusieurs moindres carrées en parallèles

Avant de voir comment formaliser un problème d'apprentissage supervisé nous allons voir un résultat assez simple généralisant les calculs précédents sur les moindres carrés .

Soit $A \in \mathcal{M}_{mn}(\mathbb{R})$ une matrice (non nécessairement carrée). On suppose que les vecteurs colonnes de A sont linéairement indépendants. On rappelle que la norme de Frobenius de A au carrée est donnée par :

$$\|A\|_F^2 = \sum_{i=1}^m \sum_{j=1}^n a_{ij}^2$$

Soient $A \in \mathcal{M}_{mn}(\mathbb{R})$ et $B \in \mathcal{M}_{mk}(\mathbb{R})$, on cherche à résoudre le problème suivant :

$$\min_{X \in \mathcal{M}_{nk}(\mathbb{R})} \|AX - B\|_F^2$$

Nous allons montrer que :

$$\|AX - B\|_F^2 = \sum_{i=1}^k \|Ax_i - b_i\|_2^2$$

où x_i et b_i notent les vecteurs colonnes i des matrices X et B respectivement. En effet il suffit de considérer la multiplication matricielle par blocs en colonnes :

$$AX - B = \begin{pmatrix} Ax_1 - b_1 & Ax_2 - b_2 & \cdots & Ax_k - b_k \end{pmatrix}$$

et de remarquer que la norme de Frobenius d'une matrice $M \in \mathcal{M}_{lc}(\mathbb{R})$ se ramène à la somme des normes 2 de ces colonnes :

$$\|M\|_F^2 = \left\| \begin{pmatrix} m_1 & m_2 & \cdots & m_c \end{pmatrix} \right\|_F^2 = \sum_{i=1}^c \|m_i\|_2^2$$

où m_i note le vecteur colonne i de M . On a alors le résultat demandée :

$$\|AX - B\|_F^2 = \sum_{i=1}^k \|Ax_i - b_i\|_2^2$$

Revenons à notre problème. On cherche X qui minimise $\|AX - B\|_F^2 = \sum_{i=1}^k \|Ax_i - b_i\|_2^2$. Or minimiser une somme de carrés (de nombre positif) revient à minimiser chaque terme de la somme i.e. chaque $\|Ax_i - b_i\|_2^2$. Pour tout $i \in \llbracket 1, k \rrbracket$, on doit ainsi résoudre un problème de la forme :

$$\min_{x \in \mathbb{R}^n} \|Ax_i - b_i\|_2^2$$

Nous avons déjà vu qu'une solution au sens des moindres carrés vérifie équation normale :

$$A^T Ax_i = A^T b_i$$

Par hypothèse A est de rang plein, la matrice $A^T A$ est donc inversible et notre problème admet une unique solution :

$$x_i^* = (A^T A)^{-1} A^T b_i$$

La solution au problème global est alors :

$$X^* = \begin{pmatrix} (A^T A)^{-1} A^T b_1 & (A^T A)^{-1} A^T b_2 & \cdots & (A^T A)^{-1} A^T b_k \end{pmatrix} = (A^T A)^{-1} A^T B$$

Ainsi une solution X^* au sens des moindres carrées est donnée par :

$$X^* = (A^T A)^{-1} A^T B$$

On peut généraliser le résultat précédent au cas où la matrice A n'est pas nécessairement de rang plein grâce à la décomposition en valeurs singulières et au pseudo-inverse. En se limitant à la solution de norme (de Frobenius) minimale on obtient la solution :

$$X^* = A^\dagger B$$

Exemple 11. Problème : donner la meilleure approximation d'un inverse à droite pour une matrice $A \in \mathcal{M}_{mn}(\mathbb{R})$ de rang n .

Un inverse à droite d'une matrice $A \in \mathcal{M}_{mn}(\mathbb{R})$ est une matrice $X \in \mathcal{M}_{nm}(\mathbb{R})$ telle que :

$$AX = I_m$$

Or ce problème n'admet pas nécessairement de solution. Par exemple pour la matrice :

$$M = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}$$

si l'on cherche $X \in \mathcal{M}_{2,3}(\mathbb{R})$ telle que :

$$MX = \begin{pmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ 0 & 0 & 0 \end{pmatrix} \neq I_3$$

On cherche alors la matrice X qui « s'approche le plus » de la solution i.e.

$$\min_{X \in \mathcal{M}_{nm}(\mathbb{R})} \|AX - I_m\|_F^2$$

D'après ce qui précède et comme A est de rang n (i.e. de rang plein) on obtient comme solution :

$$X^* = A^\dagger I_m = A^\dagger = V \Sigma^\dagger U^T$$

pour une SVD de $A = U \Sigma V^T$. On retrouve le fait que le pseudo-inverse est la meilleure approximation de l'inverse au sens de la norme de Frobenius. Sur l'exemple de M donnée, on trouve immédiatement comme « meilleure » approximation d'un inverse à droite de M :

$$X^* = M^T$$

(cet exemple se généralise à toutes matrices dont les colonnes sont composées de vecteurs orthogonaux deux à deux).

4.2.2. Formalisation d'un problème d'apprentissage supervisé

Nous allons décrire un problème de classification « linéaire » en k catégories où k est un entier plus grand que 2. Supposons donnés un ensemble fini E de taille m de donnée partitionnable en k sous-ensembles disjoints E_i de points de $\mathbb{R}^n$, chaque ensemble représente des données appartenant à une des k catégories :

$$E_i = \{u \in \mathbb{R}^n \mid u \text{ appartient à la catégorie } i.\} \quad \forall i \in \llbracket 1, k \rrbracket$$

Ces données forment ce que l'on appelle les données d'entraînements. Concrètement E est la donnée d'un tableau dont le nombre de ligne correspond à la taille du jeu de données et dont le nombre de colonne correspond à la taille d'une donnée ici n :

■	c_1	c_2	$\cdots$	c_n
u_1	u_{11}	u_{12}	$\cdots$	u_{1n}
u_2	u_{21}	u_{22}	$\cdots$	u_{2n}
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
u_m	u_{m1}	u_{m2}	$\cdots$	u_{mn}

D'un point de vue matriciel E est la matrice :

$$E := \begin{pmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ u_{21} & u_{22} & \cdots & u_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ u_{m1} & u_{m2} & \cdots & u_{mn} \end{pmatrix}$$

Si l'on reprend l'exemple de la reconnaissance de la présence de voiture sur une image, ici $k = 2$, l'ensemble E_1 forme les images dont on sait qu'elles contiennent des voitures et E_2 les images dont on sait qu'elles ne comportent aucune voiture.

L'idée est de trouver une fonction qui permet de « classifier » les données. Généralement on cherche une fonction $f : \mathbb{R}^n \rightarrow \mathbb{R}^k$ telle que :

$$f(u) = e_i^T$$

si u appartient à la catégorie i i.e. est dans E_i . On rappelle que le vecteur $e_i \in \mathbb{R}^k$ note le i -ème vecteur de la base canonique. L'intérêt d'une telle fonction est de pouvoir l'utiliser pour prédire de manière automatique dans quelle catégorie se place une donnée inconnue. Toujours en reprenant l'exemple des voitures/non-voitures, supposons donnée une image qui n'est pas dans les données d'entraînement x (vu comme un vecteur), alors il suffira de considérer la valeur de $f(x)$ et de regarder si cette valeur se « rapproche » le plus de e_1 ou de e_2 .

Comme il n'est pas clair qu'un tel problème admet une solution nous allons rechercher une solution au sens des moindres carrés (i.e. le meilleur compromis possibles), c'est-à-dire nous allons chercher f qui minimise la quantité suivante :

$$L_E = \sum_{i=1}^k \sum_{u \in E_i} \|f(u) - e_i^T\|_2^2$$

Exprimer sous cette forme, ce problème est trop général car l'ensemble des fonctions de $\mathbb{R}^n$ dans $\mathbb{R}^k$ est trop « vaste ». Nous allons rechercher f parmi les fonctions affines de la forme :

$$f_W(x) := \begin{pmatrix} x & 1 \end{pmatrix}^T W$$

avec $W \in \mathcal{M}_{n+1,k}$ à déterminer. Le vecteur $\begin{pmatrix} x & 1 \end{pmatrix}^T$ est un vecteur ligne par blocs que taille $n+1$ (car x est un vecteur de taille n). Nous justifierons cette convention « ligne » d'écriture car dans la pratique les données d'entraînement sont fournis sous la forme de ligne.

Sous cette forme nous pouvons réécrire matriciellement la quantité L_E sous la forme :

$$L_E(W) := \|\tilde{E}W - B\|_F^2$$

avec B la matrice de taille (m, k) et où une ligne j vaut e_i^T si u_j la ligne j de E est dans la catégorie i , et

$$\tilde{E} := \begin{pmatrix} E & 1_m \end{pmatrix}$$

avec 1_m le vecteur colonne de taille m ne contenant que des 1. On notera la dépendance en W car c'est cette matrice qui nous faut déterminer.

Ainsi pour minimiser $L(E) = \|\tilde{E}W - B\|_F^2$ on peut résoudre ce problème en prenant la solution de norme minimale :

$$W = \tilde{E}^\dagger B$$

Sous Python on pourra utiliser la fonction de Numpy : `np.linalg.pinv` afin de produire le pseudo-inverse.

Remarque 4.1 :

Le calcul du pseudo-inverse avec Numpy utilise une version « partielle » de la décomposition en valeurs singulières. En effet une SVD ne serait en pratique pas réalisable (problème de stockage mémoire en particuliers) sur des matrices avec un très grand nombre de colonne ou de ligne.

4.2.3. Application à l'exemple de la reconnaissance de chiffres manuscrits

Nous allons utiliser les résultats développés précédemment pour classer les chiffres manuscrits.

Pour tester nos algorithmes nous allons utiliser la base de données classique MNIST (Mixed National Institute of Standards and Technology). Cette base de données contient un jeu de 60 000 images de chiffres écrits à la main et 10 000 images tests afin de vérifier les performances des algorithmes. Chacune de ces images de chiffres manuscrits est une image en nuance de gris de taille 28×28 . Elle servait encore de test référence pour les algorithmes d'apprentissages supervisés il y a quelques années. On peut trouver ces données sur le site : https://www.kaggle.com/oddrational/mnist-in-csv#mnist_test.csv.

Commençons par télécharger les deux fichiers :

- `mnist_train.csv` : c'est le jeu de données d'entraînement qui va nous permettre de générer un hyperplan séparateur.
- `mnist_test.csv` : c'est le jeu de données « test » qui nous permettra de vérifier les performances de notre programme.

Commençons par charger ces données (n'oublier pas Numpy et OpenCV :

```
1 train_data = np.genfromtxt("mnist_train.csv", delimiter=",")
2 test_data = np.genfromtxt("mnist_test.csv", delimiter=",")
```

La première colonne de chacun des deux fichiers de données contient les « étiquettes » qui sont les chiffres représentés sur l'image. C'est ces données qui nous fournissent les catégories de chacune des lignes (chiffre) du fichier.

Une image de chiffre est implémenter sous la forme d'un vecteur ligne : opération de vectorisation d'un vecteur (les lignes d'une matrice sont mises côtés à côtés).

On peut visualiser une image vectorialisée en utilisant par exemple les commandes suivantes :

```
1 ligne=118
2 img=train_data[ligne,1:].reshape(28,28)
3 plt.imshow(img)
```

On visualise ici l'image numéro 118 (la 118 ligne du fichier de données) :

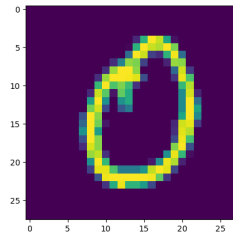


FIGURE 4.2.1. – Image 118 de minst_train

Afin d'appliquer le formalisme précédent nous allons devoir séparer les données (l'ensemble E précédent) et les étiquettes/catégories. Les étiquettes étant en première colonne on réalise cela avec les commandes python suivantes :

```
1 data=train_data[:,1:]
2 labels=train_data[:,0]
```

Il faut à présent créer notre matrice $\tilde{E}$, ce qui donne sous python :

```
1 Etilde=m,n=np.shape(data)
2 Etilde=np.hstack((data, np.ones((m,1))))
```

puis de résoudre numériquement :

$$W^* = \tilde{E}^\dagger B$$

Ici le problème est de produire la matrice B . Cette matrice est produite en utilisant les « labels » de la manière suivante :

```
1 B=np.zeros((m,10))
2 for i in range(m):
3     B[i,int(labels[i])]=1
```

Puis sous python cela donne :

```
1 W=np.linalg.pinv(Etilde)@B
```

Afin de tester notre « apprentissage », il nous faut évaluer notre fonction sur des données autres. Les données contenues dans test_data sont justement là pour cela. Soit x une ligne du fichier test_data où l'on exclue son label, il nous faut calculer :

$$f_W(x) := \begin{pmatrix} x & 1 \end{pmatrix}^T W$$

Cette fonction s'écrit sous python :

```
1 def f(x,W):
2     xtilde=np.hstack((x,1))
3     return xtilde.T @ W
```

Prenons l'exemple du x suivant :

```
1 ligne=200
2 x=test_data[ligne,1:]
3 rep=f(x,W)
4 plt.bar(np.arange(10),rep)
5 plt.xticks(range(0,10))
```

Ces commandes renvoient le vecteur sortie de f sous la forme d'une sortie graphique :

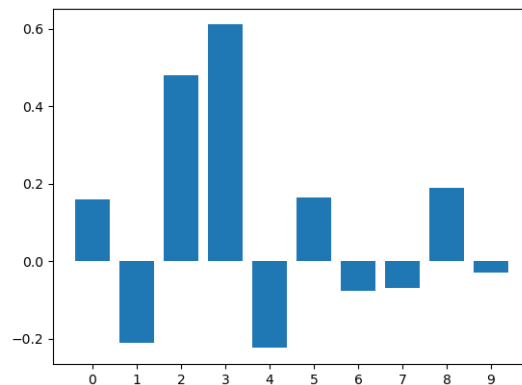


FIGURE 4.2.2. – Résultat pour la ligne 200 de test_data

La plus grande des valeurs est obtenue sur la 3ème composante. On peut donc en déduire que notre algorithme prédit pour le chiffre x un 3. Nous pouvons vérifier cette prédiction grâce à la colonne labels du fichier test_data :

```
1 test_data[ligne,0]
```

Cette commande renvoie comme sortie un 3. Notre algorithme ne s'est pas trompé.

Quitte à prendre la plus grande des valeurs de f nous allons créer une fonction prédiction qui renverra un entier correspondant à l'indice de la plus grande composante de $f(x)$:

```
1 def f_prediction(x,W) :
2     return np.argmax(f(x,W))
```

Cette fonction nous renvoie bien 3 pour le vecteur x test précédent. Néanmoins pour tester l'efficacité de notre algorithme nous allons utiliser toutes les 10 000 données du fichier test_data. Nous allons présenter cela sous la forme d'une matrice, appelée **matrice de confusion** M de taille (10,10) dont les coefficients m_{ij} sont données par :

$m_{ij} :=$ "nombre de x dont la catégorie est i vu par notre algorithme comme étant un j ".

Avec cette définition plus cette matrice est diagonale meilleur est notre algorithme et on peut également définir le **taux de réussite** par :

$$t_r := \frac{\text{Trace}(M)}{\sum_{i=1}^{10} \sum_{j=1}^{10} m_{ij}}$$

Programmons cela :

```
1 M=np.zeros((10,10))
2 labels_test=test_data[:,0]
3 ltest,ctest=np.shape(test_data)
4 for i in range(ltest) :
5     x=test_data[i,1:]
6     pred= f_prediction(x,W)
7     M[int(labels_test[i]),pred]+=1
```

Ce programme nous donne les valeurs suivantes pour M , la matrice de confusion :

$i \backslash j$	0	1	2	3	4	5	6	7	8	9
0	944	0	1	2	2	7	14	2	7	1
1	0	1107	2	2	3	1	5	1	14	0
2	18	54	813	26	15	0	42	22	37	5
3	4	17	23	880	5	17	9	21	22	12
4	0	22	6	1	881	5	10	2	11	44
5	23	18	3	72	24	659	23	14	39	17
6	18	10	9	0	22	17	875	0	7	0
7	5	40	16	6	26	0	1	884	0	50
8	14	46	11	30	27	40	15	12	759	20
9	15	11	2	17	80	1	1	77	4	801

On trouve comme taux de réussite :

$$t_r = 0.8603$$

i.e. 86% de réussite sur les 10 000 données tests.

Cet algorithme simple donne des résultats corrects, nous verrons que l'on peut faire beaucoup mieux au cours de ce module.

Cependant : quand est-il des moindres carrés totaux ?

Exercice 12. En raisonnant sur les colonnes de la matrice B , implémenter une résolution au sens des moindres carrés totaux. Que constatez-vous sur les résultats obtenus ? Commenter.

Deuxième partie

Décompositions biconjugués (ouverture)

Troisième partie

Annexes

Rappels sur les éléments de base du calcul matriciel

5.1. Quelques notions générales

Définition 5.1 :

Formellement **une matrice à coefficients dans un ensemble** E est la donnée d'une suite « double indicée » i.e. une matrice A est la donnée d'une application :

$$A : \llbracket 1, m \rrbracket \times \llbracket 1, n \rrbracket \rightarrow E$$

On notera $A(i, j) := a_{ij}$ le coefficient (i, j) de la matrice A et nous dirons que m est le nombre de lignes et n est le nombre de colonnes. Nous noterons l'ensemble de telles applications :

$$\mathcal{M}_{mn}(E) := \text{Fct}(\llbracket 1, m \rrbracket \times \llbracket 1, n \rrbracket, E)$$

ensemble appelé **ensemble des matrices de taille (m, n) à coefficients dans E** .

On peut alors représenter une matrice sous forme d'un « tableau » :

$$A := (a_{ij})_{(i,j) \in \llbracket 1, m \rrbracket \times \llbracket 1, n \rrbracket} := \begin{pmatrix} a_{11} & \cdots & \cdots & \cdots & a_{1n} \\ \vdots & \ddots & \ddots & & \vdots \\ \vdots & \ddots & a_{ii} & \ddots & \vdots \\ \vdots & & \ddots & \ddots & \vdots \\ a_{m1} & \cdots & \cdots & \cdots & a_{mn} \end{pmatrix}$$

Nous ferons régulièrement l'abus de noter une matrice par la donnée de ses coefficients :

$$A := (a_{ij})$$

sans préciser le nombre de ligne et colonne lorsque ceux-ci sont connus ou non nécessaires suivant le contexte.

Attention : par convention les vecteurs seront, sauf mention du contraire, considérés comme des vecteurs colonnes.

Définition 5.2 :

On appelle **transposée** d'une matrice $A = (a_{ij}) \in \mathcal{M}_{mn}(E)$, la matrice notée A^T définie par :

$$A^T := (a_{ij}^T = a_{ji}) = \begin{pmatrix} a_{11} & \cdots & \cdots & \cdots & a_{m1} \\ \vdots & \ddots & \ddots & & \vdots \\ \vdots & \ddots & a_{ii} & \ddots & \vdots \\ \vdots & & \ddots & \ddots & \vdots \\ a_{1n} & \cdots & \cdots & \cdots & a_{mn} \end{pmatrix} \in \mathcal{M}_{nm}(E)$$

Lorsque l'ensemble E a un peu de « structure », comme par exemple $E = \mathbb{R}$ ou $E = \mathbb{C}$, les opérations sur l'ensemble E peuvent en général s'étendre ou servir à définir de nouvelles opérations sur l'ensemble des matrices $\mathcal{M}_{mn}(E)$. En particulier cette année nous nous limiterons la majorité du temps au cas où E est $\mathbb{R}$ avec sa structure canonique de $\mathbb{R}$ –espace vectoriel.

Remarque 5.1 :

La structure sur l'espace E est aussi, voire plus importante que l'ensemble E lui-même. En guise d'exemple, on rappelle que l'ensemble $\mathbb{R}_+^*$ n'est pas un espace-vectoriel pour la structure additive usuelle des réels (l'opération $+$). On peut néanmoins définir une structure de $\mathbb{R}$ –espace vectoriel sur $\mathbb{R}_+^*$ en le munissant des opérations suivantes pour $(x, y) \in \mathbb{R}_+^* \times \mathbb{R}_+^*$ et $\lambda \in \mathbb{R}$:

$$x \oplus y := xy \quad \lambda \odot x := x^\lambda$$

où xy note la multiplication usuelle et x^λ l'exponentiation usuelle entre réels.

Dans la suite de ce texte et sauf mentions explicites du contraire, nous nous limiterons au cas où $E = \mathbb{R}$ avec sa structure canonique de $\mathbb{R}$ –espace vectoriel.

Lorsque E est un espace vectoriel, en notant 0 l'élément neutre de l'espace vectoriel (pour la structure additive), nous noterons 0_{mn} la matrice de taille (m, n) dont tous les coefficients sont 0 . On appelle une telle matrice : **une matrice nulle**. Nous ferons régulièrement l'abus de noter 0 sans préciser la taille de la matrice lorsque le contexte suffit à connaître la taille.

La structure additive sur l'espace vectoriel permet de définir l'addition (ou somme) de deux matrices de même taille :

Définition 5.3 :

Soient A, B deux matrices de même taille à coefficients réels. On définit **la somme de A et B** , notée $A + B$ par :

$$A + B := (a_{ij} + b_{ij})$$

L'autre opération naturelle qui apparaît sur les matrices est la multiplication matricielle.

Définition 5.4 :

Soient $A \in \mathcal{M}_{mn}(\mathbb{R}), B \in \mathcal{M}_{np}(\mathbb{R})$ deux matrices dont le nombre de colonne de la première est égal au nombre de ligne de la seconde. On définit **le produit de A et B** , noté AB par :

$$AB := C = (c_{ij}) := \left(\sum_{k=1}^n a_{ik} b_{kj} \right)$$

Citons quelques propriétés simples :

Proposition 5.1 : Propriétés élémentaires de la transposition

- $\forall A \in \mathcal{M}_{mn}(\mathbb{R}), (A^T)^T = A$
- $\forall (A, B) \in \mathcal{M}_{mn}(\mathbb{R})^2, (A + B)^T = A^T + B^T$
- $\forall (A, B) \in \mathcal{M}_{mn}(\mathbb{R}) \times \mathcal{M}_{np}(\mathbb{R}), (AB)^T = B^T A^T$

La transposition permet en particulier de traduire des opérations entre vecteurs ou matrices usuelles comme le rappelle l'exemple suivant.

Corollaire 5.1 : Version matricielle du produit scalaire euclidien

Soit $(x, y) \in \mathbb{R}^n \times \mathbb{R}^n$ deux vecteurs de $\mathbb{R}^n$ que nous considérerons donnés dans la base canonique et sous forme de vecteurs colonnes :

$$x := \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} \quad \text{et} \quad y := \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}$$

Le produit scalaire euclidien s'écrit alors sous forme matricielle :

$$\langle x, y \rangle = \sum_{i=1}^n x_i y_i = x^T y = y^T x$$

Parmi les matrices particulières qui interviendront cette année citons :

Définition 5.5 :

Soit $A = (a_{ij}) \in \mathcal{M}_{mn}(\mathbb{R})$. Nous dirons que A est :

- **diagonale** si : $a_{ij} = 0$ pour tout $i \neq j$,

Dans ce cas il nous arrivera de noter une telle matrice de la manière suivante :

$$A := \text{diag}(a_{11}, a_{22}, \dots, a_{kk})$$

avec $k = \min(m, n)$.

- **à bande de largeur** $2q + 1$, où $q \in \mathbb{N}^*$, si : $a_{ij} = 0$ pour tout $|i - j| > q$.
- **triangulaire supérieure** si : $a_{ij} = 0$ pour tout $i > j$,
- **triangulaire inférieure** si : $a_{ij} = 0$ pour tout $i < j$.

Rappelons brièvement encore quelques notions importantes que nous utiliserons régulièrement cette année.

Définition 5.6 : Image et noyau

Soit $A = (a_{ij}) \in \mathcal{M}_{mn}(\mathbb{R})$. On appelle :

- **noyau** de A , l'ensemble noté :

$$\ker(A) := \{x \in \mathbb{R}^n \mid Ax = 0_m\}$$

- **image** de A , l'ensemble noté :

$$\text{Im}(A) := \{y \in \mathbb{R}^m \mid \exists x \in \mathbb{R}^n, Ax = y\}$$

Géométriquement, en considérant le produit scalaire euclidien usuel, le noyau de A est l'ensemble des vecteurs orthogonaux aux lignes de A .

On rappelle les propriétés évidentes :

Proposition 5.2 :

Les ensembles $\ker(A)$ et $\text{Im}(A)$ sont respectivement des sous-espaces vectoriels de $\mathbb{R}^n$, respectivement $\mathbb{R}^m$.

Enfin rappelons une notion importante : la notion de rang d'une matrice.

Soit $A \in \mathcal{M}_{mn}(\mathbb{R})$. On appelle :

- **le rang en colonnes** de A , l'entier noté :

$$r_c(A) := \dim(\text{Im}(A))$$

- **le rang en lignes** de A , l'entier noté :

$$r_l(A) := r_c(A^T)$$

On a alors la propriété suivante :

Proposition 5.3 :

Soit $A \in \mathcal{M}_{mn}(\mathbb{R})$, on a : $r_c(A) = r_l(A)$.

On notera alors :

$$rg(A) := r_c(A) = r_l(A)$$

et on appellera **rang de** A ce nombre.

Citons pour finir le très important (d'un point de vue structurel) théorème du rang :

Théorème 5.1 : Théorème du rang

Soit $A \in \mathcal{M}_{mn}(\mathbb{R})$. On a l'égalité suivante :

$$rg(A) + \dim(ker(A)) = n$$

En particulier cela signifie pour une matrice A de taille (m, n) que :

$$rg(A) \leq \min(m, n)$$

Si le rang de A est égal au nombre de lignes (ici m) nous parlerons de **matrice de rang plein en lignes**. Respectivement si le rang de A est égal au nombre de colonnes (ici n) nous parlerons de **matrice de rang plein en colonnes**.

Lors de cette troisième année nous allons utiliser en permanence l'outil informatique afin de réaliser des calculs qui auraient été impossibles de faire à la main. Voyons un exemple rapide du maniement des matrices et vecteurs en Python que nous commenterons en classe.

Un exemple en Python : la fougère de Barnsley La fougère de Barnsley est un motif fractal, ressemblant à une fougère, réalisé en utilisant le déplacement aléatoire d'un point dans le plan. On commence par choisir un point initial, ici le point de coordonnées $P_0 := (0, 0)$ et on réalise de manière successive un déplacement :

$$P_{n+1} := f_i(P_n)$$

avec f_i une transformation affine choisie aléatoirement parmi quatre transformations affines données. On trace alors l'ensemble :

$$\mathcal{P} := \{P_n \mid n \in \llbracket 0, N \rrbracket\}$$

avec N un nombre entier fixé (noté dans le programme suivant : nbpts).

En Python nous utiliserons tout particulièrement les bibliothèques **NumPy** (pour l'algèbre linéaire), **Matplotlib** (pour tracer avec Python) et **OpenCV** (image, vidéo et machine learning mais nous y reviendrons en cours d'année...). Sous Python les matrices (tableaux) sont créées avec la commande `np.array()` de la bibliothèque NumPy. On rappelle que pour afficher un résultat dans la console, on pourra utiliser la commande `print()`. Nous travaillerons avec l'éditeur Spyder intégré dans la distribution Anaconda¹.

1. Vous pouvez télécharger cette distribution à l'adresse suivante : <https://www.anaconda.com/distribution/>

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 #optionnel : ajoute une barre de progression
5 # pour installation : -dans la console (ou prompt) mettre : pip install tqdm
6 from tqdm import tqdm
7 #desactivation de l'affichage
8 plt.ioff()
9
10 #définition des transformations affines
11 def f1(x):
12     return np.array([[0,0],[0,0.16]]) @ x
13
14 def f2(x) :
15     return np.array([[0.85,0.04],[-0.04,0.85]]) @ x + np.array([[0],
16     [1.6]])
17
18 def f3(x) :
19     return np.array([[0.2,-0.26],[0.23,0.22]]) @ x + np.array([[0],
20     [1.6]])
21
22 def f4(x) :
23     return np.array([[-0.15,0.28],[0.26,0.24]]) @ x + np.array([[0],
24     [0.44]])
25
26 f_liste = [f1, f2, f3, f4]
27
28 nbpts = 3000
29 #point initial
30 x=np.zeros((2,1))
31 #matrice contenant la suite des déplacements futurs
32 composantes=np.zeros((nbpts ,2))
33 composantes[0,:]=x.T
34
35 for i in tqdm(range(1,nbpts)) :
36     #tracé des points
37     plt.plot(composantes[i-1,0],composantes[i-1,1],'.g',)
38     #choix aléatoire d'une transformation affine avec des probabilités respectives 0.01, 0.85, 0.07, 0.07
39     f = np.random.choice(f_liste, p=[0.01, 0.85, 0.07, 0.07])
40     #stockage des déplacements
41     composantes[i,:]=f(composantes[i-1,:].reshape(2,1)).T
42
43 #sauvegarde de l'image
44 plt.savefig('fractale_fougere.jpg')

```

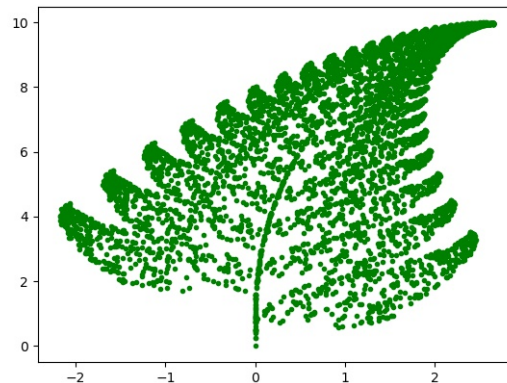


FIGURE 5.1.1. – Sortie du programme : la fougère de Barnsley

5.2. Manipulation des matrices par blocs

Dans certaine situation il sera intéressant de manipuler les matrices par blocs. Comme il est dit dans un gros livre : rien de nouveau sous le soleil, la manipulation des matrices par blocs n'apporte rien de nouveau d'un point de vue formel, l'addition et la multiplication de matrices restent inchangées, mais cela permet de « partitionner » les calculs pour faire apparaître des structures ou produire des algorithmes efficaces. Voyons ce que cela signifie.

Soit $A \in \mathcal{M}_{mn}(\mathbb{R})$. Nous pouvons considérer cette matrice, non pas à coefficients réels, mais à coefficients des matrices (des sous-matrices de A).

Définition 5.7 :

Soit $A \in \mathcal{M}_{mn}(\mathbb{R})$. Une **forme partitionnée de A** (ou plus simplement **une partition**) est une écriture de A sous une forme :

$$A := \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}$$

avec $A_{11} \in \mathcal{M}_{m_1 n_1}(\mathbb{R})$, $A_{12} \in \mathcal{M}_{m_1 n_2}(\mathbb{R})$, $A_{21} \in \mathcal{M}_{m_2 n_1}(\mathbb{R})$ et $A_{22} \in \mathcal{M}_{m_2 n_2}(\mathbb{R})$ où les entiers m_1, m_2, n_1, n_2 vérifient :

$$m_1 + m_2 = m \quad \text{et} \quad n_1 + n_2 = n$$

Attention les entiers m_1, m_2, n_1, n_2 peuvent être nuls. Nous appellerons **motif** ou **taille de la partition** le quadruplet d'entier : (m_1, m_2, n_1, n_2) et les sous-matrices A_{ij} seront appelées **blocs de la partition**.

Beaucoup de notions se rapportant aux matrices ont leurs « versions par blocs ». Citons-en quelques unes. On peut voir la transposition en la **transposition par blocs** de A de la manière suivante :

$$A^T = \begin{pmatrix} A_{11}^T & A_{21}^T \\ A_{12}^T & A_{22}^T \end{pmatrix}$$

Soit $B \in \mathcal{M}_{mn}(\mathbb{R})$ une seconde matrice de même taille que A et même taille de partition. Alors on définit l'**addition par blocs de A et B** par :

$$A + B := \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} + \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} = \begin{pmatrix} A_{11} + B_{11} & A_{12} + B_{12} \\ A_{21} + B_{21} & A_{22} + B_{22} \end{pmatrix}$$

De même on peut définir une multiplication par blocs et comme dans le cas de l'addition il faudra faire bien attention aux compatibilités des tailles. Soit $C \in \mathcal{M}_{np}(\mathbb{R})$, partitionnée de la manière suivante :

$$C := \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

avec $C_{11} \in \mathcal{M}_{n_1 p_1}(\mathbb{R})$, $C_{12} \in \mathcal{M}_{n_1 p_2}(\mathbb{R})$, $C_{21} \in \mathcal{M}_{n_2 p_1}(\mathbb{R})$ et $C_{22} \in \mathcal{M}_{n_2 p_2}(\mathbb{R})$. On définit la **multiplication par blocs de A et C** par :

$$AC := \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11}C_{11} + A_{12}C_{21} & A_{11}C_{12} + A_{12}C_{22} \\ A_{21}C_{11} + A_{22}C_{21} & A_{21}C_{12} + A_{22}C_{22} \end{pmatrix}$$

On retrouve la formule classique du produit matriciel mais où cette fois les coefficients scalaires sont remplacés par des matrices.

Ce qui vient d'être dit précédemment peut se généraliser facilement à des partitions plus complexes. En effet les blocs A_{ij} , qui sont des matrices, peuvent également être partitionnés ,etc...

5.3. Le cas particulier des matrices carrées

Dans cette section nous allons rappeler quelques notions et définitions sur le cas particulier des matrices qui ont autant de lignes que de colonnes : les matrices dites carrées.

Fixons un entier $n \in \mathbb{N}^*$.

Définition 5.8 :

Nous dirons qu'une **matrice est carrée de taille (ou d'ordre) n** si $A \in \mathcal{M}_{nn}(E)$.

Dans ce cas nous noterons :

$$\mathcal{M}_n(E) := \mathcal{M}_{nn}(E)$$

Citons pour rappel, plusieurs types de matrices carrées qui interviendront régulièrement dans le cours de cette troisième année dans le cas où $E = \mathbb{R}$:

Une matrice importante en raison de ses propriétés structurelles est la matrice identité.

Définition 5.9 :

On appelle **matrice identité de taille n** , noté $I_n \in \mathcal{M}_n(\mathbb{R})$, la matrice définie par :

$$I_n := (\delta_{ij})$$

où δ_{ij} est le symbole de Kronecker ^a.

a. On rappelle que le symbole de Kronecker δ_{ij} vaut 1 si $i = j$ et 0 sinon.

Donnons pêle-mêle les exemples suivants :

Définition 5.10 :

Soit A une matrice carrée de taille n :

- si $A = A^T$ alors la matrice A est dite **matrice symétrique**,
- si $A = -A^T$ alors la matrice A est dite **matrice antisymétrique**,
- la matrice A est **symétrique semi-définie positive** si A est symétrique et :

$$\forall x \in \mathbb{R}^n, x^T A x \geq 0$$

- la matrice A est **symétrique définie positive** si A est symétrique et :

$$\forall x \in \mathbb{R}^n, x^T A x \geq 0 \text{ et } (x^T A x = 0 \iff x = 0_{\mathbb{R}^n})$$

- s'il existe une matrice B de taille n telle que : $AB = I_n$, nous dirons que A est **inversible** (ou **non-singulière**) et nous appellerons B son **inverse**. On notera classiquement A^{-1} l'inverse d'une matrice A lorsqu'il existe.

D'après le théorème du rang, la dernière propriété est équivalente à l'existence d'une matrice C de taille n telle que :

$$CA = I_n$$

De plus on en déduit l'unicité de l'inverse (ce qui justifie la notation) lorsqu'un inverse existe. Ainsi on retiendra la propriété simple suivante :

Proposition 5.4 :

Soit A une matrice carrée de taille n alors A est inversible si et seulement si $rg(A) = n$.

Parmi les matrices inversibles, retenons les matrices orthogonales. Ces matrices sont les grandes amies des apprentis physiciens, elles incarnent les changements de repère orthonormé mais également des « numériciens » en herbe pour leurs propriétés « pratiques/effectives ».

Définition 5.11 :

Une matrice carrée $U \in \mathcal{M}_n(\mathbb{R})$ est dite **orthogonale** si elle est inversible et si son inverse est sa transposée U^T i.e :

$$U^T U = I_n \quad \text{et} \quad U U^T = I_n$$

On note $\mathbb{O}_n$ l'ensemble des matrices orthogonales de taille n .

De la définition découle directement la proposition suivante.

Proposition 5.5 :

Notons u_i les vecteurs colonnes d'une matrice orthogonale $U \in \mathbb{O}_n$. On a alors :

$$u_i^T u_j = \begin{cases} 0 & \text{si } i \neq j \\ 1 & \text{si } i = j \end{cases}$$

Ainsi les vecteurs colonnes d'une matrice orthogonale forment une base orthonormée de $\mathbb{R}^n$ pour le produit scalaire usuel (=euclidien). Le même raisonnement reste vrai pour les vecteurs lignes d'une telle matrice. De même (exercice) il est facile de voir que pour une matrice orthogonale on a une majoration de la norme des coefficients :

$$\forall (i, j) \in \llbracket 1, n \rrbracket^2, \quad |u_{ij}| \leq 1$$

5.3.1. Valeurs propres et vecteurs propres

Dans cette sous-section, sauf mention du contraire, A notera une matrice carrée de taille n à coefficients réels. Commençons par rappeler quelques définitions essentielles.

Définition 5.12 :

On appelle :

- **valeur propre** de A tout $\lambda \in \mathbb{C}$ tel qu'il existe $v \in \mathbb{R}^n$, $v \neq 0_{\mathbb{R}^n}$ tel que :

$$Av = \lambda v$$

L'élément v est appelé **vecteur propre** de A associé à la valeur propre λ . L'ensemble des valeurs propres d'une matrice A est appelé **le spectre de A** et sera noté $\text{Spec}(A)$. Nous dirons également que (λ, v) est un **couple propre** de A .

- **rayon spectral** de A le réel positif défini par :

$$\rho(A) = \max \{ |\lambda| \mid \lambda \text{ valeur propre de } A \}$$

Un couple propre (λ, v) vérifie :

$$Av = \lambda v \Leftrightarrow (A - \lambda I_n)v = 0$$

et donc $v \in \ker(A - \lambda I_n)$. Ainsi un vecteur propre est un vecteur non nul appartenant à $\ker(A - \lambda I_n)$.

Les vecteurs propres étant des vecteurs non nuls (i.e. $\neq 0_{\mathbb{R}^n}$), pour que de tels vecteurs existent il faut que la matrice :

$$A - \lambda I_n$$

ne soit pas inversible (dans le cas contraire $\ker(A - \lambda I_n) = \{0\}$). Et par suite les valeurs propres sont les $\lambda \in \mathbb{C}$ tels que :

$$\det(A - \lambda I_n) = 0$$

Cette quantité est un polynôme en λ . De part son importance théorique on pose :

Définition 5.13 :

On appelle **polynôme caractéristique** de la matrice A le polynôme suivant :

$$P_A(\lambda) := \det(A - \lambda I_n)$$

Les valeurs propres d'une matrice carrée A sont donc les racines du polynôme caractéristique.

On rappellera alors le théorème suivant :

Théorème 5.2 : Théorème de Cayley-Hamilton

Le polynôme caractéristique d'une matrice carrée A est annulateur de A i.e. :

$$P_A(A) = 0_{\mathcal{M}_n(\mathbb{R})}$$

Enfin dans cette section sur les matrices carrées on rappelle l'important théorème sur le plan théorique (bien que plus difficile à appliquer de manière pratique sur des « grosses » matrices) : **la décomposition de Jordan**.

Théorème 5.3 : Théorème de la décomposition de Jordan

Pour toute matrice carrée $A \in \mathcal{M}_n(\mathbb{C})$, il existe une matrice inversible $P \in \mathcal{M}_n(\mathbb{C})$ et une matrice $J \in \mathcal{M}_n(\mathbb{C})$ telles que :

$$A = PJP^{-1}$$

et où J est une matrice diagonale par blocs i.e. de la forme suivante :

$$J := \begin{pmatrix} J_1 & & \\ & J_2 & \\ & & \ddots \\ & & & J_p \end{pmatrix}$$

où chaque bloc $J_k \in \mathcal{M}_{n_k}(\mathbb{C})$ avec les n_k vérifiant : $\sum_{k=1}^p n_k = n$ est :

- soit du type :

$$J_k = \lambda_k I_{n_k}$$

- soit du type :

$$J_k = \lambda_k I_{n_k} + N_{n_k}$$

avec N_{n_k} une matrice (nilpotente) carrée de taille n_k de la forme :

$$N_{n_k} := \begin{pmatrix} 0 & 1 & & \\ & 0 & \ddots & \\ & & \ddots & 1 \\ & & & 0 \end{pmatrix}$$

et où les scalaires $\lambda_1, \dots, \lambda_p$ sont l'ensemble des valeurs propres de A .

Dans de nombreux exemples concrets par exemple en modélisation numérique (résolution approchée d'équations différentielles ou aux dérivées partielles) les matrices apparaissant seront symétriques. Un résultat important à garder en tête est donc le suivant.

Théorème 5.4 :

Soit $A \in \mathcal{M}_n(\mathbb{R})$ une matrice carrée symétrique. Alors les valeurs propres de A sont réelles.

Démonstration. Soient $A \in \mathcal{M}_n(\mathbb{R})$ une matrice carrée symétrique et $\lambda \in \text{Spec}(A)$. Notons $v = v_1 + iv_2$ un vecteur associé avec v_1 le vecteur contenant les parties réelles des composantes de v et v_2 le vecteur contenant les parties imaginaire des composantes de v . Pour un vecteur à coefficient complexe v , le module est donnée matriciellement par la multiplication par son conjugué (=conjugué de chaque composante) transposé² :

$$|v| = \bar{v}^T v$$

Nous allons utiliser cette idée dans la suite de la démonstration. On a alors d'un côté en utilisant la symétrie de A :

$$\begin{aligned} \bar{v}^T A v &= (v_1 - iv_2)^T A (v_1 + iv_2) \\ &= v_1^T A v_1 + v_2^T A v_2 \end{aligned} \quad \text{car } v_1^T A v_2 = v_2^T A v_1 \text{ par symétrie de } A$$

et en utilisant le fait que v est vecteur propre :

$$\begin{aligned} \bar{v}^T A v &= (v_1 - iv_2)^T A (v_1 + iv_2) \\ &= \lambda (v_1 - iv_2)^T (v_1 + iv_2) \\ &= \lambda (\|v_1\|_2^2 + \|v_2\|_2^2) \end{aligned}$$

Comme ces deux valeurs sont réelles et égales, il vient nécessairement que λ est réel. □

Pour finir cette section rappelons deux applications importantes définies sur les matrices carrées : la trace et le déterminant.

² On appelle pour des vecteurs ou matrices à coefficients complexes l'opération de conjugaison avec transposition : l'opération de **transconjugaison** et le vecteur ou la matrice obtenu(e) le **transconjugué**.

Pour définir le déterminant³ d'une matrice carrée A , on introduit la notation suivante : on notera $A_{|i,j}$ la sous-matrice de A obtenue en supprimant la ligne i et la colonne de j de A .

Définition 5.14 : Définition récursive du déterminant

Soit A une matrice carrée de taille n . On définit le **déterminant** de A , noté $\det(A)$, le scalaire obtenu de la façon suivante :

- si $n = 1$:

$$\det(A) = a_{11}$$

,

- si $n > 1$:

$$\det(A) = \sum_{j=1}^n (-1)^{1+j} a_{1j} \det(A_{|1,j})$$

Les termes $(-1)^{1+j} \det(A_{|1,j})$ sont appelés les **cofacteurs** de A et les termes $\det(A_{|i,j})$ sont appelés des **mineurs**.

Il découle de cette définition les propriétés suivantes que nous rappelons.

Proposition 5.6 : Quelques propriétés basiques du déterminant

Soient A et B des matrices carrées de même taille n à coefficients réels. On a :

- $\det(A^T) = \det(A)$
- $\det(AB) = \det(A) \det(B)$,
- Pour tout $\lambda \in \mathbb{R}$, $\det(\lambda A) = \lambda^n \det(A)$,
- Si A est inversible, $\det(A^{-1}) = \frac{1}{\det(A)}$,
- $\det(I_n) = 1$.

D'un point de vue géométrique le déterminant d'une matrice est l'hyper-volume orienté engendré par les vecteurs colonnes (ou lignes) de la matrice A . Ce volume peut être nul si les vecteurs colonnes (ou lignes) ne sont pas linéairement indépendants.

Définition 5.15 :

Soit A une matrice carrée de taille n . On définit la trace de A , noté $Tr(A)$, le scalaire obtenu de la façon suivante :

$$Tr(A) = \sum_{i=1}^n a_{ii}$$

On peut montrer (exercice) les propriétés suivantes de la trace d'une matrice.

Proposition 5.7 : Quelques propriétés basiques de la trace

Soient A et B des matrices carrées de même taille n à coefficients réels. On a :

- $Tr(A^T) = Tr(A)$
- $Tr(A + B) = Tr(A) + Tr(B)$,
- Pour tout $\lambda \in \mathbb{R}$, $Tr(\lambda A) = \lambda Tr(A)$,
- $Tr(AB) = Tr(BA)$,
- $Tr(I_n) = n$.

L'existence d'une décomposition de Jordan permet de démontrer les propriétés suivantes (que nous élevons au rang de théorème par leurs importances pratiques et théoriques).

3. Il y a de nombreuses manières de définir le déterminant d'une matrice, toutes ces façons étant équivalentes nous avons choisi celle qui nous semble la plus « simple » à introduire au sens où il n'est pas nécessaire d'introduire d'autres notions supplémentaires.

Théorème 5.5 :

Soit $A \in \mathcal{M}_n(\mathbb{C})$ et notons $\lambda_1, \dots, \lambda_n$ l'ensemble des valeurs propres de A comptées avec leurs multiplicités. On a alors :

$$\boxed{Tr(A) = \sum_{i=1}^n \lambda_i} \quad \text{et} \quad \boxed{\det(A) = \prod_{i=1}^n \lambda_i}$$

Démonstration. Avec les notations précédentes, fixons une décomposition de Jordan de A :

$$A = PJP^{-1}$$

On a alors la trace :

$$Tr(A) = Tr(PJP^{-1}) = Tr(P^{-1}PJ) = Tr(J) = \sum_{i=1}^n \lambda_i$$

et pour le déterminant :

$$\det(A) = \det(PJP^{-1}) = \det(P)\det(J)\det(P^{-1}) = \det(J) = \prod_{i=1}^n \lambda_i$$

car le déterminant d'une matrice triangulaire est égal au produit de ses termes diagonaux. □

5.4. Lien entre matrice et application linéaire

Nous allons commencer par rappeler la définition d'une application linéaire entre deux espaces vectoriels puis nous reverrons brièvement le lien de cette notion avec la notion de matrice. Lorsqu'il n'y aura pas d'ambiguïté possible nous ferons l'abus de notation de noter avec un « + » la structure additive des espaces vectoriels.

Nous noterons $\mathbb{K}$ un ensemble⁴ qui dans la majorité des cas que nous aurons à traiter sera $\mathbb{R}$ ou $\mathbb{C}$ avec leurs structures canoniques.

Définition 5.16 :

Soient E et F deux $\mathbb{K}$ -espaces vectoriels. On dit qu'une application :

$$f : E \rightarrow F$$

est $\mathbb{K}$ -linéaire si pour tout $(\lambda, x, y) \in \mathbb{K} \times E \times E$:

$$f(x + \lambda y) = f(x) + \lambda f(y)$$

Par exemple dans la section précédente, nous avons vu que la trace vue comme une application :

$$trace : \mathcal{M}_n(\mathbb{C}) \rightarrow \mathbb{C}$$

est une application linéaire. Le déterminant vu comme application entre les vecteurs colonnes d'une matrice lui est une application n -linéaire (= linéaire par rapport à chacune de ses n variables).

Remarque 5.2 :

Encore une fois ceux sont les structures sur l'espace E et F qui sont importantes dans la définition de linéarité. En reprenant l'exemple de l'ensemble $\mathbb{R}_+^*$ muni de la structure de $\mathbb{R}$ -espace vectoriel sur $\mathbb{R}_+^*$ définie de la manière suivante pour $(x, y) \in \mathbb{R}_+^* \times \mathbb{R}_+^*$ et $\lambda \in \mathbb{R}$:

$$x \oplus y := xy \quad \lambda \odot x := x^\lambda$$

Il est facile de montrer que la fonction logarithme :

$$\ln : \mathbb{R}_+^* \rightarrow \mathbb{R}$$

est une application $\mathbb{R}$ -linéaire (où $\mathbb{R}$ est muni de sa structure de $\mathbb{R}$ -espace vectoriel canonique). De même la fonction exponentielle :

$$\exp : \mathbb{R} \rightarrow \mathbb{R}_+^*$$

est une application $\mathbb{R}$ -linéaire au sens précédent.

4. Pour être précis $\mathbb{K}$ doit être ce que l'on nomme un corps i.e. un ensemble muni de lois vérifiant certains propriétés calculatoires.

Le lien entre applications linéaires (entre espaces vectoriels de dimension finie) et les matrices est le suivant. Soient E et F deux $\mathbb{K}$ -espaces vectoriels de dimension finie. Notons $n = \dim(E)$ et $m = \dim(F)$. Considérons une application :

$$f : E \rightarrow F$$

Fixons $\mathcal{B} := (e_1, \dots, e_n)$ une base de E et $\mathcal{B}' := (e'_1, \dots, e'_m)$ une base de F . On définit alors une matrice, notée $Mat_{\mathcal{B}, \mathcal{B}'}(f)$, appelée **la matrice de f dans les bases $\mathcal{B}$ et $\mathcal{B}'$** , la matrice définie de la manière suivante : les colonnes de $Mat_{\mathcal{B}, \mathcal{B}'}(f)$ sont les vecteurs ordonnés $f(e_i)$ exprimés dans la base $\mathcal{B}'$.

Ce que l'on peut représenter graphiquement sous la forme :

$$Mat_{\mathcal{B}, \mathcal{B}'}(f) := \begin{pmatrix} \overset{f(e_1)}{\downarrow} a_{11} & \cdots & \overset{f(e_n)}{\downarrow} a_{1n} \\ \vdots & & \vdots \\ \underset{\uparrow}{a_{m1}} & \cdots & a_{mn} \end{pmatrix}$$

expression de $f(e_1)$ dans la base $\mathcal{B}'$

où les $a_{ij} \in \mathbb{K}$.

On peut utiliser cette forme matricielle pour évaluer l'application linéaire. En effet avec les notations précédentes, si l'on note $x \in \mathbb{K}^n$ exprimé dans la base $\mathcal{B}$:

$$x := \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix}$$

Par convention on rappelle que les composantes des vecteurs seront représentées par des vecteurs colonnes. On peut calculer « la valeur » de $y := f(x)$ dans la base $\mathcal{B}'$, i.e. les composantes du vecteur $f(x)$ dans la base $\mathcal{B}'$, grâce à la matrice $Mat_{\mathcal{B}, \mathcal{B}'}(f)$ et son produit matriciel avec x :

$$y = f(x) = \begin{pmatrix} y_1 \\ \vdots \\ y_m \end{pmatrix} = Mat_{\mathcal{B}, \mathcal{B}'}(f) x$$

Corollaire 5.2 :

Voyons un exemple simple. Soit l'application $f : \mathbb{R}^3 \rightarrow \mathbb{R}^3$ définie par :

$$f(x, y, z) := (x - y, x + 2z, 2x - y + 2z)$$

Il est facile de vérifier que cette application est linéaire. Choisissons la base canonique de $\mathbb{R}^3$ pour $\mathcal{B}$ ainsi que pour $\mathcal{B}'$. Nous allons exprimer $Mat_{\mathcal{B}, \mathcal{B}'}(f)$ pour ce choix de bases. Notons : $e_1 = (1, 0, 0)$, $e_2 = (0, 1, 0)$ et $e_3 = (0, 0, 1)$ les vecteurs de la base canonique. On a alors :

$$f(e_1) = (1, 1, 2) \quad f(e_2) = (-1, 0, -1) \quad f(e_3) = (0, 2, 2)$$

En identifiant $\mathbb{R}^3$ avec les matrices colonnes $\mathcal{M}_{3,1}(\mathbb{R})$ on obtient :

$$Mat_{\mathcal{B}, \mathcal{B}'}(f) = \begin{pmatrix} 1 & -1 & 0 \\ 1 & 0 & 2 \\ 2 & -1 & 2 \end{pmatrix}$$

On définit le **rang** et le **noyau d'une application linéaire** par le rang et le noyau d'une matrice $Mat_{\mathcal{B}, \mathcal{B}'}(f)$ pour un choix de bases données. On montre que le rang et le noyau sont indépendants des bases choisies.

Formes bilinéaires et formes quadratiques

Dans ce chapitre et comme dans les chapitres précédents nous nous limiterons au cas des $\mathbb{R}$ –espaces vectoriels¹.

6.1. Définitions générales et propriétés de bases

Définition 6.1 :

Soit E un $\mathbb{R}$ –espace vectoriel. On appelle **forme bilinéaire** une application linéaire par rapport à chacune de ses variables de la forme :

$$E \times E \rightarrow \mathbb{R}$$

i.e. à valeurs dans le corps des scalaires, ici : $\mathbb{R}$.

Définition 6.2 :

Soit une forme bilinéaire $f : E \times E \rightarrow \mathbb{R}$. Nous dirons que f est :

- **symétrique** si pour tout $(x, y) \in E \times E$:

$$f(x, y) = f(y, x)$$

- **antisymétrique** si pour tout $(x, y) \in E \times E$:

$$f(x, y) = -f(y, x)$$

Définition 6.3 :

Soit une forme bilinéaire symétrique $f : E \times E \rightarrow \mathbb{R}$. Nous dirons que f est un **produit scalaire** sur E si elle vérifie les deux propriétés suivantes :

- **définie** : pour tout $x \in E$, $f(x, x) = 0 \Rightarrow x = 0$,
- **positive** : pour tout $x \in E$, $f(x, x) \geq 0$.

On résume souvent la définition précédente en disant qu'un produit scalaire est une application bilinéaire symétrique définie positive.

Définition 6.4 :

Soient E un $\mathbb{R}$ –espace vectoriel et $f : E \rightarrow \mathbb{R}$. Nous dirons que f est une **fonction quadratique** (ou **quadrique**) s'il existe une forme bilinéaire symétrique $g : E \times E \rightarrow \mathbb{R}$, une application linéaire $h : E \rightarrow \mathbb{R}$ et $\alpha \in \mathbb{R}$ tels que pour tout $x \in E$:

$$f(x) = g(x, x) + h(x) + \alpha$$

Lorsque la forme linéaire h est la fonction nulle et $\alpha = 0$, on parle de **forme quadratique**.

Nous rencontrerons régulièrement au cours de l'année les quadriques et formes quadratiques lorsque nous nous intéresserons au comportement local de fonction de classe $\mathcal{C}^2$. En effet le développement de Taylor à l'ordre 2 d'une fonction $f : E \rightarrow \mathbb{R}$

1. Les définitions et résultats qui vont suivre trouvent une expression (à quelques modifications près) pour des $\mathbb{K}$ –espaces vectoriels généraux (de dimension finie en particulier).

avec $E = \mathbb{R}^n$ en un point $x \in E$ fixé, s'écrit pour tout vecteur $h \in E$ « suffisamment petit » :

$$f(x+h) = f(x) + \nabla_x f^T h + \frac{1}{2} h^T \text{Hess}_x f h + o\left(\|h\|_2^2\right)$$

où $\|\bullet\|_2$ représente la norme 2. En négligeant le terme en petit o (ce qui fait sens lorsque h est « petit »), l'étude locale du comportement de f autour du point x revient à étudier une fonction quadratique :

$$q(h) := f(x) + \nabla_x f^T h + \frac{1}{2} h^T \text{Hess}_x f h$$

De plus lorsque x est un point critique i.e. que

$$\nabla_x f = 0_n$$

où 0_n note le vecteur nul de $\mathbb{R}^n$, la fonction q est une forme quadratique.

Comme pour les applications linéaires, les formes bilinéaires sont reliées aux matrices de la manière suivante. Fixons une base $\mathcal{B} := (e_1, \dots, e_n)$ de l'espace vectoriel $E = \mathbb{R}^n$ et une quadrique $f : E \rightarrow \mathbb{R}$. Alors par définition il existe une forme bilinéaire symétrique $g : E \times E \rightarrow \mathbb{R}$, une application linéaire $h : E \rightarrow \mathbb{R}$ et $\alpha \in \mathbb{R}$ tels que pour tout $x \in E$:

$$f(x) = g(x, x) + h(x) + \alpha$$

En notant de manière matricielle par abus : $x := (x_1, \dots, x_n)^T$, on a alors dans la base $\mathcal{B}$ l'expression matricielle de $f(x)$ sous la forme :

$$f(x) = x^T G x + H^T x + \alpha$$

avec $G := (g(e_i, e_j))_{(i,j) \in \llbracket 1, n \rrbracket^2}$ et $H := (h(e_1), \dots, h(e_n))^T$.

Produit de Kronecker

Dans le chapitre qui suit nous allons introduire une opération importante entre les matrices : le produit de Kronecker. Nous reverrons cette notion très souvent en troisième année. Commençons par deux définitions qui nous seront utiles pour réécrire les systèmes linéaires.

Définition 7.1 :

Pour une matrice $A \in \mathcal{M}_{mn}(\mathbb{R})$, on définit l’**opérateur vec** , noté $\text{vec}(A)$, par la formule suivante :

$$\text{vec}(A) := \begin{pmatrix} a_1 & a_2 & \cdots & a_n \end{pmatrix}^T$$

où $a_i := (a_{1i}, a_{2i}, \dots, a_{mi})^T$ est le i -ième vecteur colonne de la matrice A .

Définition 7.2 :

Soient $A \in \mathcal{M}_{mn}(\mathbb{R})$ et $B \in \mathcal{M}_{st}(\mathbb{R})$ deux matrices. On définit le **produit de Kronecker** de A et B , noté $A \otimes B$, par la matrice par blocs suivante :

$$A \otimes B := \begin{pmatrix} a_{11}B & \cdots & a_{1n}B \\ \vdots & & \vdots \\ a_{m1}B & \cdots & a_{mn}B \end{pmatrix}$$

La matrice $A \otimes B$ est donc une matrice à $(m \times s)$ lignes et $(n \times t)$ colonnes.

Ainsi le produit de Kronecker est défini pour tout couple de matrice et cela sans restriction de taille.

Citons quelques propriétés du produit de Kronecker.

Proposition 7.1 :

Soient A, B, C et D des matrices quelconques (non nécessairement carrées). Lorsque les opérations suivantes sont biens définies nous avons :

- $(A \otimes B)^T = A^T \otimes B^T$
- $A \otimes (B \otimes C) = (A \otimes B) \otimes C$
- $(AB) \otimes (CD) = (A \otimes C)(B \otimes D)$
- $(A + B) \otimes (C + D) = (A \otimes C) + (A \otimes D) + (B \otimes C) + (B \otimes D)$

Dans le cas où les matrices sont carrées nous retiendrons les résultats suivants.

Proposition 7.2 :

Supposons à présent que $A \in \mathcal{M}_m(\mathbb{R})$ et $B \in \mathcal{M}_n(\mathbb{R})$ sont des matrices carrées. On a les résultats suivants :

- Soient λ une valeur propre de A et v un vecteur propre associé, μ une valeur propre de B et w un vecteur propre associé.
 - $\lambda\mu$ est une valeur propre de $A \otimes B$ associée au vecteur propre $v \otimes w$.
 - Les valeurs propres de $A \otimes B$ sont exactement les produits des valeurs propres de A et de B .
- $\text{Tr}(A \otimes B) = \text{Tr}(A) \times \text{Tr}(B)$
- $\det(A \otimes B) = \det(A)^n \times \det(B)^m$
- Si A et B sont inversibles alors $A \otimes B$ est inversible et l'on a :

$$(A \otimes B)^{-1} = A^{-1} \otimes B^{-1}$$

Démonstration. Les premières assertions sont évidentes. Pour l'inverse on a bien :

$$(A \otimes B) (A^{-1} \otimes B^{-1}) = (AA^{-1}) \otimes (BB^{-1}) = I_m \otimes I_n = I_{mn}$$

Soient (λ, v) et (μ, w) deux couples propres de A et B respectivement. Alors il est clair que :

$$(Av) \otimes (Bw) = (\lambda v) \otimes (\mu w) = \lambda\mu (v \otimes w)$$

Montrons que les valeurs propres de $A \otimes B$ sont exactement les produits des valeurs propres de A et de B . On se place dans $\mathbb{C}$ et l'on note les décompositions de Jordan de A et B respectivement :

$$A = PJ_1P^{-1} \quad B = QJ_2Q^{-1}$$

Les matrices $(P \otimes Q) (A \otimes B) (P^{-1} \otimes Q^{-1})$ et $A \otimes B$ étant semblables (égales à changement de base près) on a :

$$\begin{aligned} (P \otimes Q) (A \otimes B) (P^{-1} \otimes Q^{-1}) &= ((PA) \otimes (QB)) (P^{-1} \otimes Q^{-1}) \\ &= (PAP^{-1}) \otimes (QBQ^{-1}) \\ &= J_1 \otimes J_2 \end{aligned}$$

d'où : $\text{Spec}(A \otimes B) = \text{Spec}(J_1 \otimes J_2) = \{\lambda_i \mu_j \mid \lambda_i \in \text{Spec}(A) \quad \mu_j \in \text{Spec}(B)\}$ □

Dans ce cours nous étudions les systèmes linéaires s'écrivant sous la forme suivante :

$$Ax = b$$

avec A une matrice et x et b des vecteurs (x étant le vecteur inconnu). Le grand intérêt pour nous de l'opérateur vec du produit de Kronecker est que ces opérations permettent de transformer certains systèmes linéaires d'équations complexes en des systèmes linéaires de la forme précédente.

Dans ce cours nous étudions les systèmes linéaires s'écrivant sous la forme suivante :

$$Ax = b$$

avec A une matrice et x et b des vecteurs (x étant le vecteur inconnu). Nous allons voir que l'opérateur vec et le produit de Kronecker permettent de transformer certains systèmes linéaires d'équations « complexes » en des systèmes linéaires de la forme précédente.

On cherche à résoudre les équations de la forme suivante :

$$AXB = C$$

où $A \in \mathcal{M}_{mn}(\mathbb{R})$, $B \in \mathcal{M}_{st}(\mathbb{R})$ et $C \in \mathcal{M}_{mt}(\mathbb{R})$ sont des matrices données et $X \in \mathcal{M}_{ns}(\mathbb{R})$ est une matrice inconnue.

Proposition 7.3 :

Avec les notations précédentes on a :

$$AXB = C \Leftrightarrow (B^T \otimes A) \text{vec}(X) = \text{vec}(C)$$

Démonstration. On note : $X := \begin{pmatrix} x_1 & x_2 & \cdots & x_s \end{pmatrix}$ la matrice X vue en blocs colonnes (i.e. les x_i sont les vecteurs colonnes de X). On a :

$$\begin{aligned} AXB &= A \begin{pmatrix} x_1 & x_2 & \cdots & x_s \end{pmatrix} B \\ &= \begin{pmatrix} Ax_1 & Ax_2 & \cdots & Ax_s \end{pmatrix} B \\ &= \begin{pmatrix} Ax_1 & Ax_2 & \cdots & Ax_s \end{pmatrix} \begin{pmatrix} b_{11} & b_{12} & \cdots & b_{1t} \\ b_{21} & b_{22} & \ddots & \vdots \\ \vdots & \ddots & \ddots & \vdots \\ b_{s1} & \cdots & \cdots & b_{st} \end{pmatrix} \\ &= \begin{pmatrix} \sum_{k=1}^s b_{k1} Ax_k & \sum_{k=1}^s b_{k2} Ax_k & \cdots & \sum_{k=1}^s b_{kt} Ax_k \end{pmatrix} \end{aligned}$$

où la dernière matrice est encore vue par blocs colonnes. Appliquons l'opération vec (=vectorisation) on a alors :

$$\begin{aligned} vec(AXB) &= \begin{pmatrix} \sum_{k=1}^s b_{k1} Ax_k \\ \sum_{k=2}^s b_{k2} Ax_k \\ \vdots \\ \sum_{k=1}^s b_{kt} Ax_k \end{pmatrix} \\ &= \begin{pmatrix} b_{11}A & b_{21}A & \cdots & b_{s1}A \\ b_{12}A & b_{22}A & \ddots & \vdots \\ \vdots & \ddots & \ddots & \vdots \\ b_{1t}A & \cdots & \cdots & b_{st}A \end{pmatrix} vec(X) \\ &= (B^T \otimes A) vec(X) \end{aligned}$$

D'où le résultat en revenant à l'égalité initiale :

$$AXB = C \Leftrightarrow (B^T \otimes A) vec(X) = vec(C)$$

□

Corollaire 7.1 :

En se servant du résultat précédent nous allons résoudre l'équation suivante :

$$\begin{pmatrix} 1 & 3 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} x_{11} & x_{12} \\ x_{21} & x_{22} \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 2 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

D'après ce qui précède on obtient :

$$\begin{aligned} &\begin{pmatrix} 1 & 3 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} x_{11} & x_{12} \\ x_{21} & x_{22} \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 2 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \\ \Leftrightarrow &\begin{pmatrix} 1 & 3 & 2 & 6 \\ 1 & 0 & 2 & 0 \\ 0 & 0 & 1 & 3 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} x_{11} \\ x_{21} \\ x_{12} \\ x_{22} \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \end{pmatrix} \\ \Leftrightarrow &\begin{cases} x_{11} = -2 \\ x_{21} = 1 \\ x_{12} = 1 \\ x_{22} = -\frac{1}{3} \end{cases} \end{aligned}$$

Ainsi la matrice solution est :

$$X = \begin{pmatrix} -2 & 1 \\ 1 & -\frac{1}{3} \end{pmatrix}$$

Nous allons voir un second type d'équation que nous allons pouvoir résoudre avec le produit de Kronecker.

Proposition 7.4 :

Soit le système :

$$AX + XB = C$$

où $A \in \mathcal{M}_m(\mathbb{R})$, $B \in \mathcal{M}_n(\mathbb{R})$ et $C \in \mathcal{M}_{mn}(\mathbb{R})$ sont des matrices données et $X \in \mathcal{M}_{mn}(\mathbb{R})$ est une matrice inconnue. Alors on a :

$$AX + XB = C \Leftrightarrow [(I_n \otimes A) + (B^T \otimes I_m)] \text{vec}(X) = \text{vec}(C)$$

Démonstration. Comme l'opération vectorisation est linéaire on a d'après ce qui précède :

$$\begin{aligned} & AX + XB = C \\ \Leftrightarrow & AXI_n + I_mXB = C \\ \Leftrightarrow & \text{vec}(AXI_n + I_mXB) = \text{vec}(C) \\ \Leftrightarrow & \text{vec}(AXI_n) + \text{vec}(I_mXB) = \text{vec}(C) \\ \Leftrightarrow & (I_n \otimes A) \text{vec}(X) + (B^T \otimes I_m) \text{vec}(X) = \text{vec}(C) \\ \Leftrightarrow & [(I_n \otimes A) + (B^T \otimes I_m)] \text{vec}(X) = \text{vec}(C) \end{aligned}$$

□

Corollaire 7.2 :

En se servant du résultat précédent nous allons déterminer toutes les matrices carrées X de $\mathcal{M}_2(\mathbb{R})$ dont le produit matriciel commute avec la matrice suivante :

$$A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$$

(i.e tel que : $AX = XA$). Le problème se ramène à la résolution de l'équation :

$$AX - XA = 0_{2,2} \Leftrightarrow [(I_2 \otimes A) - (A^T \otimes I_2)] \text{vec}(X) = 0_4$$

Ainsi il nous faut résoudre :

$$\begin{aligned} & [(I_2 \otimes A) - (A^T \otimes I_2)] \text{vec}(X) = 0_4 \\ \Leftrightarrow & \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ -1 & 0 & 0 & 1 \\ 0 & 0 & -1 & 0 \end{pmatrix} \begin{pmatrix} x_{11} \\ x_{21} \\ x_{12} \\ x_{22} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} \\ \Leftrightarrow & \boxed{X = \begin{pmatrix} \lambda & 0 \\ 0 & \lambda \end{pmatrix} \quad \lambda \in \mathbb{R}} \end{aligned}$$

Chapitre 8

Un peu de python

Dans ce court chapitre nous rappellerons quelques commandes Python importante pour le calcul matriciel. Il forme un court complément/aide aux programmes Python donnés régulièrement dans ce cours. Un problème (ou une souplesse pour certains...) avec Python est que ce langage ne possède pas de manière native les outils nécessaires à l'algèbre linéaire. Nous utiliserons tout particulièrement les bibliothèques **NumPy** (pour l'algèbre linéaire) **Matplotlib** (pour tracer avec Python) et **OpenCV** (image, vidéo et machine learning). Il est conseillé de débiter chacun de ses programmes par l'importation « réflexe » de ces trois librairies.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import cv2
```

Sous Python les matrices (tableaux) sont créées avec la commande `np.array()`. On rappelle que pour afficher un résultat dans la console, on pourra utiliser la commande `print()`. Nous travaillerons avec l'éditeur Spyder intégré dans la distribution Anaconda¹.

Cette section donnera un aperçu rapide des commandes de bases de la bibliothèque NumPy. Nous survolerons les commandes par des exemples car :

- les détails des commandes se trouvent aisément sur internet,
- et nous pensons que la programmation s'apprend efficacement par la pratique et non en « lisant » des lignes de codes.

8.1. Les commandes de base

Pour écrire une matrice (ou un vecteur) dans Python, nous allons utiliser le type `array`. Par exemple :

```
1 np.array([2,3,2])
```

produit le vecteur « ligne/colonne » :

$$\begin{pmatrix} 2 & 3 & 2 \end{pmatrix}$$

Il faut bien ici faire attention au fait que cette écriture produit un vecteur d'un type étrange, ni ligne, ni colonne, c'est en fait une sorte de « liste » composées des coefficients 2, 3 et 2. Pour avoir un « vrai » vecteur ligne il nous faut ajouter les crochets [] comme suit :

```
1 np.array([[2,3,2]])
```

dans ce cas on aura bien une matrice de taille 1×3 comme la commande `np.shape` permet de le vérifier :

```
1 np.shape(np.array([[2,3,2]]))
```

qui donne comme sortie :

$$(1, 3)$$

La commande `np.shape` permet de récupérer sous la forme d'un « tuple » le nombre de ligne et le de colonne d'un `array`.

Une solution alternative à l'ajout des crochets et d'utiliser l'option `np.newaxis`. Ainsi la commande :

```
1 np.array([2,3,2]).reshape(-1,1)
```

1. Vous pouvez télécharger cette distribution à l'adresse suivante : <https://www.anaconda.com/distribution/>

renvoie une matrice de taille 3×1 i.e. un vrai vecteur colonne.

Si maintenant nous souhaitons avoir une « vraie » matrice i.e. avoir plusieurs lignes et colonnes il suffit de rajouter les lignes en ajoutant des blocs de la forme `[ ]` séparés par des virgules. Par exemple :

```
1 np.array([[1,0,0],[0,2,0],[0,0,3]])
```

renvoie la matrice suivante :

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{pmatrix}$$

La librairie NumPy propose également des types « pré-rentés » de matrices :

- La commande :

```
1 np.zeros((2,3))
```

renvoie une matrice composée uniquement de 0 de taille 2×3 :

$$\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

- La commande :

```
1 np.ones((3,2))
```

renvoie une matrice composée uniquement de 1 de taille 3×2 :

$$\begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{pmatrix}$$

- La commande :

```
1 np.eye(4)
```

renvoie la matrice identité de taille 4×4 :

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Définissons une matrice :

```
1 M=np.array([[0,1],[2,3],[4,5]])
```

i.e. la matrice :

$$\begin{pmatrix} 0 & 1 \\ 2 & 3 \\ 4 & 5 \end{pmatrix}$$

Pour récupérer les coefficients d'une telle matrice il suffit d'utiliser les commandes suivantes :

- La commande :

```
1 M[0,1]
```

renvoie le nombre 1 : première ligne (Python commençant en 0 pour les indices) et seconde colonne.

- La commande :

```
1 M[:,1]
```

renvoie la seconde colonne de M sous forme d'un vecteur ligne/colonne :

$$(1 \ 3 \ 5)$$

- La commande :

```
1 M[0,:]
```

renvoie la première ligne de M sous forme d'un vecteur ligne/colonne :

$$(0 \ 1)$$

Voyons à présent quelques opérations sur et entre les matrices ($=np.array$).

- L'opération de transposition s'écrit simplement :

```
1 M.T
```

et renvoie la matrice suivante :

$$\begin{pmatrix} 0 & 2 & 4 \\ 1 & 3 & 5 \end{pmatrix}$$

Attention cependant l'opération de transposition n'affecte pas les vecteurs de type « ligne/colonne ». Par exemple :

```
1 np.array([[2,3,2]]).T
```

renvoie le même vecteur de type « ligne/colonne ».

- On peut également sommer les coefficients des colonnes :

```
1 np.sum(M,axis=0)
```

ce qui donne le vecteur ligne/colonne :

$$\begin{pmatrix} 6 & 9 \end{pmatrix}$$

- ou sommer les coefficients des lignes :

```
1 np.sum(M,axis=1)
```

ce qui renvoie le vecteur ligne/colonne :

$$\begin{pmatrix} 1 & 5 & 9 \end{pmatrix}$$

Définissons deux matrices que nous allons stocker sous les noms de variables A et B :

```
1 A,B=np.array([[0,1],[2,1],[3,2]]),np.array([[1,0],[0,1],[4,5]])
```

qui incarne donc les matrices :

$$A = \begin{pmatrix} 0 & 1 \\ 2 & 1 \\ 3 & 2 \end{pmatrix} \quad B = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 4 & 5 \end{pmatrix}$$

Nous pouvons alors :

- les sommer :

```
1 A+B
```

qui donne :

$$\begin{pmatrix} 1 & 1 \\ 2 & 2 \\ 7 & 7 \end{pmatrix}$$

- les multiplier coefficients par coefficients i.e. le coefficient ij de la nouvelle matrice obtenue est donnée par $a_{ij} \times b_{ij}$:

```
1 A*B
```

qui renvoie :

$$\begin{pmatrix} 0 & 0 \\ 0 & 1 \\ 12 & 10 \end{pmatrix}$$

Cette opération s'appelle le **produit d'Hadamard** (ou **produit de Schur**) de matrices.

- et l'opération la plus importante pour nous : effectuer le produit matriciel (attention aux respects des dimensions). Par exemple :

```
1 np.dot(A,B.T)
```

correspond la matrice :

$$AB^T = \begin{pmatrix} 0 & 1 \\ 2 & 1 \\ 3 & 2 \end{pmatrix} \begin{pmatrix} 1 & 0 & 4 \\ 0 & 1 & 5 \end{pmatrix} = \begin{pmatrix} 0 & 1 & 5 \\ 2 & 1 & 13 \\ 3 & 2 & 22 \end{pmatrix}$$

On pourra également utiliser la commande suivante qui réalise ce produit matriciel :

```
1 A @ B.T
```

- On peut également appliquer des fonctions « usuelles » tous les coefficients d'une matrice :

- Par exemple :

```
1 np.cos(A)
```

renvoie une valeur approchée (les nombres réels n'existant pour un ordinateur...) :

$$\begin{pmatrix} \cos(0) & \cos(1) \\ \cos(2) & \cos(1) \\ \cos(3) & \cos(2) \end{pmatrix} \simeq \begin{pmatrix} 1 & 0.540302 \\ -0.41614684 & 0.54030231 \\ -0.9899925 & -0.41614684 \end{pmatrix}$$

- et pour la fonction exponentielle :

```
1 np.exp(A)
```

renvoie :

$$\begin{pmatrix} \exp(0) & \exp(1) \\ \exp(2) & \exp(1) \\ \exp(3) & \exp(2) \end{pmatrix} \simeq \begin{pmatrix} 0 & 2.7182818 \\ 7.3890561 & 2.7182818 \\ 20.085537 & 7.3890561 \end{pmatrix}$$

8.2. Quelques rappels sur les matrices (carrées principalement)

En guise d'exemple définissons sous Python la matrice C suivante (on peut utiliser le raccourci Ctrl+M pour les petites matrices dans Spyder) :

```
1 C=np.array([[0,1,2],[1,2,1],[7,1,0]])
```

qui donne donc formellement la matrice :

$$C = \begin{pmatrix} 0 & 1 & 2 \\ 1 & 2 & 1 \\ 7 & 1 & 0 \end{pmatrix}$$

Les commandes à connaître sont les suivantes :

- **le calcul du déterminant** :

```
1 np.linalg.det(C)
```

qui donne : -19.000000000000004 . La valeur exacte étant -19 , on peut déjà entrevoir le problème des erreurs d'approximations pour des matrices bien plus grandes...

- **le calcul de l'inverse** :

```
1 np.linalg.inv(C)
```

qui donne la matrice :

$$\begin{pmatrix} 0.052631579 & -0.10526316 & 0.15789474 \\ -0.36842105 & 0.73684211 & -0.10526316 \\ 0.68421053 & -0.36842105 & 0.052631579 \end{pmatrix}$$

(**Exercice** : Comparer avec l'inverse calculé à la main). En particulier si on effectue le **produit matriciel** :

```
1 C @ np.linalg.inv(C)
```

à défaut d'obtenir la matrice identité nous obtenons la sortie suivante :

$$\begin{pmatrix} 1 & 0 & -1.3877788e-17 \\ 0 & 1 & -6.9388939e-18 \\ 0 & 0 & 1 \end{pmatrix}$$

- On peut également obtenir le calcul approché des **valeurs propres** et **vecteurs propres** par la commande :

```
1 np.linalg.eig(C)
```

Cette commande renvoie un tuple dont la première composante représente les valeurs propres (approchées) :

```
1 np.linalg.eig(C)[0]
```

donne le vecteur « ligne/colonne » suivant :

$$\begin{pmatrix} -3.7041829 & 4.5856107 & 1.1185721 \end{pmatrix}$$

et la seconde colonne :

```
1 np.linalg.eig(C)[1]
```

la matrice de passage, ici :

$$\begin{pmatrix} 0.45801262 & -0.43711487 & 0.20207548 \\ 0.074991336 & -0.46646655 & -0.82608933 \\ -0.88577691 & -0.76898605 & 0.52606266 \end{pmatrix}$$

(**Exercice** : comparer avec un calcul fait à la main).

- Il est également possible d'avoir les coefficients du **polynôme caractéristique** grâce à la commande :

```
1 np.poly(C)
```

qui donne en sortie un vecteur « ligne/colonne » :

$$(1 \quad -2 \quad -16 \quad 19)$$

Ces coefficients correspondent en réalité à l'opposée des coefficients du polynôme caractéristique de C qui est :

$$P_C(\lambda) = -\lambda^3 + 2\lambda^2 + 16\lambda - 19$$

- Enfin on peut également obtenir **la trace d'une matrice** par la commande :

```
1 np.trace(C)
```

8.3. Les matrices et les images avec Opencv

La librairie Matplotlib permet d'afficher des images. Nous allons dans cette section voir comment il est possible de manipuler des images avec Python en utilisant la librairie OpenCV. Pour installer OpenCV, parmi les multiples façon de procéder on pourra ouvrir Spyder et taper la commande suivante directement dans la console :

```
pip install opencv-python
```

Dans cette section nous allons décrire des lignes de commande qui permettent d'importer une image (format *jpg* ou *png*) grâce à Python sous forme de matrice (très grande matrice en général).

- On commence par enregistrer l'image à importer dans le dossier où se trouve votre fichier python (il est conseillé de ne pas prendre une image de trop grande qualité pour la santé de votre ordinateur...). Par exemple l'image d'une fraise dont le nom de fichier est : fraise.jpg .
- On importe alors l'image grâce à la librairie OpenCV :

```
1 import cv2
2 image=cv2.imread('fraise.jpg')
```

image est composée de trois matrices, chacune représentant les intensités des couleurs primaires RVB (rouge, vert, bleu). Vous pouvez voir dans l'explorateur de variable qu'une image est en réalité la donnée de trois matrices : une matrice pour l'intensité des rouge, une pour le vert et la dernière couche pour le bleu. Chacune de ces intensités est codée par un « entier » (type uint8) compris entre 0 et 255 (i.e. sur 2^8 bits).

- On peut soit travailler avec la matrice « triple » *image* soit si la couleur nous importe peu convertir l'image en niveau de gris. Pour cela on peut faire la moyenne des intensités des trois couches de couleurs « primaires » pour obtenir une seule matrice :

```
1 M = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

La matrice M est alors une version matricielle de notre image importée. Une commande alternative est la suivante :

```
1 M=cv2.imread('fraise.jpg',0)
```

qui importe directement l'image en niveau de gris.

- On peut afficher l'image en niveaux de gris en utilisant la commande suivante :

```
1 cv2.imshow('fraise',M)
```

Les niveaux de gris sont incarnés par des nombres de 0 à 255.

- On peut également enregistrer la matrice sous forme d'un fichier image (ici jpg) :

```
1 cv2.imwrite('fraise_gris.jpg', M)
```

Dans les commandes qu'il est de bon ton de connaître on citera :

- `cv2.convertScaleAbs`, qui permet de convertir une matrice qui contient par exemple des flottants en entier de type *uint8*. C'est dans le format *uint8* qu'il faudra afficher vos images sous peine de voir une image toute blanche ou toute noire suivant le cas...
- et `cv2.resize` : la commande `resize` permet de redimensionner les images.

8.3.1. Produire une sortie vidéo avec OpenCV

Dans cette sous-section nous allons voir comment fonctionne la commande : `cv2.VideoWriter` de la librairie OpenCV (ainsi que d'autres commandes de cette librairie au passage). Nous donnerons comme exemple d'application de notre code un résultat que nous démontrerons en A3 : **la compression d'image grâce au théorème d'Eckart-Young-Mirsky**.

La commande `cv2.VideoWriter` permet de compiler une liste d'images de même format en une vidéo.

On rappelle que l'on peut charger (=importer une image sous forme de données RGB) une image grâce à OpenCV :

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import cv2
4 M=cv2.imread('toulouse.jpeg') #l'image 'toulouse.jpeg' doit être dans le répertoire de travail
5 cv2.imshow('Toulouse',M) #affiche l'image
```

On rappelle également que l'affichage de l'image importée sous forme de données M s'effectue avec la commande suivante :

```
1 cv2.imshow('Toulouse',M) #affiche l'image dans une fenêtre de titre 'Toulouse'
```

qui produit la sortie suivante :



On définit une fonction `compression2(M,k)` où M est une matrice, k un entier et dont la sortie est une matrice de rang k et qui renvoie également le taux de compression tc :

```
1 def compression2(M,k) :
2     U,S,VT=np.linalg.svd(M)
3     m,n=M.shape
4     tc=1-(k*(m+n+1)/(m*n))
5     N=np.zeros(np.shape(M))
6     for i in range(0,k) :
7         Ui=U[:,i][np.newaxis].T
8         Vi=VT[i,:][np.newaxis]
9         N=N+S[i]*Ui @ Vi
10    N=cv2.cvtColor(N, cv2.COLOR_RGB2GRAY) #pour convertir la matrice de flottant N en matrice de type uint8
11    return N,tc
```

Nous démontrerons le pourquoi de ces calculs et formules dans le module Ma313 en troisième année.

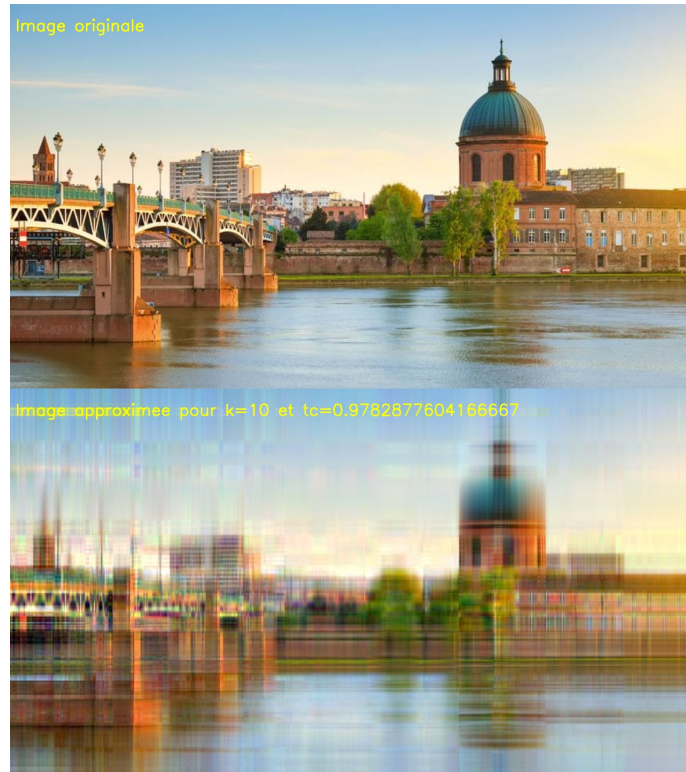
On génère ensuite une liste vide que l'on remplira d'images :

```
1 images=[] #liste vide
2 for k in range(0,80,2) : #on choisit un pas de 2
3     N=np.zeros(M.shape)
4     for i in range(0,p) :
5         N[:,i]=compression2(M[:,i],k)[0]
6         tc=compression2(M[:,i],k)[1]
7     N=cv2.cvtColor(N, cv2.COLOR_RGB2GRAY)
8     cv2.putText(M,'Image originale',(10,50),cv2.FONT_HERSHEY_SIMPLEX,1,(0,255,255),2)
9     cv2.putText(N,'Image approximee pour k={} et tc={}'.format(k,tc),(10,50),cv2.FONT_HERSHEY_SIMPLEX,1,(0,255,255),2)
10    aff= cv2.hconcat([M,N]) #aff= cv2.hconcat([M,N]) pour une concatenation horizontale
11    images.append(aff)
```

On peut enregistrer n'importe quelle image de la liste de sortie, par exemple pour l'image 5 :

```
1 cv2.imwrite('sortie_toulouse.jpg', images[5])
```

qui produit la sortie suivante :



Dans le code précédent on utilise la commande `cv2.putText` pour insérer un texte dans l'image.

Les instructions de cette commande sont de la forme :

```
1 cv2.putText(image, texte, position d origine du texte, police, taille police, couleur texte, épaisseur)
```

Attention : Dans OpenCV une position (10,50) signifie : le pixel de la colonne 10 et de la ligne 50. La convention (ligne,colonne) pour les indices des matrices est inversée dans OpenCV en (colonne,ligne).

On génère alors une vidéo à l'aide des commandes suivantes :

```
1 l,c,p=aff.shape
2 sortie = cv2.VideoWriter('Compression_imagesSVDvertical.avi',cv2.VideoWriter_fourcc(*'MJPG'),1., (c,l))
3
4 for i in range(1,len(images)):
5     sortie.write(images[i])
6 sortie.release() #supprime les données de sorties
7 cv2.destroyAllWindows() #ferme les fenêtres d'affichage si besoin
```

La commande `cv2.VideoWriter` permet de générer un type d'objet particulier qui permettra d'assembler une liste d'image en une vidéo..

Les instructions de cette commande sont de la forme :

```
1 cv2.VideoWriter(titre video autre format *.avi, codec de compression, nombre d'image par seconde (fps), taille
    image de la vidéo)
```

Remarque : On retrouve la convention inversée de OpenCV : (c, l) (et non classiquement (l, c) pour les matrices).

Un codec vidéo permet entre autre chose de produire une vidéo compressée. On pourra installer une librairie contenant ces « fameux » codecs en utilisant la commande suivante dans la console de Spyder (ou dans le Prompt d'Anaconda) :

```
pip install ffmpeg-python
```

8.3.2. Utilisation de l'apprentissage profond

La librairie OpenCV contient aussi des éléments d'apprentissages artificiels. Nous allons donner un exemple simple qui récupère la sortie vidéo de votre webcam et détecte les visages et les yeux présents. Pour cela créer un dossier dans lequel vous mettrez votre programme Python ainsi que les fichiers : `haarcascade_eye.xml` et `haarcascade_frontalface_default.xml`, téléchargeable sur <https://github.com/opencv/opencv/tree/master/data/haarcascades>. Le principe des classifieurs sera détaillé en troisième année (pour ceux qui choisiront l'option système). Nous donnons en exemple le programme suivant qui détecte les visages et les yeux à partir de votre webcam.

```

1 import cv2
2 import numpy as np
3
4
5 eyecascade = cv2.CascadeClassifier('haarcascade_eye.xml')
6 facecascade=cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
7
8 #Debut de la capture vidéo
9 video=cv2.VideoCapture(0)
10 compteur=0
11 while True:
12     compteur=compteur+1
13     check, frame = video.read()
14     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
15     l,c=gray.shape
16     faces = facecascade.detectMultiScale(gray, 1.3, 5)
17     for (x,y,w,h) in faces:
18         if h > 0 and w > 0:
19             cv2.rectangle(frame,(x,y),(x+w,y+h),(255,0,0),2)
20             visage_gray = gray[y:y+h, x:x+w]
21             eyes = eyecascade.detectMultiScale(visage_gray)
22     for (ex,ey,ew,eh) in eyes:
23         if ex>0 and ey>0 :
24             cv2.rectangle(frame,(x+ex,y+ey),(x+ex+ew,y+ey+eh),(0,255,0),2)
25
26     cv2.imshow("Webcam", frame)
27     key=cv2.waitKey(1)
28     #permet d'interrompre la vidéo par l'appuie de la touche 'q' du clavier.
29     if key== ord('q') :
30         break
31 print(compteur)
32 video.release()
33 cv2.destroyAllWindows

```

Exercice 13. Essayez de comprendre (en vous aidant de l'explorateur de variable et de l'aide d'OpenCV) les commandes précédentes.

Pour ceux intéressés par l'utilisation de la librairie OpenCV on renvoie à l'ouvrage très simple et illustrer d'exemple : *OpenCV 3.x with Python by example* de Gabriel Garrido et Prateek Joshi.

Index

A

addition par blocs, 62
analyse en composantes principales, 24
Analyse procustéenne (linéaire), 37
application $\mathbb{K}$ -linéaire, 67
approximation de Schmidt, 18

B

blocs de la partition, 62

C

centrage, 25
cercle des corrélations, 27
cofacteurs, 66
complexité (information), 16
contribution relative (individu), 28
contribution relative (variable), 28
coordonnées factorielles des variables, 27
couple propre, 64

D

Décomposition en valeurs singulières (SVD) , 12
décomposition en valeurs singulières réduite, 13
décomposition en valeurs singulières, 9
déterminant, 66
déterminant, 66

F

fonction quadratique (quadrique), 69
forme bilinéaire antisymétrique, 69
forme bilinéaire symétrique, 69
forme bilinéaire, 69

I

image (matrice), 59
inertie projeté, 27
inertie totale, 27
information, 15

L

la fougère de Barnsley, 60
Le pseudo-inverse, 43

M

matrice à bande, 59
matrice antisymétrique, 63
matrice associé à une application linéaire, 68
matrice de confusion, 53

matrice de rang plein en colonnes, 60
matrice de rang plein en lignes, 60
matrice diagonale, 59
matrice identité, 63
matrice inversible (non-singulière), 63
matrice nulle, 58
matrice orthogonale, 7, 63
matrice symétrique, 63
matrice triangulaire inférieure, 59
matrice triangulaire supérieure, 59
matrice, 57
matrices par blocs, 61
mineurs, 66
moindres carrés totaux, 45
multiplication par blocs, 62

N

norme de Frobenius, 7
noyau (matrice), 59

O

oordonnées factorielles des individus, 27
opérateur vec, 71
orthogonalité entre matrices, 8

P

polynôme caractéristique, 64
produit (matrice), 58
produit de Kronecker, 71
produit scalaire euclidien (version matricielle), 58
produit scalaire, 69

Q

qualité de représentation (individu), 28

R

rang, 59
rayon spectral, 64
redondance (information), 15
robustesse (information), 17

S

Solution au sens des moindres carrés, 43
somme (matrice), 58
Sous-multiplicativité de la norme, 8
structuration (information), 16
symétrique définie positive, 63
symétrique semi-définie positive, 63

symbole de Kronecker, 63

T

taille de la partition, 62

Théorème d'Eckart-Young-Mirsky, 18

Théorème d'entrelacement, 16

Théorème de Cayley-Hamilton, 64

Théorème de la décomposition de Jordan, 65

Théorème du rang, 60

trace, 66

transconjugué, 65

transposée, 57

transposition par blocs, 62

V

valeur propre, 64

valeurs singulières, 11

variables qualitatives, 23

variables quantitatives, 23

vecteur propre, 64

Bibliographie

Bibliographie

- [1] Luca Amodei and Jean-Pierre Dedieu. *Analyse numérique matricielle : cours et exercices corrigés*. Dunod, 2008.
- [2] Brigitte Escofier and Jérôme Pagès. *Analyses factorielles simples et multiples-5e éd. : cours et études de cas*. Dunod, 2023.
- [3] Gene H Golub and Charles F Van Loan. *Matrix computations*. JHU press, 2013.
- [4] Frédéric Rotella and Irène Zambettakis. *Traitement des systèmes linéaires*. Ellipses, 2015.